

زبان برنامه نویسی سلام

راهنمای کامل آموزشی

از نخستین برنامه تا توسعه حرفه‌ای

زبانی دوزبانه برای همه:
کودکان، دانش‌آموزان، دانشجویان و برنامه‌نویسان حرفه‌ای

تیم زبان سلام

ویرایش نخست

این کتاب راهنمای رسمی آموزش زبان برنامه‌نویسی سلام است. همه نمونه‌کدهای این کتاب با کامپایلر رسمی salam آزموده شده‌اند و قابل اجرا هستند.

کلیه حقوق برای تیم زبان سلام محفوظ است.

چکیده

سلام یک زبان برنامه‌نویسی نوین، ساده و دوزبانه است که به شما اجازه می‌دهد برنامه‌های خود را هم به فارسی و هم به انگلیسی بنویسید. در سلام می‌توانید واژگان کلیدی مانند تابع، اگر، برای و بازگشت را به فارسی به کار ببرید، یا اگر بخواهید همان برنامه را با if، for و ret به انگلیسی بنویسید. هر دو شکل، به یک برنامه یکسان تبدیل می‌شوند و با همان سرعت اجرا می‌شوند.

چرا سلام دوزبانه است؟

سال‌هاست که آموزش برنامه‌نویسی برای فارسی‌زبانان با یک مانع پنهان همراه بوده است: زبان انگلیسی. کودکی که هنوز انگلیسی نیاموخته، یا دانش‌آموزی که می‌خواهد نخستین گام‌ها را بردارد، پیش از آنکه با منطق برنامه‌نویسی آشنا شود، ناچار است با واژگان بیگانه دست‌وپنجه نرم کند. سلام این مانع را برمی‌دارد. شما می‌توانید با زبان مادری خود بیندیشید و بنویسید:

۱	تابع اصلی:
۲	چاپ "سلام، دنیا!"
۳	پایان

در همان حال، سلام پلی به جهان گسترده برنامه‌نویسی بین‌المللی نیز هست. چون هر واژه فارسی یک معادل دقیق انگلیسی دارد، خواننده به تدریج و بدون فشار، واژگان استاندارد جهانی را نیز می‌آموزد و آماده می‌شود تا در آینده با هر زبان دیگری کار کند.

این کتاب برای چه کسانی است؟

این کتاب کوشیده است همه را در بر بگیرد:

- **کودکان زیر ده سال:** با مثال‌های بازی‌گونه مانند کشیدنِ مثلثِ ستاره، جدولِ ضرب و بازیِ فیزباز، گام‌به‌گام و با زبانی ساده.
- **دانش‌آموزان:** برای یادگیریِ مفاهیمِ پایهٔ برنامه‌نویسی، الگوریتم‌ها و حلِ مسئله.
- **دانشجویان:** برای درکِ ساختارهای داده، شیء‌گرایی، کلی‌سازی و طراحیِ نرم‌افزار.
- **برنامه‌نویسان حرفه‌ای:** برای کار با ویژگی‌های پیشرفته مانند واسط‌ها، بستارها، هم‌روندی، ارتباط با کدِ خارجی و توسعهٔ وب.

چگونه این کتاب را بخوانیم؟

اگر تازه‌کار هستید، کتاب را از فصلِ نخست و به ترتیب بخوانید؛ هر فصل بر فصلِ پیش استوار است. اگر تجربه‌ای پیشین دارید، می‌توانید مستقیم به فصلِ موردِ نظر بروید. در سراسر کتاب با این نشانه‌ها روبه‌رو می‌شوید:

یادداشت

نکته‌ای برای روشن‌تر شدنِ مطلب یا یادآوریِ چیزی مهم.

نکته

ترفند یا روشی که کارِ شما را آسان‌تر یا برنامه‌تان را بهتر می‌کند.

هشدار

هشدار دربارهٔ اشتباهی رایج که بهتر است از آن دوری کنید.

نگاهی به درون (اختیاری، برای خوانندگان پیشرفته)

بخشی ژرف‌تر دربارهٔ آنکه چرا و چگونه چیزی در درون زبان کار می‌کند. خواندن این بخش‌ها اختیاری است؛ اگر تازه‌کارید می‌توانید از آن‌ها بگذرید و بعدها بازگردید.

در پایان هر فصل، چند تمرین آمده است. پاسخ همهٔ تمرین‌ها در پیوست انتهای کتاب گردآوری شده است؛ اما پیش از دیدن پاسخ، خودتان بکوشید. برنامه‌نویسی با انجام دادن آموخته می‌شود، نه فقط با خواندن.

به دنیای سلام خوش آمدید. بیایید نخستین برنامه را با هم بنویسیم.

فهرست مطالب

۳	چکیده
۱	۱ شروع کار
۱	۱.۱ برنامه‌نویسی یعنی چه؟
۲	۲.۱ کامپایلر چیست و چرا به آن نیاز داریم؟
۲	۳.۱ آماده‌سازی کامپایلر
۳	۴.۱ نخستین برنامه: سلام، دنیا!
۴	۱.۴.۱ اجرای برنامه
۴	۵.۱ کالبدشکافی نخستین برنامه
۶	۶.۱ همان برنامه به انگلیسی
۷	۷.۱ چاپ با خط و بدون خط
۷	۸.۱ خلاصه فصل
۸	۹.۱ تمرین‌ها
۹	۲ متغیرها و انواع داده
۹	۱.۲ متغیر؛ جعبه‌ای با نام
۱۰	۲.۲ چهار شیوه ساختن متغیر
۱۰	۱.۲.۲ متغیر: جعبه‌ای که می‌توان عوضش کرد
۱۰	۲.۲.۲ ثابت: جعبه‌ای که قفل است
۱۰	۳.۲.۲ اعلان ساده: ثابت اما بی‌واژه
۱۱	۴.۲.۲ خودکار: بگذارید سلام نوع را حدس بزنند
۱۲	۳.۲ انواع پایه داده
۱۲	۱.۳.۲ اعداد درست (صحیح)
۱۲	۲.۳.۲ اعداد اعشاری
۱۳	۳.۳.۲ متن (رشته)
۱۳	۴.۳.۲ مقادیر منطقی (درست/نادرست)

۱۴	۵.۳.۲ کاراکتر (تک‌نویسه)
۱۴	۴.۲ جدول کامل انواع پایه
۱۵	۵.۲ تبدیل نوع با «بعنوان»
۱۵	۶.۲ قاعده‌های نام‌گذاری
۱۶	۷.۲ خلاصه فصل
۱۶	۸.۲ تمرین‌ها
۱۸	۳ عملگرها و عبارت‌ها
۱۸	۱.۳ عبارت چیست؟
۱۸	۲.۳ عملگرهای حسابی
۱۹	۱.۲.۳ تقسیم صحیح در برابر تقسیم اعشاری
۲۰	۲.۲.۳ باقی‌مانده (%) : قهرمان پنهان
۲۰	۳.۳ عملگرهای مقایسه‌ای
۲۱	۴.۳ عملگرهای منطقی
۲۲	۵.۳ پیوند رشته‌ها
۲۳	۶.۳ ترتیب اولویت عملگرها
۲۴	۷.۳ عملگرها روی انواع خودمان
۲۵	۸.۳ خلاصه فصل
۲۵	۹.۳ تمرین‌ها
۲۶	۴ شرط‌ها و تصمیم‌گیری
۲۶	۱.۴ اگر: نخستین تصمیم
۲۷	۲.۴ وگرنه: راه دوم
۲۷	۳.۴ وگرنه اگر: چند راه
۲۹	۴.۴ شرط در یک خط
۳۰	۵.۴ شرط‌های تودرتو
۳۰	۶.۴ مثال کامل: بازی حدس ساده
۳۱	۷.۴ خلاصه فصل
۳۲	۸.۴ تمرین‌ها
۳۳	۵ حلقه‌ها
۳۳	۱.۵ تاوقتی: تا زمانی که شرط برقرار است
۳۴	۲.۵ برای: حلقه شمارنده‌دار
۳۵	۳.۵ شکل‌کشی با حلقه‌های تودرتو
۳۶	۴.۵ تکرار: ساده‌ترین راه تکرار
۳۹	۵.۵ بشکن و ادامه: کنترل دقیق‌تر حلقه

۴۰		مثال کامل: بازی فیزباز	۶.۵
۴۰		خلاصه فصل	۷.۵
۴۱		تمرین‌ها	۸.۵
۴۲		۶ توابع	
۴۲		تابع چیست و چرا به آن نیاز داریم؟	۱.۶
۴۲		ساختن نخستین تابع	۲.۶
۴۴		پارامترها: ورودی تابع	۳.۶
۴۴		بازگشت: خروجی تابع	۴.۶
۴۴		۱.۴.۶ توابع بدون خروجی	
۴۵		بازگشتی: تابعی که خودش را صدا می‌زند	۵.۶
۴۶		ترتیب تعریف توابع	۶.۶
۴۶		ترکیب توابع	۷.۶
۴۷		خلاصه فصل	۸.۶
۴۷		تمرین‌ها	۹.۶
۴۸		۷ آرایه‌ها و بردارها	
۴۸		آرایه: ردیفی از جعبه‌ها با اندازه ثابت	۱.۷
۴۹		۱.۱.۷ دسترسی به عناصر با اندیس	
۴۹		۲.۱.۷ طول آرایه با len	
۵۰		۲.۷ بردار: آرایه‌ای که رشد می‌کند	
۵۱		۱.۲.۷ همان بردار به انگلیسی	
۵۱		۲.۲.۷ بردار به مثابه پشته	
۵۲		۳.۷ آرایه یا بردار؟ کدام را برگزینیم؟	
۵۳		خلاصه فصل	۴.۷
۵۳		تمرین‌ها	۵.۷
۵۴		۸ رشته‌ها و کاراکترها	
۵۴		۱.۸ یادآوری: رشته چیست؟	
۵۴		۲.۸ چسباندن رشته‌ها	
۵۵		۳.۸ پیمایش نویسه به نویسه	
۵۶		۴.۸ ابزارهای آماده کتابخانه رشته	
۵۷		۵.۸ دنباله‌های گریز	
۵۸		۶.۸ کاراکتر در برابر رشته	
۵۸		۷.۸ مثال کامل: شمارش واژه‌ها	
۵۹		۸.۸ خلاصه فصل	

۵۹	۹.۸ تمرین‌ها
۶۰	۹ ساختارها و شمارشی‌ها
۶۰	۱.۹ ساختار: ساختن نوع دلخواه خود
۶۱	۱.۱.۹ میدان‌ها (fields)
۶۱	۲.۱.۹ ساختن یک نمونه
۶۲	۳.۱.۹ متدها: توابع درون ساختار
۶۲	۲.۹ تغییر داده‌ها از درون متد
۶۳	۳.۹ مثال کامل: حساب بانکی
۶۴	۴.۹ عملگرها برای ساختارها
۶۵	۵.۹ شمارشی: مجموعه‌ای از نام‌های ثابت
۶۵	۱.۵.۹ شکل یک خطی و کاربرد واقعی
۶۶	۶.۹ خلاصه فصل
۶۷	۷.۹ تمرین‌ها
۶۸	۱۰ واسط‌ها و چندریختی
۶۸	۱.۱۰ مسئله: رفتار مشترک، پیاده‌سازی متفاوت
۶۸	۲.۱۰ تعریف و پیاده‌سازی واسط
۶۹	۱.۲.۱۰ اعلان واسط
۷۰	۲.۲.۱۰ پیاده‌سازی واسط
۷۰	۳.۲.۱۰ استفاده همگانی با dyn
۷۱	۳.۱۰ چرا واسط‌ها ارزشمندند؟
۷۱	۴.۱۰ واسط در برابر کلی‌سازی
۷۱	۵.۱۰ خلاصه فصل
۷۲	۶.۱۰ تمرین‌ها
۷۳	۱۱ کلی‌سازی
۷۳	۱.۱۱ مسئله: تکرار یک منطق برای انواع گوناگون
۷۴	۲.۱۱ راه حل: پارامتر نوع
۷۵	۳.۱۱ ساختارهای کلی
۷۵	۴.۱۱ کلی‌سازی با چند پارامتر نوع
۷۶	۵.۱۱ چرا کلی‌سازی سریع است؟
۷۶	۶.۱۱ کلی‌سازی در برابر واسط: کدام را برگزینیم؟
۷۷	۷.۱۱ خلاصه فصل
۷۷	۸.۱۱ تمرین‌ها

۷۸	۱۲ بستارها و عبارتهای لامبدا
۷۸	۱.۱۲ تابع به مثابه مقدار
۷۹	۱.۱.۱۲ نوع تابع
۷۹	۲.۱.۱۲ عبارت لامبدا
۷۹	۳.۱.۱۲ تابعی که تابع می گیرد
۷۹	۲.۱۲ بستار: لامبدايي که محیطش را به خاطر می سپارد
۸۰	۳.۱۲ بستار با چند دستور
۸۲	۴.۱۲ خلاصه فصل
۸۲	۵.۱۲ تمرین ها
۸۳	۱۳ مدیریت خطا و اجرای به تعویق افتاده
۸۳	۱.۱۳ تعلیق: کاری که حتماً باید انجام شود
۸۴	۲.۱۳ چرا تعلیق این قدر مفید است؟
۸۵	۳.۱۳ چند تعلیق و ترتیب اجرا
۸۵	۴.۱۳ مدیریت خطا با مقدار بازگشتی
۸۷	۵.۱۳ بررسی مرزها و اجرای امن
۸۷	۶.۱۳ ترکیب تعلیق و مدیریت خطا
۸۸	۷.۱۳ خلاصه فصل
۸۸	۸.۱۳ تمرین ها
۸۹	۱۴ ماژول ها و بسته ها
۸۹	۱.۱۴ چرا برنامه را بشکنیم؟
۸۹	۲.۱۴ وارد کردن کتابخانه های آماده
۹۰	۳.۱۴ ساختن ماژول خودمان
۹۰	۱.۳.۱۴ عمومی در برابر خصوصی
۹۱	۲.۳.۱۴ استفاده از ماژول خودمان
۹۲	۴.۱۴ دوزبانه سازی ماژول های خودمان
۹۳	۵.۱۴ ساختار یک پروژه چندفایلی
۹۳	۶.۱۴ خلاصه فصل
۹۴	۷.۱۴ تمرین ها
۹۵	۱۵ کتابخانه استاندارد
۹۵	۱.۱۵ ریاضی
۹۶	۲.۱۵ رشته
۹۷	۳.۱۵ نگاشت: واژه نامه کلید-مقدار
۹۸	۱.۳.۱۵ پیمایش یک نگاشت

۹۹	۴.۱۵ دیگر ماژول‌های کاربردی
۱۰۰	۱.۴.۱۵ نمونه: عدد تصادفی
۱۰۱	۵.۱۵ خلاصه فصل
۱۰۱	۶.۱۵ تمرین‌ها
۱۰۲	۱۶ کار با فایل و سیستم
۱۰۲	۱.۱۶ نوشتن و خواندن فایل
۱۰۳	۲.۱۶ آرگومان‌های خط فرمان
۱۰۴	۳.۱۶ متغیرهای محیطی
۱۰۵	۴.۱۶ تشخیص سیستم عامل در زمان کامپایل
۱۰۶	۵.۱۶ خلاصه فصل
۱۰۷	۶.۱۶ تمرین‌ها
۱۰۸	۱۷ همروندی
۱۰۸	۱.۱۷ نخ چیست؟
۱۰۹	۲.۱۷ مسئله مشترک: داده‌ای که چند نخ به آن دست می‌زنند
۱۱۰	۱.۲.۱۷ تقسیم کار
۱۱۰	۲.۲.۱۷ محافظت با قفل
۱۱۱	۳.۲.۱۷ انتظار با گروه انتظار
۱۱۱	۳.۱۷ خطرهای همروندی
۱۱۲	۴.۱۷ خلاصه فصل
۱۱۲	۵.۱۷ تمرین‌ها
۱۱۳	۱۸ ارتباط با کد خارجی
۱۱۳	۱.۱۸ فراخوانی یک تابع خارجی
۱۱۴	۲.۱۸ همان مفهوم به فارسی
۱۱۴	۳.۱۸ پیوند به یک کتابخانه
۱۱۵	۴.۱۸ کاربرد عملی: خواندن متغیر محیطی از C
۱۱۶	۵.۱۸ خلاصه فصل
۱۱۷	۶.۱۸ تمرین‌ها
۱۱۸	۱۹ چارچوب وب و سامانه چیدمان
۱۱۸	۱.۱۹ نخستین صفحه وب
۱۱۹	۲.۱۹ عناصرها و ویژگی‌ها
۱۲۰	۳.۱۹ عناصرهای پرکاربرد
۱۲۰	۱.۳.۱۹ فهرست

۱۲۱		۲.۳.۱۹ دکه
۱۲۱		۴.۱۹ ساختن یک فرم
۱۲۲		۵.۱۹ سبک‌دهی و ظاهر
۱۲۲		۶.۱۹ خلاصه فصل
۱۲۳		۷.۱۹ تمرین‌ها
۱۲۴		۲۰ نکات پیشرفته و بهترین روش‌ها
۱۲۴		۱.۲۰ پاس دادن به ارجاع
۱۲۵		۲.۲۰ قالب‌بندی خودکار کد
۱۲۶		۳.۲۰ آزمون‌نویسی
۱۲۷		۴.۲۰ اشکال‌زدایی
۱۲۷		۵.۲۰ کارایی و حافظه
۱۲۸		۶.۲۰ اصول کد خوب
۱۲۹		۷.۲۰ راه پیش رو
۱۲۹		۸.۲۰ خلاصه فصل
۱۳۰		۹.۲۰ تمرین‌ها
۱۳۱		آ فهرست کامل واژگان کلیدی
۱۳۱		۱.آ واژگان کلیدی
۱۳۳		۲.آ انواع داده
۱۳۳		۳.آ ماژول‌های کتابخانه استاندارد
۱۳۴		۴.آ فرمان‌های کامپایلر
۱۳۵		۵.آ راهنماهای فایل
۱۳۶		ب دستور زبان (گرامر)
۱۳۶		۱.ب ساختار کلی برنامه
۱۳۶		۲.ب اعلان متغیر
۱۳۷		۳.ب تابع
۱۳۷		۴.ب شرط
۱۳۷		۵.ب حلقه‌ها
۱۳۸		۶.ب ساختار و شمارشی
۱۳۹		۷.ب واسط و چندریختی
۱۳۹		۸.ب کلی‌سازی
۱۳۹		۹.ب لامبدا و بستار
۱۴۰		۱۰.ب ماژول و دسترسی
۱۴۰		۱۱.ب اجرای به تعویق افتاده و خارجی

۱۴۱	ب.۱۲ چیدمان (وب)
۱۴۱	ب.۱۳ قاعده‌های کلی

۱۴۲	ج برگه مرجع سریع
۱۴۲	ج.۱ اسکلت برنامه
۱۴۲	ج.۲ متغیر و چاپ
۱۴۳	ج.۳ عملگرها
۱۴۳	ج.۴ شرط
۱۴۳	ج.۵ حلقه‌ها
۱۴۳	ج.۶ تابع
۱۴۴	ج.۷ آرایه و بردار
۱۴۴	ج.۸ رشته
۱۴۴	ج.۹ نگاشت
۱۴۵	ج.۱۰ ساختار و شمارشی
۱۴۵	ج.۱۱ واسط و کلی‌سازی
۱۴۵	ج.۱۲ لامبدا و تعلیق
۱۴۶	ج.۱۳ مازول و خارجی
۱۴۶	ج.۱۴ فرمان‌های پرکاربرد

۱۴۷	د پاسخ تمرین‌ها
۱۴۷	فصل ۱
۱۴۸	فصل ۲
۱۴۸	فصل ۳
۱۴۹	فصل ۴
۱۴۹	فصل ۵
۱۵۰	فصل ۶
۱۵۱	فصل ۷
۱۵۲	فصل ۸
۱۵۲	فصل ۹
۱۵۳	فصل ۱۰
۱۵۳	فصل ۱۱
۱۵۴	فصل ۱۲
۱۵۴	فصل ۱۳
۱۵۵	فصل ۱۴
۱۵۵	فصل ۱۵
۱۵۶	فصل ۱۶

۱۵۶		فصل ۱۷
۱۵۷		فصل ۱۸
۱۵۷		فصل ۱۹
۱۵۷		فصل ۲۰

فصل ۱

شروع کار

در این فصل نخستین گام‌ها را برمی‌داریم: کامپایلر سلام را آماده می‌کنیم، نخستین برنامه خود را می‌نویسیم و آن را اجرا می‌کنیم. نگران نباشید؛ اگر تا امروز حتی یک خط برنامه ننوشته‌اید، این فصل دست شما را می‌گیرد و قدم به قدم پیش می‌برد.

۱.۱ برنامه‌نویسی یعنی چه؟

رایانه ماشینی است بسیار سریع اما بسیار ساده‌دل. خودش نمی‌داند چه باید بکند؛ باید به او دستور بدهیم. مجموعه‌ای از دستورها که به ترتیب نوشته شوند تا رایانه کاری انجام دهد، یک **برنامه** نام دارد و زبانی که با آن این دستورها را می‌نویسیم، یک **زبان برنامه‌نویسی** است.

سلام یکی از این زبان‌هاست؛ با این ویژگی ممتاز که می‌توانید دستورها را به زبان فارسی بنویسید. برای نمونه، جمله «چیزی را چاپ کن» در سلام چنین نوشته می‌شود: "سلام" چاپ.

یادداشت

رایانه دقیقاً همان کاری را می‌کند که به او می‌گویید — نه بیشتر و نه کمتر. پس اگر برنامه آن‌طور که می‌خواستید کار نکرد، احتمالاً اشتباه از دستوری است که داده‌اید، نه از رایانه. این موضوع کاملاً طبیعی است و حتی برنامه‌نویسان حرفه‌ای هم هر روز با آن روبه‌رو می‌شوند.

۲.۱ کامپایلر چیست و چرا به آن نیاز داریم؟

برنامه‌ای که شما می‌نویسید برای انسان خواناست، اما رایانه آن را مستقیماً نمی‌فهمد. به برنامه‌ای نیاز داریم که نوشته‌ما را به زبانی که رایانه می‌فهمد ترجمه کند. این مترجم، **کامپایلر** نام دارد. کامپایلر زبان سلام `salam` نامیده می‌شود.

`salam` کارهای زیادی برای ما انجام می‌دهد: متن برنامه را می‌خواند، درستی آن را واری می‌کند، خطاها را گزارش می‌دهد و در پایان یک فایل اجرایی می‌سازد که می‌توانید آن را روی رایانه اجرا کنید.

نگاهی به درون: مسیر یک برنامه (اختیاری، برای خوانندگان پیشرفته)

وقتی `salam` برنامه شما را می‌سازد، آن را از چند مرحله می‌گذراند: تحلیل واژگانی (شکستن متن به نشانه‌ها)، تجزیه (ساختن درخت نحوی)، تحلیل معنایی (واری انواع و نام‌ها)، و سرانجام تولید کد که برنامه شما را به زبان C ترجمه می‌کند و سپس به فایل اجرایی تبدیل می‌نماید. در سراسر این کتاب گاه‌به‌گاه به این مراحل اشاره خواهیم کرد، اما برای نوشتن برنامه نیازی به دانستن جزئیات آن‌ها نیست.

۳.۱ آماده‌سازی کامپایلر

برای کار با سلام به کامپایلر `salam` نیاز دارید. این کامپایلر از روی کد منبع ساخته می‌شود. در پوشه اصلی پروژه سلام، فرمان زیر را اجرا کنید:

```
۱ sh tools/build-compiler.sh
```

پس از پایان، فایل اجرایی `salam` (یا در ویندوز `salam.exe`) در پوشه پروژه ساخته می‌شود. راه دیگر، ساخت با `CMake` است:

```
۱ cmake -B build
```

```
۲ cmake --build build
```


نکته

برای آنکه مطمئن شوید کامپایلر درست ساخته شده است، فرمان زیر را اجرا کنید تا شماره نسخه را ببینید:

۱ `salam -v`

یادداشت

در این کتاب فرض می‌کنیم `salam` در دسترس شماست؛ یعنی یا در پوشه جاری قرار دارد (آن را با `salam` صدا می‌زنید) یا در مسیر سیستم نصب شده است (آن را با `salam` صدا می‌زنید). برای سادگی، در سراسر کتاب `salam` می‌نویسیم.

۴.۱ نخستین برنامه: سلام، دنیا!

سنتی دیرینه در میان برنامه‌نویسان هست: نخستین برنامه‌ای که در هر زبان تازه می‌نویسند، برنامه‌ای است که عبارت «سلام، دنیا!» را نمایش می‌دهد. بیایید همین سنت را پاس بداریم.

فایلی به نام `hello.salam-01` بسازید و این چند خط را در آن بنویسید:

۱ // تابع اصلی در واقع نقطه شروع نرم افزار شماست.

۲ تابع اصلی:

۳ نام رشته = "سلام"

۴ چاپ "سلام"، نام

۵ چاپ "۲" + "۳"، " = 3 + 2

۶ پایان

یادداشت

پسوند همه فایل‌های برنامه سلام `salam` است. این پسوند به ابزارها می‌گوید که محتوای فایل، کد زبان سلام است.

۱.۴.۱ اجرای برنامه

ساده‌ترین راه برای دیدن نتیجه، استفاده از مفسر سلام است. در ترمینال بنویسید:

```
۱ salam exec 01-hello.salam
```

و خروجی چنین خواهد بود:

```
۱ سلام،
۲ سلام ۲
۳ + ۳ = 5
```

اگر می‌خواهید یک فایل اجرایی مستقل بسازید (که سریع‌تر اجرا می‌شود و برای انتشار برنامه مناسب است)، از فرمان `build` استفاده کنید:

```
۱ salam build 01-hello.salam --output=hello.exe
۲ ./hello.exe
```

نکته

تفاوت `exec` و `build` در چیست؟ `exec` برنامه را بی‌درنگ اجرا می‌کند و برای آزمایش و یادگیری عالی است. `build` یک فایل اجرایی می‌سازد که می‌توانید بارها و بدون نیاز به کامپایلر اجراش کنید و برای تحویل نهایی برنامه به کار می‌رود. در سراسر این کتاب، برای سادگی بیشتر از `exec` استفاده می‌کنیم.

۵.۱ کالبدشکافی نخستین برنامه

اکنون خط‌به‌خط ببینیم در این برنامه چه گذشت. هیچ بخشی را سرسری نگیرید؛ همین چند خط، دروازه ورود به همه مفاهیم بعدی است.

خط نخست – یک توضیح: هر خطی که با `//` آغاز شود، یک توضیح (comment) است و کامپایلر آن را نادیده می‌گیرد. توضیحات برای انسان نوشته می‌شوند، نه برای رایانه. شکل ویژه `!!` یک اطلاعات سرآمد (metadata) است که به ویرایشگرها و محیط‌های آزمایش می‌گوید این فایل یک برنامه اجرایی (app) با عنوان «سلام» است. برای اجرای برنامه ضروری نیست، اما عادت خوب است.

۱ تابع اصلی:

این خط می‌گوید: «یک تابع به نام اصلی را تعریف کن.» تابع، بسته‌ای از دستورهاست که نامی دارد. تابعی که نامش اصلی است، نقطه شروع هر برنامه سلام است؛ یعنی وقتی برنامه را اجرا می‌کنید، رایانه از اینجا کار را آغاز می‌کند. نشانه دونقطه (: در پایان این خط می‌گوید که بدنه تابع از خط بعد آغاز می‌شود.

یادداشت

نام اصلی در فارسی، دقیقاً معادل main در انگلیسی است. اگر همین برنامه را به انگلیسی بنویسید، خط نخست چنین می‌شود: `func main:`.

۱ نام رشته = "سلام"

اینجا یک متغیر به نام نام می‌سازیم. یک متغیر را مانند جعبه‌ای تصور کنید که چیزی در آن نگه می‌داریم. در این جعبه، متن "سلام" را گذاشته‌ایم. واژه رشته می‌گوید که نوع این جعبه از جنس متن است (در فصل بعد بیشتر درباره انواع خواهیم گفت). هر متنی را میان دو علامت نقل قول (") می‌نویسیم. دقت کنید که این خط با چند فاصله از سر سطر آغاز شده است. به این فاصله تورفتگی (indentation) می‌گوییم و نشان می‌دهد که این خط درون تابع اصلی قرار دارد.

۱ چاپ "سلام"، نام

واژه چاپ به رایانه می‌گوید چیزی را روی صفحه نمایش بدهد. اینجا دو چیز را چاپ می‌کنیم: نخست متن ثابت "سلام"، و سپس محتوای متغیر نام (که همان "سلام" است). این دو با کاما (,) از هم جدا شده‌اند و سلام میان آن‌ها یک فاصله می‌گذارد. به همین دلیل خروجی «سلام، سلام» می‌شود.

۱ چاپ ۲ + ۳ = ۵

سلام تنها متن چاپ نمی‌کند؛ می‌تواند حساب هم بکند. عبارت $2 + 3$ پیش از چاپ محاسبه می‌شود و نتیجه (یعنی ۵) چاپ می‌گردد. توجه کنید که $2 + 3 = 5$ درون نقل قول

است و پس یک متن ثابت است که بی‌کم‌وکاست چاپ می‌شود، اما $2 + 3$ بیرونِ نقل قول است و پس یک محاسبه است.

هشدار

به تفاوت ارقام فارسی (۲، ۳) و ارقام لاتین (2، 3) دقت کنید. درون یک متن نقل قول دار، هر رقمی فقط یک نویسه است و چاپ می‌شود؛ اما برای محاسبه، سلام از ارقام لاتین استفاده می‌کند. پس $2 + 3$ یک جمع واقعی است، اما اگر بنویسید " $۲ + ۳$ " تنها یک متن است و جمعی در کار نیست.

۱ پایان

واژه پایان به سلام می‌گوید که بدنه تابع اصلی به پایان رسید. هر تابعی که با : باز می‌شود، باید با پایان بسته شود. این قاعده‌ای است که در سراسر زبان تکرار می‌شود: بلوک‌ها با دونقطه باز و با پایان بسته می‌شوند.

۶.۱ همان برنامه به انگلیسی

یکی از زیبایی‌های سلام این است که هر برنامه را می‌توان به انگلیسی هم نوشت. این برنامه دقیقاً معادل انگلیسی نمونه بالاست:

```

۱ // Hello
۲ func main:
۳     name str = "Salam"
۴     println "Hello,", name
۵     println "2 + 3 =", 2 + 3
۶ end

```

می‌بینید که ساختار یکسان است؛ تنها واژگان تغییر کرده‌اند: تابع $\leftrightarrow$ func، اصلی $\leftrightarrow$ main، رشته $\leftrightarrow$ str، چاپ $\leftrightarrow$ println و پایان $\leftrightarrow$ end. جدول کامل این معادل‌ها را در پیوست پایان کتاب خواهید یافت.

یادداشت

خط LANG: fa // که در بسیاری از نمونه‌های این کتاب می‌بینید، به کامپایلر می‌گوید زبان این فایل فارسی است. اگر این خط را ننویسید، سلام پیش‌فرض را زبان انگلیسی می‌گیرد.

۷.۱ چاپ با خط و بدون خط

سلام دو شیوه اصلی برای نمایش خروجی دارد:

- چاپ (معادل `println`): چیزی را نمایش می‌دهد و سپس به خط بعد می‌رود.
- بنویس (معادل `print`): چیزی را نمایش می‌دهد اما به خط بعد نمی‌رود؛ یعنی نوشته بعدی در همان خط ادامه می‌یابد.

به این مثال دقت کنید:

۱	تابع اصلی:
۲	بنویس "سلام"
۳	بنویس "دنیا"
۴	چاپ
۵	چاپ "خط تازه"
۶	پایان

خروجی:

۱	سلام
۲	دنیا خط
۳	تازه

دو فرمان بنویس هر دو در یک خط نوشتند و سپس چاپ (که یک متن خالی را چاپ می‌کند) ما را به خط بعد برد. این ترفند ساده — نوشتن با بنویس و رفتن به خط بعد با چاپ — را در فصل حلقه‌ها برای کشیدن شکل‌ها بارها به کار خواهیم برد.

۸.۱ خلاصه فصل

در این فصل آموختیم:

- برنامه مجموعه‌ای از دستورهاست و کامپایلرِ salam آن را برای رایانه ترجمه می‌کند.
- هر برنامهٔ سلام از تابع اصلی آغاز می‌شود.
- بلوک‌ها با : باز و با پایان بسته می‌شوند.
- با چاپ و بنویس خروجی نمایش می‌دهیم.
- توضیحات با // نوشته می‌شوند و کامپایلر آن‌ها را نادیده می‌گیرد.
- هر برنامه را می‌توان به فارسی یا انگلیسی نوشت.

۹.۱ تمرین‌ها

- تمرین ۱.۱.** برنامه‌ای بنویسید که نام و سن شما را در دو خط جداگانه چاپ کند.
- تمرین ۲.۱.** برنامه‌ای بنویسید که حاصل $12 * 8$ را به همراه یک متن توضیحی چاپ کند؛ برای نمونه: $96 = 8 * 12$.
- تمرین ۳.۱.** برنامهٔ «سلام، دنیا!» را یک بار به فارسی و یک بار به انگلیسی بنویسید و هر دو را اجرا کنید. آیا خروجی یکسان است؟
- تمرین ۴.۱.** با استفاده از بنویس، نام و نام‌خانوادگی خود را در یک خط و با یک فاصله میانشان چاپ کنید.

فصل ۲

متغیرها و انواع داده

در فصل پیش با متغیر آشنا شدیم و آن را به جعبه‌ای تشبیه کردیم که چیزی در آن نکه می‌داریم. در این فصل ژرف‌تر می‌شویم: انواع گوناگون داده را می‌شناسیم، یاد می‌گیریم متغیر را چگونه بسازیم، و درمی‌یابیم چرا «نوع» هر متغیر اهمیت دارد.

۱.۲ متغیر؛ جعبه‌ای با نام

تصور کنید چند جعبه دارید و روی هر کدام برچسبی زده‌اید: «سن»، «قد»، «نام». در هر جعبه چیزی گذاشته‌اید. هر وقت به محتوای جعبه نیاز داشتید، کافی است نام روی برچسب را صدا بزنید. در برنامه‌نویسی، این جعبه‌ها همان **متغیرها** هستند. ساده‌ترین شکل ساختن یک متغیر در سلام چنین است:

۱	تابع اصلی:
۲	متغیر سن صحیح ۳۲ = 30
۳	چاپ سن
۴	پایان

این خط را چنین بخوانید: «یک متغیر به نام **سن** که نوعش **صحیح ۳۲** (عدد درست) است بساز و مقدار ۳۰ را در آن بگذار.» ساختار همیشه یکسان است:

متغیر نام نوع = مقدار

۲.۲ چهار شیوه ساختن متغیر

سلام چهار واژه کلیدی برای ساختن متغیر دارد و هر کدام معنایی متفاوت دارند. به این برنامه کوچک — که تقریباً همان نمونه موجود در پوشه مثال‌های سلام است — نگاه کنید:

۱	تابع اصلی:
۲	متغیر سن صحیح ۳۲ = 30
۳	ثابت سال تولد صحیح ۳۲ = 1994
۴	قد اعشاری ۶۴ = 75.1
۵	نام خودکار = "سارا"
۶	سن = سن + 1
۷	چاپ (نام، سن، سال تولد، قد)
۸	پایان

بیاید چهار شیوه را از هم تفکیک کنیم:

۱.۲.۲ متغیر: جعبه‌ای که می‌توان عوضش کرد

۱	متغیر سن صحیح ۳۲ = 30
---	-----------------------

واژه متغیر (معادل mut در انگلیسی) جعبه‌ای می‌سازد که محتوایش را بعداً می‌توان تغییر داد. به همین دلیل در خط 1 + سن = سن توانستیم مقدار سن را یکی زیاد کنیم؛ حالا سن برابر ۳۱ است.

۲.۲.۲ ثابت: جعبه‌ای که قفل است

۱	ثابت سال تولد صحیح ۳۲ = 1994
---	------------------------------

واژه ثابت (معادل const) جعبه‌ای می‌سازد که پس از پُر شدن، دیگر تغییر نمی‌کند. سال تولد کسی هیچ‌گاه عوض نمی‌شود، پس درست است که آن را ثابت بگیریم. اگر بکوشید مقدار یک ثابت را تغییر دهید، کامپایلر خطا می‌گیرد و به شما هشدار می‌دهد.

۳.۲.۲ اعلان ساده: ثابت اما بی‌واژه

- ۱ قد
- ۲ $اعشاری = 1.75$

اگر هیچ واژه‌ای (متغیر یا ثابت) ننویسید و مستقیم با نام آغاز کنید، یک متغیر تغییرناپذیر ساخته می‌شود — درست مانند ثابت. این شکل کوتاه و تمیز، شیوه پیش‌فرض سلام است: مگر آنکه واقعاً نیاز به تغییر داشته باشید، متغیر را تغییرناپذیر بگذارید.

۴.۲.۲ خودکار: بگذارید سلام نوع را حدس بزند

- ۱ نام
- ۲ $سارا = "سارا"$

گاهی نوشتن نوع، تکراری و آزاردهنده است؛ مگر معلوم نیست که "سارا" یک متن است؟ واژه خودکار (معادل auto) به سلام می‌گوید: «خودت از روی مقدار، نوع را تشخیص بده.» چون "سارا" میان نقل قول است، سلام درمی‌یابد که نوعش رشته است.

نکته

استفاده از خودکار هم کد را کوتاه‌تر می‌کند و هم خواناتر؛ اما در جاهایی که نوع روشن نیست (مثلاً یک عدد خام که نمی‌دانید درست است یا اعشاری)، نوشتن صریح نوع به خواننده کد شما کمک می‌کند.

یادداشت

به دو شیوه صدا زدن چاپ دقت کنید: در فصل پیش نوشتیم نام "سلام"، چاپ (بدون پرانتز) و اینجا نوشتیم (... سن، چاپ نام،) (با پرانتز). هر دو درست‌اند. پرانتز اختیاری است، اما بسیاری از برنامه‌نویسان آن را برای روشنی بیشتر می‌پسندند. در این کتاب هر دو شکل را خواهید دید.

۳.۲ انواع پایه داده

هر متغیر یک نوع دارد که می‌گوید چه جور داده‌ای را می‌تواند نگه دارد. نوع، هم به سلام کمک می‌کند که برنامه شما را درست بفهمد، و هم از بسیاری خطاها جلوگیری می‌کند. مهم‌ترین انواع پایه در سلام این‌ها هستند:

۱.۳.۲ اعداد درست (صحیح)

عددهای بدون اعشار — مانند ۰، ۷، ۱۵، و ۱۰۰ — را اعداد درست یا صحیح می‌نامیم. سلام چند نوع عدد درست دارد که در اندازه با هم فرق می‌کنند:

نام فارسی	نام انگلیسی	توضیح
صحیح	i32 / int	عدد درست معمول (۳۲ بیتی)
صحیح ۳۲	i32	عدد درست ۳۲ بیتی
صحیح ۶۴	i64	عدد درست ۶۴ بیتی (برای عددهای خیلی بزرگ)

برای کارهای روزمره صحیح ۳۲ کافی است. اگر با عددهای بسیار بزرگ سر و کار دارید (مثلاً فکتوریل اعداد یا شمارش چیزهای فراوان)، از صحیح ۶۴ استفاده کنید.

۱	تابع اصلی:
۲	متغیر کوچک صحیح ۳۲ = 1000
۳	متغیر بزرگ صحیح ۶۴ = 9000000000
۴	چاپ (کوچک، بزرگ)
۵	پایان

۲.۳.۲ اعداد اعشاری

عددهایی که بخش اعشاری دارند — مانند ۱/۷۵، ۳/۱۴ و ۰/۵ — را اعداد اعشاری می‌نامیم. در سلام نوع آن‌ها اعشاری ۶۴ است (معادل f64):

۱	تابع اصلی:
۲	قد اعشاری ۶۴ = 75.1
۳	وزن اعشاری ۶۴ = 5.68
۴	چاپ (قد، وزن)
۵	پایان

هشدار

در عددهای اعشاری، نقطه اعشار همیشه با نقطه لاتین (.) نوشته می‌شود، نه ممیز فارسی. پس بنویسید 1.75 نه «۱/۷۵». همچنین برای آنکه عددی اعشاری شمرده شود، باید نقطه داشته باشد: 2 یک عدد درست است اما 2.0 یک عدد اعشاری.

۳.۳.۲ متن (رشته)

هر دنباله‌ای از نویسه‌ها — یک واژه، یک جمله، یا حتی یک نام — یک رشته (string) است. رشته‌ها را همیشه میان دو نقل قول می‌نویسیم و نوعشان رشته است (معادل str):

۱	تابع اصلی:
۲	شهر رشته = "اصفهان"
۳	پیام رشته = "به سلام خوش آمدید"
۴	چاپ (شهر، پیام)
۵	پایان

۴.۳.۲ مقادیر منطقی (درست/نادرست)

گاهی فقط دو پاسخ ممکن است: آری یا نه، روشن یا خاموش، درست یا نادرست. برای این موارد نوع منطقی (معادل bool) را داریم که تنها دو مقدار می‌پذیرد: درست (true) و نادرست (false):

۱	تابع اصلی:
۲	باران منطقی = درست
۳	آفتاب منطقی = نادرست
۴	چاپ (باران، آفتاب)
۵	پایان

مقادیر منطقی هنگام تصمیم‌گیری بسیار به کار می‌آیند؛ در فصل شرط‌ها (فصل ۴) نقش کلیدی آن‌ها را خواهیم دید.

۵.۳.۲ کاراکتر (تک‌نویسه)

اگر تنها به یک نویسه نیاز داشته باشید — مثلاً حرف A یا رقم ۷ — از نوع کاراکتر (معادل char) استفاده می‌کنید. کاراکترها را میان تک‌تقل‌قول می‌نویسیم:

۱	تابع اصلی:
۲	حرف کاراکتر = 'A'
۳	چاپ حرف
۴	پایان

یادداشت

تفاوت 'A' و "A" چیست؟ نخستی یک کاراکتر (تک‌نویسه) است و دومی یک رشته دنباله‌ای از نویسه‌ها که اینجا فقط یک نویسه دارد). در بسیاری از زبان‌ها این تمایز مهم است.

۴.۲ جدول کامل انواع پایه

فارسی	انگلیسی	چه چیزی نگه می‌دارد
صحیح / صحیح ۳۲	i32 / int	عدد درست ۳۲ بیتی
صحیح ۶۴	i64	عدد درست ۶۴ بیتی
اعشاری ۶۴	f64	عدد اعشاری
اعشاری ۳۲	f32	عدد اعشاری کوچک‌تر
منطقی	bool	درست یا نادرست
کاراکتر	char	یک نویسه
رشته	str	دنباله‌ای از نویسه‌ها
خودکار	auto	نوع را خود سلام تشخیص می‌دهد

نکته

نام‌های فارسی و انگلیسی انواع هر دو در سلام کار می‌کنند، حتی در یک برنامه فارسی. یعنی می‌توانید بنویسید $30 = \text{صحیح } ۳۲$ سن متغیر یا $30 = \text{سن } i32$ اما برای یکدستی کد بهتر است در هر برنامه یک شیوه را برگزینید.

۵.۲ تبدیل نوع با «بعنوان»

گاهی می‌خواهید یک مقدار را از نوعی به نوع دیگر تبدیل کنید؛ مثلاً یک عدد درست را به اعشاری، یا برعکس. برای این کار از واژه بعنوان (معادل as) استفاده می‌کنید:

۱	تابع اصلی:
۲	متغیر شمار صحیح $۳۲ = 7$
۳	متغیر دقیق اعشاری $۶۴ = \text{شمار بعنوان اعشاری } ۶۴$
۴	چاپ (دقیق)
۵	پایان

اینجا شمار (که یک عدد درست است) به اعشاری ۶۴ تبدیل شد. به این کار تبدیل نوع (cast) می‌گویند و در فصل‌های بعد — به‌ویژه هنگام کار با شمارشی‌ها و آرایه‌ها — بارها به آن نیاز خواهیم داشت.

۶.۲ قاعده‌های نام‌گذاری

نام متغیرها را خودتان برمی‌گزینید، اما چند قاعده ساده هست:

- نام می‌تواند فارسی، انگلیسی یا ترکیبی باشد: سن، age، شماره حساب.
- نام می‌تواند شامل زیرخط (_) باشد: سال_تولد.
- نام نمی‌تواند با یک رقم آغاز شود.
- نام نباید یکی از واژگان کلیدی زبان (مانند تابع یا اگر) باشد.

نکته

نام خوب، نامی است که معنای متغیر را برساند. مجموع نمرات بسیار گویاتر از م است. کدی که شش ماه بعد بخوانید، باید همچنان قابل فهم باشد — و نام خوب نیمی از این فهم را تأمین می‌کند.

نگاهی به درون: نوع چگونه به ایمنی کمک می‌کند (اختیاری، برای خوانندگان پیشرفته)

سلام یک زبان ایستا-نوع (statically typed) است؛ یعنی نوع هر متغیر پیش از اجرا و در زمان کامپایل مشخص است. این ویژگی به کامپایلر اجازه می‌دهد بسیاری از خطاها را پیش از اجرای برنامه بگیرد. برای نمونه، اگر بکوشید یک رشته را با یک عدد جمع بزنید، یا مقدار اعشاری را در جعبه‌ای از نوع صحیح بگذارید، کامپایلر همان لحظه به شما هشدار می‌دهد — به جای آنکه برنامه هنگام اجرا و شاید در دست کاربر، ناگهان از کار بیفتد.

۷.۲ خلاصه فصل

- متغیر، جعبه‌ای نام‌دار برای نگه‌داشتن داده است.
- چهار شیوه اعلان داریم: متغیر (تغییرپذیر)، ثابت (تغییرناپذیر)، اعلان ساده (تغییرناپذیر)، و خودکار (تشخیص خودکار نوع).
- انواع پایه: صحیح، صحیح ۶۴، اعشاری ۶۴، منطقی، کاراکتر و رشته.
- با بعنوان می‌توان نوع یک مقدار را تبدیل کرد.
- نام‌های فارسی و انگلیسی انواع هر دو معتبرند.

۸.۲ تمرین‌ها

تمرین ۱.۲. سه متغیر بسازید: نام شهر (رشته)، جمعیت آن (صحیح ۶۴) و میانگین دمای سالانه (اعشاری ۶۴). سپس هر سه را در یک خط چاپ کنید.

تمرین ۲.۲. یک ثابت به نام π با مقدار 3.14 بسازید و کوشش کنید در خط بعد مقدارش را تغییر دهید. کامپایلر چه می‌گوید؟

تمرین ۳.۲. متغیری از نوع منطقی بسازید که نشان دهد امروز تعطیل است یا نه، و آن را چاپ کنید.

تمرین ۴.۲. عدد درست 9 را در یک متغیر بگذارید، سپس آن را اعشاری ۶۴ بعنوان تبدیل کرده و در متغیر دیگری بگذارید و هر دو را چاپ کنید.

تمرین ۵.۲. برنامه‌ای بنویسید که از خودکار برای ساختن سه متغیر (یک عدد، یک اعشاری و یک رشته) استفاده کند و هر سه را چاپ کند.

فصل ۳

عملگرها و عبارت‌ها

تا اینجا یاد گرفتیم چگونه داده را در متغیرها نگه داریم. اما برنامه‌نویسی فقط نگه داشتن داده نیست؛ بیشتر کار، محاسبه و مقایسه و تصمیم‌گیری روی داده‌هاست. ابزار این کارها عملگرها هستند. در این فصل با خانواده عملگرهای سلام آشنا می‌شویم.

۱.۳ عبارت چیست؟

هر چیزی که محاسبه شود و به یک مقدار برسد، یک عبارت (expression) است. $2 + 3$ یک عبارت است که به 5 می‌رسد. $18 > \text{سن}$ هم یک عبارت است که به درست یا نادرست می‌رسد. عملگرها همان نشانه‌هایی‌اند که این محاسبه‌ها را انجام می‌دهند: $+$ ، $-$ ، $>$ و مانند آن‌ها.

۲.۳ عملگرهای حسابی

برای کار با اعداد، این عملگرها را داریم:

عملگر	کار	نمونه
+	جمع	$8 \rightarrow 5 + 3$
-	تفریق	$2 \rightarrow 5 - 3$
*	ضرب	$15 \rightarrow 5 * 3$
/	تقسیم	$3 \rightarrow 10 / 3$
%	باقی‌مانده تقسیم	$1 \rightarrow 10 \% 3$
**	توان	$1024 \rightarrow 2^{**10}$

دو عملگر آخر کمی نیاز به توضیح دارند.

۱.۲.۳ تقسیم صحیح در برابر تقسیم اعشاری

به این برنامه — برگرفته از مثال‌های واقعی سلام — نگاه کنید:

۱	تابع اصلی:
۲	الف اعشاری ۶۴ = $0.2 / 0.7$
۳	چاپ الف
۴	ب $i32 = 2 / 7$
۵	چاپ ب
۶	پایان

خروجی چنین است:

۱	3.5
۲	3

چرا $7 / 2$ برابر 3 شد، نه 3.5؟ زیرا وقتی هر دو عدد درست (صحیح) باشند، سلام تقسیم صحیح انجام می‌دهد و بخش اعشاری را دور می‌اندازد. اما $7.0 / 2.0$ چون اعشاری است، نتیجه دقیق 3.5 را می‌دهد.

هشدار

این یکی از رایج‌ترین سرچشمه‌های اشتباه برای تازه‌کارهاست. اگر می‌خواهید نتیجه تقسیم اعشاری باشد، دست‌کم یکی از دو عدد را اعشاری بنویسید، یا با عنوان تبدیلش کنید:

۱	تابع اصلی:
۲	ب $i32 = 7$
۳	دقیق اعشاری ۶۴ = (ب بعنوان اعشاری ۶۴) $0.2 /$
۴	چاپ دقیق
۵	پایان

که چاپ می‌کند 3.5.

۲.۲.۳ باقی‌مانده (%) : قهرمانِ پنهان

عملگر % باقی‌مانده یک تقسیم را می‌دهد. $3 \% 10$ برابر ۱ است، چون $10 = 3 \times 3 + 1$. این عملگر ساده کاربردهای فراوان دارد؛ مهم‌ترینش تشخیص بخش‌پذیری است: عددی بر ۲ بخش‌پذیر (زوج) است اگر باقی‌مانده تقسیمش بر ۲ صفر باشد:

۱	تابع اصلی:
۲	عدد صحیح $12 = 3^2$
۳	اگر عدد $2 \% 0 == 0$:
۴	چاپ "زوج است"
۵	وگرنه:
۶	چاپ "فرد است"
۷	پایان
۸	پایان

نکته

بازی معروف «فیزباز» که در فصل بعد می‌سازیم، تماماً بر پایه همین عملگر % بنا شده است. هر جا با «هر چند تا یک‌بار»، «زوج/فرد» یا «بخش‌پذیری» سر و کار داشتید، احتمالاً % ابزار شماست.

۳.۳ عملگرهای مقایسه‌ای

این عملگرها دو مقدار را مقایسه می‌کنند و پاسخشان همیشه منطقی است — یعنی درست یا نادرست:

عملگر	معنا	نمونه
==	برابر است با	$5 == 5 \rightarrow$ درست
!=	برابر نیست با	$5 != 3 \rightarrow$ درست
<	کوچک‌تر از	$3 < 5 \rightarrow$ درست
>	بزرگ‌تر از	$9 > 2 \rightarrow$ درست
<=	کوچک‌تر یا مساوی	$5 <= 5 \rightarrow$ درست
>=	بزرگ‌تر یا مساوی	$4 >= 7 \rightarrow$ نادرست

هشدار

به تفاوت $=$ و $==$ بسیار دقت کنید! یک $=$ یعنی «این مقدار را بگذار در آن متغیر» (انتساب)، اما دو $==$ یعنی «آیا این دو برابرند؟» (مقایسه). جابه‌جا گرفتن این دو، اشتباهی رایج است.

۴.۳ عملگرهای منطقی

گاهی می‌خواهیم چند شرط را با هم ترکیب کنیم. عملگرهای منطقی این کار را می‌کنند:

عملگر	معنا	درست است وقتی...
&&	و (and)	هر دو طرف درست باشند
	یا (or)	دست‌کم یک طرف درست باشد
!	نقیض (not)	طرفِ مقابلِ نادرست باشد

این برنامه کوچک — برگرفته از آزمون‌های واقعی سلام — هر سه را نشان می‌دهد:

۱	تابع اصلی:
۲	الف منطقی = درست
۳	ب منطقی = نادرست
۴	چاپ الف && ب, الف ب, !الف
۵	چاپ 3 < 5, 5 <= 5, 5 > 9, 2
۶	پایان

خروجی:

```
۱ false true false
۲ true true true
```

• ب && الف نادرست است، چون ب نادرست است و «و» نیاز دارد که هر دو درست باشند.

• ب || الف درست است، چون دست‌کم یکی (الف) درست است.

- !الف نادرست است، چون الف درست بود و نقیضش نادرست می‌شود.

نکته

یک کاربرد روزمره: «اگر هوا آفتابی است و تعطیل است، به پارک برویم.» این جمله در سلام چنین می‌شود:

اگر آفتابی && تعطیل:

چاپ "به پارک برویم"

پایان

۱
۲
۳

۵.۳ پیوند رشته‌ها

عملگر + تنها برای اعداد نیست؛ روی رشته‌ها هم کار می‌کند و دو متن را به هم می‌چسباند. به این مثال واقعی نگاه کنید:

تابع اصلی:

نام رشته = "سلام"

سلام رشته = نام + "دنیا"

چاپ سلام

چاپ نام.len()

چاپ سلام.len()

پایان

۱
۲
۳
۴
۵
۶
۷

خطِ دنیا" + نام دو رشته را به هم چسباند و حاصل «سلام دنیا» شد. همچنین نام.len() طول رشته (تعداد نویسه‌ها) را می‌دهد. در فصل ۸ بسیار بیشتر درباره کار با رشته‌ها خواهیم گفت.

یادداشت

عملگر + وقتی هر دو طرف عدد باشند، جمع می‌کند؛ و وقتی رشته باشند، می‌چسباند. سلام از روی نوع طرفین تشخیص می‌دهد کدام کار را انجام دهد.

۶.۳ ترتیب اولویت عملگرها

وقتی چند عملگر در یک عبارت باشند، سلام به ترتیبی مشخص آن‌ها را محاسبه می‌کند — درست مانند ریاضیات مدرسه. برای نمونه در $2 + 3 * 4$ ، نخست ضرب انجام می‌شود و سپس جمع، پس نتیجه ۱۴ است نه ۲۰. ترتیب کلی (از زودتر به دیرتر) چنین است:

۱. پرانتز: (...)

۲. نقیض و منفی یکانی: !، -

۳. توان: **

۴. ضرب، تقسیم، باقی‌مانده: /، *، %

۵. جمع و تفریق: +، -

۶. مقایسه‌ها: <، >، <=، >=

۷. برابری: ==، !=

۸. و: &&

۹. یا: ||

نکته

لازم نیست این جدول را حفظ کنید! هر وقت شک داشتید، از پرانتز استفاده کنید. $4 * (3 + \text{هیچ ابهامی ندارد و خواناتر هم هست. برنامه‌نویسان باتجربه نیز برای روشنی بیشتر، فراوان از پرانتز بهره می‌برند.})$

به این مثال واقعی از تبدیل دما دقت کنید که در آن پرانتز نقش کلیدی دارد:

۱	تابع اصلی:
۲	چاپ "سلسیوس به فارنهایت"
۳	برای متغیر س = 0; <= 40 س = س + 10;
۴	ف اعشاری ۶۴ = (س بعنوان اعشاری ۶۴) * 0.9 / 0.5 + 0.32
۵	چاپ س، "س"، "ف"، "ف"
۶	پایان
۷	پایان

اینجا پرانتز اعشاری (۶۴) بعنوان (س تضمین می‌کند که نخست تبدیل نوع انجام شود و سپس ضرب و تقسیم). (فرمول تبدیل سلسیوس به فارنهایت $F = C \times \frac{9}{5} + 32$ است).

۷.۳ عملگرها روی انواع خودمان

نکته شگفت‌انگیز سلام این است که می‌توانید عملگرها را برای نوع‌های ساخته خودتان هم تعریف کنید. مثلاً اگر یک «نقطه» در صفحه بسازید، می‌توانید معنای جمع دو نقطه را خودتان مشخص کنید:

۱	ساختار نقطه:
۲	$0 = i32\ x$
۳	$0 = i32\ y$
۴	تابع عملگر + (دیگری نقطه) نقطه:
۵	بازگشت نقطه $\{ x = \text{این } x + \text{دیگری } x, y = \text{این } y + \text{دیگری } y \}$
۶	پایان
۷	پایان
۸	تابع اصلی:
۹	الف نقطه = نقطه $\{ 4 = y\ 3, = x \}$
۱۰	ب نقطه = نقطه $\{ 2 = y\ 1, = x \}$
۱۱	ج نقطه = الف + ب
۱۲	چاپ ج. x, ج. y
۱۳	پایان

اینجا با تعریف عملگر + تابع، گفتیم که جمع دو نقطه یعنی جمع مؤلفه‌های x و y . حالا ب + الف معنا پیدا می‌کند و خروجی 4 6 می‌شود. نگران نباشید اگر هنوز ساختار را نمی‌شناسید؛ در فصل ۹ به تفصیل به آن می‌پردازیم. این مثال فقط برای آن است که قدرت سلام را زود ببینید.

نگاهی به درون: عملگرها چگونه گسترش می‌یابند (اختیاری، برای خوانندگان پیشرفته)

وقتی ب + الف را روی دو «نقطه» می‌نویسید، سلام در زمان کامپایل آن را به فراخوانی تابع عملگر + تبدیل می‌کند — چیزی شبیه الف.عملگر+(ب). هیچ هزینه اضافه‌ای در زمان اجرا ندارد؛ این یک ترجمه ساده در زمان ساخت برنامه است. به همین شیوه می‌توانید

==، * و بسیاری عملگر دیگر را نیز برای نوع‌های خود تعریف کنید (فصل ۹).

۸.۳ خلاصه فصل

- عبارت، هر چیزی است که به یک مقدار می‌رسد.
- عملگرهای حسابی: $+$ $-$ $*$ $/$ $\%$ $**$.
- تقسیم دو عدد صحیح، صحیح است؛ برای نتیجه اعشاری، یک طرف را اعشاری کنید.
- عملگرهای مقایسه‌ای پاسخ منطقی می‌دهند.
- عملگرهای منطقی (&&، ||، !) شرط‌ها را ترکیب می‌کنند.
- $+$ روی رشته‌ها آن‌ها را به هم می‌چسباند.
- هنگام شک، از پرانتز برای روشن کردن ترتیب استفاده کنید.

۹.۳ تمرین‌ها

- تمرین ۱.۳.** برنامه‌ای بنویسید که محیط و مساحت یک مستطیل با طول ۸ و عرض ۵ را حساب و چاپ کند.
- تمرین ۲.۳.** با استفاده از عملگر $\%$ ، برنامه‌ای بنویسید که بگوید عدد ۲۷ بر ۳ بخش پذیر است یا نه.
- تمرین ۳.۳.** حاصل $2 + 3 * 4$ و $(2 + 3) * 4$ را در دو خط چاپ کنید و تفاوت را ببینید.
- تمرین ۴.۳.** دو متغیر منطقی بسازید (مثلاً «گرسنه» و «پول دارم») و با $\&\&$ تصمیم بگیرید که آیا باید غذا بخرد یا نه.
- تمرین ۵.۳.** نام کوچک و نام خانوادگی خود را در دو رشته جداگانه بگذارید، آن‌ها را با یک فاصله به هم بچسبانید و نام کامل را چاپ کنید. سپس طول نام کامل را نیز چاپ کنید.

فصل ۴

شرط‌ها و تصمیم‌گیری

تا اینجا برنامه‌های ما خطبه‌خط و بدون هیچ انتخابی پیش می‌رفتند. اما زندگی واقعی پر از تصمیم است: «اگر باران آمد، چتر بردار»، «اگر نمره از ۱۰ بیشتر شد، قبول هستی». در این فصل یاد می‌گیریم که برنامه‌مان هم بتواند تصمیم بگیرد.

۱.۴ اگر: نخستین تصمیم

ساده‌ترین شکل تصمیم‌گیری با واژه اگر (معادل if) انجام می‌شود:

۱	تابع اصلی:
۲	سن صحیح $32 = 20$
۳	اگر سن $>= 18$:
۴	چاپ "شما بزرگسال هستید"
۵	پایان
۶	پایان

این برنامه را چنین بخوانید: «اگر سن بزرگ‌تر یا مساوی ۱۸ بود، آنگاه پیام را چاپ کن.» اگر شرط درست باشد، بدنه اجرا می‌شود؛ وگرنه سلام از روی آن می‌پرد. ساختار همیشه یکسان است:

اگر شرط : ... بدنه ... پایان

شرط، هر عبارتی است که به درست یا نادرست برسد — درست همان عبارتهای مقایسه‌ای و منطقی فصل پیش.

۲.۴ وگرنه: راه دوم

گاهی می‌خواهیم اگر شرط درست نبود، کار دیگری انجام شود. برای این از وگرنه (معادل else) استفاده می‌کنیم:

۱	تابع اصلی:
۲	سن صحیح ۳۲ = 15
۳	اگر سن >= 18:
۴	چاپ "بزرگسال"
۵	وگرنه:
۶	چاپ "نوجوان یا کودک"
۷	پایان
۸	پایان

اکنون دقیقاً یکی از دو پیام چاپ می‌شود: یا «بزرگسال» (اگر شرط درست باشد) یا «نوجوان یا کودک» (اگر نادرست باشد). هرگز هر دو با هم چاپ نمی‌شوند.

۳.۴ وگرنه اگر: چند راه

زندگی همیشه دوراهی نیست؛ گاهی چند گزینه داریم. برای این حالت، اگر وگرنه (معادل else if) را پشت سر هم می‌چینیم. به این مثال واقعی — که نمره را به طبقه حرفی تبدیل می‌کند — نگاه کنید:

۱	تابع طبقه (ن 32i رشته:
۲	اگر ن ≥ 90 :
۳	بازگشت "الف"
۴	وگرنه اگر ن ≥ 70 :
۵	بازگشت "ب"
۶	وگرنه:
۷	بازگشت "ج"
۸	پایان
۹	پایان
۱۰	تابع اصلی:
۱۱	چاپ طبقه (95)
۱۲	چاپ طبقه (75)
۱۳	چاپ طبقه (40)
۱۴	پایان

خروجی:

۱ الفبج

سلام شرطها را / از بالا به پایین می‌آزماید و به نخستین شرطِ درست که رسید، بدنه آن را اجرا می‌کند و باقی را نادیده می‌گیرد. پس برای نمره ۹۵، نخستین شرط (≥ 90) درست است و «الف» بازگردانده می‌شود؛ سلام دیگر شرطهای بعدی را نمی‌آزماید.

هشدار

ترتیب شرطها اهمیت دارد! اگر در مثال بالا شرط ≥ 70 ن را پیش از ≥ 90 ن می‌نوشتیم، نمره ۹۵ نیز در شرط ۷۰ گیر می‌افتاد و به اشتباه «ب» می‌گرفت. همیشه شرطهای سخت‌گیرانه‌تر (محدودتر) را زودتر بیاورید.

یادداشت

در این مثال از بازگشت درون یک تابع استفاده کردیم تا نتیجه را برگردانیم. بازگشت هم مقدار را به بیرون تابع می‌فرستد و هم بی‌درنگ از تابع خارج می‌شود. توابع را در فصل ۶ به‌طور کامل می‌شناسیم.

۴.۴ شرط در یک خط

وقتی بدنه شرط کوتاه است، می‌توانید همه را در یک خط بنویسید. این مثال واقعی «سال کبیسه» سه شیوه فشردگی را نشان می‌دهد:

۱	// سال کبیسه
۲	تابع کبیسه_است (سال صحیح ۳۲) منطقی:
۳	اگر سال % 400 == 0: بازگشت درست پایان
۴	اگر سال % 100 == 0: بازگشت نادرست پایان
۵	بازگشت سال % 4 == 0
۶	پایان
۷	تابع اصلی:
۸	متغیر سالها صحیح ۳۲ [5] =, [1900, 2000, 2012, 2021, 2024]
۹	برای متغیر $i = 0; i < 5; i = i + 1$:
۱۰	اگر کبیسه_است (سالها[i]): چاپ سالها[i], "کبیسه است"
۱۱	وگرنه: چاپ سالها[i], "کبیسه نیست"
۱۲	پایان
۱۳	پایان
۱۴	پایان

اینجا چند چیز تازه می‌بینیم:

- شرط و بدنه و پایان در یک خط: پایان درست بازگشت: $0 == 400 \% \text{ سال اگر}$. این شکل فشردگی برای شرطهای کوتاه بسیار خواناست.
- تابعی که یک مقدار منطقی برمی‌گرداند — خود $0 == 4 \% \text{ سال یک عبارت منطقی}$ است و مستقیم بازگردانده می‌شود.
- یک آرایه و یک حلقه که در فصل‌های بعد به آن‌ها می‌رسیم.

نگاهی به درون: منطق سال کبیسه (اختیاری، برای خوانندگان پیشرفته)

چرا قاعده کبیسه این قدر پیچیده است؟ سالی کبیسه است که بر ۴ بخش پذیر باشد — مگر آنکه بر ۱۰۰ بخش پذیر باشد (که آنگاه کبیسه نیست)، مگر آنکه بر ۴۰۰ بخش پذیر باشد (که باز کبیسه است). به همین دلیل کد، نخست ۴۰۰، سپس ۱۰۰ و در آخر ۴ را می‌آزماید — ترتیبی که دقیقاً منطق این استثناها را پیاده می‌کند.

۵.۴ شرط‌های تودرتو

می‌توانید یک اگر را درون یک اگر دیگر بگذارید. به این نمونه دقت کنید که نخست بررسی می‌کند آیا کاربر وارد شده، و سپس آیا مدیر است:

۱	تابع اصلی:
۲	وارد شده منطقی = درست
۳	مدیر منطقی = نادرست
۴	اگر وارد شده:
۵	اگر مدیر:
۶	چاپ "خوش آمدید، مدیر گرامی"
۷	وگرنه:
۸	چاپ "خوش آمدید، کاربر عزیز"
۹	پایان
۱۰	وگرنه:
۱۱	چاپ "لطفاً ابتدا وارد شوید"
۱۲	پایان
۱۳	پایان

نکته

شرط‌های تودرتوی عمیق، کد را سخت‌خوان می‌کنند. اغلب می‌توان به جای تودرتو کردن، شرط‌ها را با && ترکیب کرد. مثلاً مدیر: && وارد شده اگر کوتاه‌تر و خواناتر از دو اگر تودرتوست — البته اگر به آن «وگرنه»ی میانی نیاز نداشته باشید.

۶.۴ مثال کامل: بازی حدس ساده

بیاید همهٔ آموخته‌ها را در یک مثال کوچک گرد آوریم. این برنامه یک دما را ارزیابی می‌کند:

۱	تابع وضعیت هوا (دما صحیح ۳۲) رشته:
۲	اگر دما < 0:
۳	بازگشت "یخبندان"
۴	وگرنه اگر دما < 15:
۵	بازگشت "سرد"
۶	وگرنه اگر دما < 30:
۷	بازگشت "معتدل"
۸	وگرنه:
۹	بازگشت "گرم"
۱۰	پایان
۱۱	پایان
۱۲	تابع اصلی:
۱۳	چاپ ۵- درجه:، وضعیت هوا (۵-)
۱۴	چاپ ۱۰- درجه:، وضعیت هوا (10)
۱۵	چاپ ۲۵- درجه:، وضعیت هوا (25)
۱۶	چاپ ۳۸- درجه:، وضعیت هوا (38)
۱۷	پایان

این الگو — یک تابع که با زنجیره‌ای از اگر وگرنه یک مقدار را به یک دسته نگاشت می‌کند — یکی از پرکاربردترین الگوهای برنامه‌نویسی است.

۷.۴ خلاصه فصل

- اگر بدنه را تنها وقتی اجرا می‌کند که شرط درست باشد.
- وگرنه راه جایگزین را برای حالتِ نادرست فراهم می‌کند.
- اگر وگرنه چند گزینه را پشتِ سرِ هم می‌آزماید.
- شرطها از بالا به پایین آزموده می‌شوند و نخستین درست، برنده است.
- شرطهای کوتاه را می‌توان در یک خط نوشت.
- به‌جای تودرتوی عمیق، شرطها را با && و || ترکیب کنید.

۸.۴ تمرین‌ها

تمرین ۱.۴. برنامه‌ای بنویسید که یک عدد بگیرد و بگوید مثبت، منفی یا صفر است.

تمرین ۲.۴. تابعی به نام بزرگ‌تر بنویسید که دو عدد بگیرد و بزرگ‌ترشان را برگرداند (با اگر و وگرنه).

تمرین ۳.۴. برنامه‌ای بنویسید که نمره‌ای بین ۰ تا ۲۰ بگیرد و بگوید: عالی (۱۸ به بالا)، خوب (۱۵ تا ۱۸)، قبول (۱۰ تا ۱۵) یا مردود (زیر ۱۰).

تمرین ۴.۴. با کمک عملگر %، تابعی بنویسید که بگوید عددی زوج است یا فرد.

تمرین ۵.۴. تابع کبیسه است را بازنویسی کنید، اما این بار به جای سه اگر جداگانه، همه را در یک عبارت منطقی بزرگ بنویسید (راهنمایی: از &&، || و پرانتز استفاده کنید).

فصل ۵

حلقه‌ها

رایانه در یک کار بسیار توانمند است: تکرار. کارهایی که برای انسان خسته‌کننده و طولانی‌اند — مثلاً جمع‌زدن هزار عدد یا چاپ صد سطر — را رایانه در یک چشم‌به‌هم‌زدن انجام می‌دهد. ابزار این تکرار، **حلقه** است. در این فصل با همه حلقه‌های سلام آشنا می‌شویم.

۱.۵ تا وقتی: تا زمانی که شرط برقرار است

ساده‌ترین حلقه، تا وقتی (معادل while) است. این حلقه بدنه خود را تا زمانی که یک شرط درست است، بارها و بارها اجرا می‌کند. به این مثال واقعی نگاه کنید که اعداد ۱ تا ۱۰ را جمع می‌زند:

۱	تابع اصلی:
۲	متغیر مجموع صحیح $0 = ۳۲$
۳	متغیر i صحیح $1 = ۳۲$
۴	تا وقتی $10: \leq i$
۵	مجموع = مجموع $i +$
۶	$1 + i = i$
۷	پایان
۸	چاپ ("مجموع ۱ تا ۱۰ برابر است با", مجموع)
۹	پایان

این حلقه را گام به گام دنبال کنیم:

۱. نخست مجموع برابر ۰ و i برابر ۱ است.

۲. سلام شرط $i \leq 10$ را می‌آزماید. درست است، پس بدنه اجرا می‌شود: مجموع برابر ۱ و i برابر ۲ می‌شود.

۳. دوباره شرط آزموده می‌شود. باز درست است... و این کار تا وقتی i به ۱۱ برسد ادامه می‌یابد.

۴. وقتی i برابر ۱۱ شد، شرط نادرست می‌شود و حلقه پایان می‌یابد.

خروجی: 55 با است برابر ۱۰ تا ۱ مجموع.

هشدار

هر حلقه تا وقتی به دو چیز نیاز دارد: شرطی که سرانجام نادرست شود، و تغییری در بدنه که ما را به آن نزدیک کند. اگر خط $i = i + 1$ را فراموش کنید، i همیشه ۱ می‌ماند، شرط همیشه درست است و حلقه هرگز پایان نمی‌یابد! به این اشتباه رایج، **حلقه بی‌پایان** می‌گویند.

۲.۵ برای: حلقه شمارنده‌دار

بسیار پیش می‌آید که می‌خواهیم کاری را به تعداد مشخصی تکرار کنیم و در ضمن یک شمارنده هم داشته باشیم. حلقه برای (معادل for) دقیقاً برای همین ساخته شده و هر سه بخش یک شمارنده را در یک خط گرد می‌آورد:

۱	تابع اصلی:
۲	برای متغیر $i = 1$; $<= i 10$; $1: + i = i$
۳	چاپ (5, "i", "*", "=" * 5) i)
۴	پایان
۵	پایان

خط برای سه بخش دارد که با نقطه‌ویرگول (;) از هم جدا می‌شوند:

برای آغاز ; شرط ; گام :

• آغاز ($i = 1$ متغیر): یک بار و در ابتدا اجرا می‌شود؛ شمارنده را می‌سازد.

• **شرط** ($i \leq 10$): پیش از هر تکرار آزموده می‌شود؛ تا درست است، حلقه ادامه دارد.

• **گام** ($i = i + 1$): پس از هر تکرار اجرا می‌شود؛ معمولاً شمارنده را پیش می‌برد.

این برنامه جدول ضرب ۵ را چاپ می‌کند. در عمل، حلقه برای و تا وقتی هم‌ارزند؛ هر کدام را با دیگری می‌توان نوشت. اما وقتی شمارنده‌ای روشن دارید، برای خوانا تر و جمع‌وجورتر است.

نکته

نام i برای شمارنده حلقه یک سنت دیرینه است (کوتاه‌شده $index$). برای حلقه‌های تودرتو از i ، j و k استفاده می‌کنند. البته می‌توانید نام‌های فارسی گویا تر مانند ردیف و ستون هم به کار ببرید.

۳.۵ شکل‌کشی با حلقه‌های تودرتو

وقتی یک حلقه را درون حلقه دیگر بگذاریم، کارهای جالبی می‌توان کرد. این مثال واقعی یک مثلث ستاره می‌کشد — نمونه‌ای محبوب نزد تازه‌کارها:

```

۱ // مثلث ستاره
۲ تابع اصلی:
۳ برای متغیر ردیف = 1; ردیف <= 5; ردیف = ردیف + 1:
۴     متغیر خط رشته = ""
۵     برای متغیر ستون = 0; ستون < ردیف; ستون = ستون + 1:
۶         چاپ خط
۷     پایان
۸ پایان
خروجی:
```

```

۱ *
۲ **
۳ ***
```

۴ ****
۵ *****

حلقه بیرونی روی ردیف‌ها (۱ تا ۵) می‌چرخد، و برای هر ردیف، حلقه درونی به تعداد شماره آن ردیف ستاره به رشته خط می‌افزاید. سپس خط چاپ می‌شود. این الگوی «ساختن یک رشته در حلقه درونی و چاپ آن در حلقه بیرونی» را بارها به کار خواهید برد.

نکته

همین برنامه را می‌توان با جدول ضرب هم نوشت: کافی است به جای افزودن ستاره، حاصل ستون * ردیف را به خط بیفزایید. مثال‌های واقعی سلام چنین برنامه‌ای نیز دارند.

۴.۵ تکرار: ساده‌ترین راه تکرار

گاهی فقط می‌خواهیم کاری را چند بار انجام دهیم و به شمارنده کاری نداریم. برای این، سلام واژه زیبای تکرار (معادل repeat) را دارد که سه شکل دارد. این مثال واقعی هر سه را نشان می‌دهد:

۱	تابع اصلی:
۲	// (۱) شکل شمارشی: بدنه را N بار اجرا می‌کند
۳	تکرار 5:
۴	چاپ "تست"
۵	پایان
۶	
۷	// شمارنده می‌تواند هر عبارت عددی باشد (یک بار محاسبه می‌شود)
۸	تعداد صحیح = 3
۹	تکرار تعداد:
۱۰	بنویس "x"
۱۱	پایان
۱۲	چاپ ""
۱۳	
۱۴	// (۲) شکل بازه‌ای: از یک آغاز تا یک پایان شامل
۱۵	تکرار 1 تا 5:
۱۶	بنویس "#"
۱۷	پایان
۱۸	چاپ ""
۱۹	
۲۰	// (۳) شکل بازه‌ای با گام: ۰، ۱۰، ۲۰، ...، ۱۰۰
۲۱	تکرار 0 تا 100 هر 10:
۲۲	بنویس "."
۲۳	پایان
۲۴	چاپ ""
۲۵	پایان

سه شکل تکرار چنین‌اند:

۱. شمارشی (N تکرار): بدنه را N بار اجرا می‌کند.

۲. بازه‌ای (ب: تا آ تکرار): برای هر مقدار از آ تا ب (شامل خودِ ب) یک بار اجرا می‌شود.

۳. بازه‌ای با گام (گ: هر ب تا آ تکرار): از آ تا ب با گام‌های گ پیش می‌رود.

یادداشت

تکرار شمارنده آشکاری در اختیار شما نمی‌گذارد — یعنی درون بدنه به شماره تکرار جاری دسترسی ندارید. اگر به آن نیاز دارید، یا یک شمارنده دستی نگه دارید (مثل مثال زیر) یا از حلقه برای استفاده کنید.

این مثال واقعی نشان می‌دهد چگونه با نگهداشتن یک شمارنده دستی، با تکرار هم می‌توان جمع زد و هم شکل کشید:

۱	// «تکرار آ تا ب:» بدنه را برای هر مقدار از آ تا ب (شامل) یک بار اجرا می‌کند.
۲	تابع اصلی:
۳	// جمع ۱ تا ۱۰
۴	متغیر مجموع صحیح = 0
۵	متغیر ک صحیح = 0
۶	تکرار 1 تا 10:
۷	ک = ک + 1
۸	مجموع = مجموع + ک
۹	پایان
۱۰	بنویس "۱+۲+...+۱۰" =
۱۱	چاپ مجموع
۱۲	
۱۳	// مثلث چپ‌چین: سطر ۲ تعداد ۲ ستاره چاپ می‌کند (سطرهای ۱ تا ۵)
۱۴	متغیر سطر صحیح = 0
۱۵	تکرار 1 تا 5:
۱۶	سطر = سطر + 1
۱۷	تکرار 1 تا سطر:
۱۸	بنویس "۲"
۱۹	پایان
۲۰	چاپ
۲۱	پایان
۲۲	پایان

۵.۵ بشکن و ادامه: کنترل دقیق‌تر حلقه

گاهی می‌خواهیم پیش از پایان طبیعی حلقه از آن بیرون بیاییم، یا یک تکرار را رد کنیم. برای این دو کار، بشکن (معادل break) و ادامه (معادل continue) را داریم. به این مثال واقعی نگاه کنید:

۱	تابع اصلی:
۲	متغیر مجموع $0 = i32$
۳	متغیر $0 = i32$
۴	تاوقتی $20 < i$
۵	$1 + i = i$
۶	اگر $0 == 2 \% i$
۷	ادامه
۸	پایان
۹	اگر $9 > i$
۱۰	بشکن
۱۱	پایان
۱۲	مجموع = مجموع + i
۱۳	پایان
۱۴	چاپ مجموع
۱۵	پایان

در این حلقه:

- ادامه وقتی i زوج است اجرا می‌شود و باعث می‌شود سلام باقی‌بده را رد کند و به تکرار بعد برود. پس اعداد زوج هرگز به مجموع افزوده نمی‌شوند.
 - بشکن وقتی i از ۹ بزرگ‌تر شود، حلقه را به کلی پایان می‌دهد.
- در نتیجه فقط اعداد فرد ۱ تا ۹ (یعنی ۱، ۳، ۵، ۷، ۹) جمع می‌شوند و خروجی ۲۵ می‌شود.

نکته

بشکن برای زمانی عالی است که دنبال چیزی می‌گردید و همین که یافتیدش، دیگر نیازی به ادامه جست‌وجو نیست. ادامه هم برای رد کردن موارد ناخواسته (مثل اعداد زوج در مثال بالا) به کار می‌آید.

۶.۵ مثال کامل: بازی فیزباز

«فیزباز» تمرینی کلاسیک است که حلقه، شرط و باقی‌مانده را با هم به کار می‌گیرد. قاعده ساده است: از ۱ تا ۱۵ بشمار؛ به جای مضرب‌های ۳ بنویس «فیز»، به جای مضرب‌های ۵ بنویس «باز»، و به جای مضرب‌های هر دو بنویس «فیزباز». این مثال واقعی سلام است:

۱	// فیزباز
۲	تابع اصلی:
۳	برای متغیر $i = 1$; $\leq i 15$; $+ i = 1$:
۴	اگر $i \% 15 == 0$: چاپ "فیزباز"
۵	وگرنه اگر $i \% 3 == 0$: چاپ "فیز"
۶	وگرنه اگر $i \% 5 == 0$: چاپ "باز"
۷	وگرنه: چاپ i
۸	پایان
۹	پایان
۱۰	پایان

هشدار

چرا نخست $0 == 15 \% i$ را می‌آزماییم؟ چون عددی که هم بر ۳ و هم بر ۵ بخش‌پذیر است، حتماً بر ۱۵ هم بخش‌پذیر است. اگر این شرط را آخر می‌گذاشتیم، عدد ۱۵ زودتر در شرط ۳٪ گیر می‌افتاد و به اشتباه فقط «فیز» چاپ می‌شد. باز هم می‌بینیم که ترتیب شرط‌ها سرنوشت‌ساز است.

۷.۵ خلاصه فصل

- تاوقتی تا زمانی که شرط درست است تکرار می‌کند.
- برای حلقه‌ای با آغاز، شرط و گام در یک خط است.
- حلقه‌های تودرتو برای شکل‌کشی و جدول‌ها عالی‌اند.
- تکرار ساده‌ترین حلقه است و سه شکل دارد: شمارشی، بازه‌ای و بازه‌ای با گام.

- بشکن حلقه را پایان می‌دهد و ادامه به تکرار بعد می‌پرد.
- مراقب حلقه‌های بی‌پایان باشید!

۸.۵ تمرین‌ها

- تمرین ۱.۵.** با حلقه برای، اعداد ۱ تا ۲۰ را چاپ کنید.
- تمرین ۲.۵.** با حلقه تا وقتی، جمع اعداد زوج ۱ تا ۱۰۰ را حساب کنید.
- تمرین ۳.۵.** با تکرار و یک شمارنده دستی، فاکتوریل عدد ۵ را حساب کنید (یعنی $1 \times 2 \times 3 \times 4 \times 5$).
- تمرین ۴.۵.** یک مثلث راست‌چین با ۵ ردیف بکشید (هر ردیف چند فاصله و سپس ستاره دارد).
- تمرین ۵.۵.** جدول ضرب کامل ۱ تا ۵ را با دو حلقه تودرتو چاپ کنید.
- تمرین ۶.۵.** با بشکن، برنامه‌ای بنویسید که اعداد را از ۱ بشمارد و به محض رسیدن به نخستین عددی که بر ۷ بخش‌پذیر است، آن را چاپ کند و بایستد.

فصل ۶

توابع

تا اینجا چند بار با تابع روبه‌رو شده‌ایم — دست‌کم تابع اصلی را در هر برنامه دیده‌ایم. در این فصل توابع را به‌طور کامل می‌شناسیم: چرا به آن‌ها نیاز داریم، چگونه می‌سازیمشان، چگونه به آن‌ها ورودی می‌دهیم و چگونه از آن‌ها خروجی می‌گیریم.

۱.۶ تابع چیست و چرا به آن نیاز داریم؟

تابع، بسته‌ای از دستورهاست که نامی دارد و کاری مشخص انجام می‌دهد. تصور کنید دستور پخت یک کیک را یک‌بار نوشته‌اید؛ از آن پس هر وقت کیک خواستید، کافی است بگویید «کیک بپز» — لازم نیست هر بار همهٔ مراحل را تکرار کنید. تابع همین نقش را دارد: یک‌بار می‌نویسیدش و بارها صدایش می‌زنید. توابع سه فایدهٔ بزرگ دارند:

- **پرهیز از تکرار:** کاری که چند بار لازم می‌شود را یک‌بار می‌نویسید.
- **خوانایی:** برنامه به بخش‌های کوچک و معنادار تقسیم می‌شود.
- **آزمون‌پذیری:** هر تابع را جداگانه می‌توان آزمود و درست کرد.

۲.۶ ساختن نخستین تابع

به این مثال واقعی نگاه کنید که دو تابع کوچک می‌سازد:

۱	تابع توان (پایه صحیح ۳۲، نما صحیح ۳۲) صحیح ۳۲:
۲	متغیر نتیجه صحیح ۳۲ $1 =$
۳	برای متغیر $i = 0$; نما $i <$; $i = i + 1$:
۴	نتیجه = نتیجه * پایه
۵	پایان
۶	بازگشت نتیجه
۷	پایان
۸	تابع بیشینه (الف صحیح ۳۲، ب صحیح ۳۲) صحیح ۳۲:
۹	اگر الف $>$ ب:
۱۰	بازگشت الف
۱۱	پایان
۱۲	بازگشت ب
۱۳	پایان
۱۴	تابع اصلی:
۱۵	چاپ ("دو به توان ده:", توان ۲), (10))
۱۶	چاپ ("بیشینه:", بیشینه (توان ۲), 4) توان ۳), (2)))
۱۷	پایان

ساختار کلی یک تابع چنین است:

تابع (نام) ورودی‌ها (نوع خروجی) : ... بدنه ... پایان

بیایید تابع توان را بشکافیم:

- توان نام تابع است.
- صحیح ۳۲ (نما صحیح ۳۲، پایه دو پارامتر (ورودی) است: هر کدام نامی و نوعی دارند و با کاما جدا شده‌اند.
- صحیح ۳۲ پس از پرانتز، نوع خروجی تابع است؛ یعنی این تابع یک عدد صحیح برمی‌گرداند.
- درون بدنه، توان را با یک حلقه حساب می‌کنیم و با نتیجه بازگشت آن را بیرون می‌دهیم.

۳.۶ پارامترها: ورودی تابع

پارامترها مانند متغیرهایی هستند که هنگام صد/ زدن تابع، مقدار می‌گیرند. وقتی می‌نویسیم (10 توان 2)، عدد 2 در پایه و عدد 10 در نما می‌نشیند. ترتیب اهمیت دارد: نخستین مقدار به نخستین پارامتر می‌رود و الی آخر.

یادداشت

هر تابع می‌تواند هیچ، یک، یا چند پارامتر داشته باشد. تابع اصلی معمولاً بی‌پارامتر است؛ به همین دلیل پس از نامش پرانتز خالی نمی‌گذاریم و مستقیم اصلی: تابع می‌نویسیم.

۴.۶ بازگشت: خروجی تابع

واژه بازگشت (معادل ret) دو کار هم‌زمان انجام می‌دهد: مقداری را به بیرون تابع می‌فرستد، و بی‌درنگ از تابع خارج می‌شود. در تابع بیشینه، اگر $b > a$ باشد، بازگشت a اجرا می‌شود و باقی تابع هرگز اجرا نمی‌شود.

نکته

از همین ویژگی «خروج بی‌درنگ» می‌توان برای ساده‌تر کردن کد استفاده کرد. تابع بیشینه را ببینید: به جای نوشتن اگر...وگرنه، نخست حالت خاص را با بازگشت مدیریت می‌کنیم و باقی را در ادامه می‌نویسیم. این الگو به «بازگشت زودهنگام» معروف است و کد را تخت‌تر و خواناتر می‌کند.

۱.۴.۶ توابع بدون خروجی

هر تابعی لازم نیست چیزی برگرداند. تابعی که فقط کاری انجام می‌دهد (مثلاً چیزی چاپ می‌کند) نوع خروجی ندارد:

۱	تابع سلام بگو (نام رشته):
۲	چاپ ("سلام", نام)
۳	چاپ ("به سلام خوش آمدی!")
۴	پایان
۵	تابع اصلی:
۶	سلام بگو ("سارا")
۷	سلام بگو ("علی")
۸	پایان

دقت کنید که پس از رشته (نام هیچ نوع خروجی نیامده — چون این تابع چیزی برنمی‌گرداند. در این‌گونه توابع، بازگشت (بدون مقدار) را می‌توان برای خروج زود هنگام به کار برد، اما اجباری نیست.

۵.۶ بازگشتی: تابعی که خودش را صدا می‌زند

یکی از زیباترین ایده‌های برنامه‌نویسی این است که یک تابع می‌تواند خودش را صدا بزند. به این کار بازگشت یا بازگشتی (recursion) می‌گویند. مثال کلاسیک، فاکتوریل است:

۱	تابع فاکتوریل (ن i32: i32)
۲	اگر $n \leq 1$:
۳	بازگشت 1
۴	پایان
۵	بازگشت $n * \text{فاکتوریل}(n - 1)$
۶	پایان
۷	تابع اصلی:
۸	چاپ فاکتوریل (5)
۹	چاپ فاکتوریل (7)
۱۰	پایان

فاکتوریل ۵ یعنی $5 \times 4 \times 3 \times 2 \times 1 = 120$. تابع چنین می‌اندیشد: «فاکتوریل ۵ برابر است با ۵ ضربدر فاکتوریل ۴، و فاکتوریل ۴ برابر است با ۴ ضربدر فاکتوریل ۳، ...» تا به فاکتوریل ۱ برسد که پاسخش روشن است: ۱.

هشدار

هر تابع بازگشتی به یک شرط توقف (پایه بازگشت) نیاز دارد — جایی که دیگر خودش را صدا نمی‌زند. در فاکتوریل، این شرط $1 \leq n$ است. اگر این شرط را فراموش کنید، تابع تا بی‌نهایت خودش را صدا می‌زند و برنامه از کار می‌افتد. این خطا مانند همان «حلقه بی‌پایان» فصل پیش است.

نگاهی به درون: بازگشت چگونه کار می‌کند (اختیاری، برای خوانندگان پیشرفته)

هر بار که تابعی صدا زده می‌شود — چه خودش، چه دیگری — رایانه یک «قاب» تازه روی پشته فراخوانی (call stack) می‌سازد که پارامترها و متغیرهای آن فراخوانی را نگه می‌دارد. در فاکتوریل ۵، پنج قاب روی هم چیده می‌شوند تا به پایه بازگشت برسیم؛ سپس یکی یکی باز می‌گردند و ضرب‌ها انجام می‌شود. به همین دلیل بازگشت بیش از اندازه عمیق می‌تواند پشته را پر کند — نکته‌ای که در فصل ۲۰ به آن باز می‌گردیم.

۶.۶ ترتیب تعریف توابع

در نمونه‌های این کتاب، توابع کمکی را معمولاً پیش از تابع اصلی می‌نویسیم. اما این تنها یک قرارداد زیباست، نه یک اجبار. سلام برنامه را در دو گذر بررسی می‌کند: نخست همه توابع را می‌شناسد، سپس بدنه‌ها را واری می‌کند. به همین دلیل می‌توانید تابعی را پیش از تعریفش هم صدا بزنید — چیزی که گاهی «اعلان پیش‌رو» نامیده می‌شود و در زبان‌های دیگر در دسترس است، اما در سلام به‌طور طبیعی کار می‌کند.

۷.۶ ترکیب توابع

زیبایی توابع آن‌جاست که خروجی یکی را می‌توان ورودی دیگری کرد. در مثال نخست این فصل دیدیم:

۱ چاپ("بیشینه:", بیشینه(توان(2), 4), توان(3), 2)))

اینجا نخست (4 توان(2, برابر ۱۶ و (2 توان(3, برابر ۹ محاسبه می‌شوند، سپس (9 بیشینه(16, برابر ۱۶ می‌شود. این «لانه‌سازی» توابع، پایه ساختن برنامه‌های بزرگ از قطعات کوچک است.

۸.۶ خلاصه فصل

- تابع، بسته‌ای نام‌دار از دستورهاست که از تکرار جلوگیری می‌کند و کد را خوانا می‌سازد.
- پارامترها ورودی تابع اند و هنگام صدا زدن مقدار می‌گیرند.
- بازگشت خروجی را می‌فرستد و از تابع خارج می‌شود.
- توابع می‌توانند بدون خروجی باشند.
- تابع بازگشتی خودش را صدا می‌زند و به شرط توقف نیاز دارد.
- خروجی یک تابع می‌تواند ورودی تابع دیگر باشد.

۹.۶ تمرین‌ها

- تمرین ۱.۶.** تابعی به نام مربع بنویسید که یک عدد بگیرد و مربع آن را برگرداند.
- تمرین ۲.۶.** تابعی به نام کمینه بنویسید که دو عدد بگیرد و کوچک‌ترشان را برگرداند.
- تمرین ۳.۶.** تابع بازگشتی فیبوناچی را بنویسید (فیبوناچی(ن- + فیبوناچی(ن-۱) = فیبوناچی(ن) (۲) با شرط توقف مناسب.
- تمرین ۴.۶.** تابعی بنویسید که یک عدد بگیرد و بگوید اول است یا نه (راهنمایی: با یک حلقه بررسی کنید که آیا مقسوم‌علیهی غیر از ۱ و خودش دارد).
- تمرین ۵.۶.** تابعی به نام م.م.ب بنویسید که بزرگ‌ترین مقسوم‌علیه مشترک دو عدد را با روش اقلیدس حساب کند.

فصل ۷

آرایه‌ها و بردارها

تا اینجا هر متغیر تنها یک مقدار نگه می‌داشت. اما گاهی به مجموعه‌ای از مقادیر نیاز داریم: نمرات یک کلاس، دماهای یک هفته، نام‌های دوستان. برای این کار از **آرایه** و **بردار** استفاده می‌کنیم. در این فصل هر دو را می‌شناسیم و تفاوتشان را درمی‌یابیم.

۱.۷ آرایه: ردیفی از جعبه‌ها با اندازه ثابت

آرایه را مانند ردیفی از جعبه‌های هم‌نوع تصور کنید که شماره‌گذاری شده‌اند. به این مثال واقعی نگاه کنید:

۱	تابع اصلی:
۲	نمرات صحیح $32[5] = [12, 18, 9, 20, 15]$
۳	متغیر مجموع صحیح $0 = 32$
۴	برای متغیر $i = 0; < i 5; + i = 1$:
۵	مجموع = مجموع + نمرات $[i]$
۶	پایان
۷	چاپ ("مجموع:", مجموع)
۸	چاپ ("میانگین:", مجموع / 5)
۹	پایان

در خط $[12, 18, 9, 20, 15] = 32[5]$ نمرات:

- صحیح $32[5]$ یعنی «آرایه‌ای از ۵ عدد صحیح».
- $[12, 18, 9, 20, 15]$ پنج مقدار اولیه است.

۱.۱.۷ دسترسی به عناصر با اندیس

هر عنصرِ آرایه یک اندیس (شماره) دارد. نکته بسیار مهم: شماره‌گذاری از صفر آغاز می‌شود، نه یک! پس در آرایه بالا:

نمرات[0] = ۱۲، نمرات[1] = ۱۸، ...، نمرات[4] = ۱۵

هشدار

چون اندیس از صفر آغاز می‌شود، در آرایه‌ای با ۵ عنصر، اندیس‌های معتبر ۰ تا ۴ هستند. اگر بکوشید به نمرات[5] دسترسی پیدا کنید، از مرز آرایه بیرون زده‌اید و این یک خطای رایج و خطرناک است. همیشه دقت کنید که اندیس از ۱ - اندازه بیشتر نشود.

۲.۱.۷ طول آرایه با len

برای دانستنِ اندازه یک آرایه، از تابع len استفاده می‌کنیم. این مثال واقعی آن را نشان می‌دهد:

۱	تابع اصلی:
۲	اعداد [5]i32 = [2, 4, 6, 8, 10]
۳	متغیر مجموع i32 = 0
۴	متغیر i i32 = 0
۵	تاوقتی i < len(اعداد):
۶	مجموع = مجموع + اعداد[i]
۷	1 + i = i
۸	پایان
۹	چاپ مجموع، len(اعداد)
۱۰	پایان

با نوشتن len(اعداد) به جای عدد ثابت ۵، اگر بعداً اندازه آرایه را عوض کنید، حلقه همچنان درست کار می‌کند. این عادت خوبی است.

۲.۷ بردار: آرایه‌ای که رشد می‌کند

آرایه‌ها یک محدودیت دارند: اندازه‌شان از همان آغاز ثابت است. اما گاهی نمی‌دانیم چند عنصر خواهیم داشت — مثلاً می‌خواهیم تا وقتی کاربر عدد می‌دهد، آن‌ها را نگه داریم. اینجاست که بردار (Vector) به کار می‌آید: مجموعه‌ای که می‌تواند رشد کند. به این مثال واقعی نگاه کنید:

۱	تابع اصلی:
۲	متغیر اعداد وکتور <صحیح ۳۲> = وکتور {}
۳	برای متغیر $i = 1$; $i \leq 5$; $i = i + 1$:
۴	اعداد. بیفزای $i * i$
۵	پایان
۶	متغیر مجموع صحیح ۳۲ = 0
۷	متغیر z صحیح ۳۲ = 0
۸	تاوقتی $z <$ اعداد. طول():
۹	مجموع = مجموع + اعداد. بگیر $(i)[0]$
۱۰	$z = z + 1$
۱۱	پایان
۱۲	چاپ ("تعداد:", اعداد. طول() بعنوان صحیح ۶۴)
۱۳	چاپ ("مجموع مربع‌ها:", مجموع)
۱۴	پایان

نکته‌های کلیدی:

- وکتور <صحیح ۳۲> یعنی «برداری از اعداد صحیح». بخش درون $<>$ نوعِ عناصر را می‌گوید.
- {} وکتور یک بردار خالی تازه می‌سازد.
- بیفزا(x) (معادلِ push) یک عنصر تازه به انتهای بردار می‌افزاید.
- طول() (معادلِ len) تعدادِ عناصرِ کنونی را می‌دهد.
- بگیر(i) (معادلِ get) عنصرِ شماره‌ی z را برمی‌گرداند. در شیوه‌ی کنونی زبان، برای خواندن مقدار از آن، یک $[0]$ در پی می‌آوریم.

یادداشت

متدها با نقطه (.) پس از نام بردار صدا زده می‌شوند: اعداد، بیفزای (...). این همان شیوه‌ای است که در فصل ساختارها (فصل ۹) برای متدهای دلخواه خودمان هم به کار می‌بریم.

۱۰.۲.۷ همان بردار به انگلیسی

نام‌های متدهای بردار — مانند خود انواع — هم فارسی و هم انگلیسی دارند. این نمونه واقعی همان کار را با نام‌های انگلیسی انجام می‌دهد:

۱	تابع اصلی:
۲	متغیر و <code>Vector = Vector<i32></code> {}
۳	متغیر <code>i = i32 0</code>
۴	تاوقتی <code>i < 4</code> :
۵	<code>push(i) * 10</code>
۶	<code>i = i + 1</code>
۷	پایان
۸	چاپ و <code>len()</code>
۹	متغیر مجموع <code>i32 0 = 0</code>
۱۰	<code>i = 0</code>
۱۱	تاوقتی <code>i < len()</code> :
۱۲	مجموع = مجموع + <code>get(i)[0]</code>
۱۳	<code>i = i + 1</code>
۱۴	پایان
۱۵	چاپ مجموع
۱۶	و <code>free()</code>
۱۷	پایان

در پایان کار، و `free()` (معادل آزاد) حافظه بردار را آزاد می‌کند. این کار برای بردارهایی که عمرشان به پایان رسیده، یک عادت پاکیزه است.

۲۰.۲.۷ بردار به مثابه پشته

یک کاربرد زیبا و رایج بردار، ساختن پشته (stack) است: ساختاری که «آخرین چیزی که گذاشتی، نخستین چیزی است که برمی‌داری.» این مثال واقعی، با افزودن و برداشتن،

یک پشته می‌سازد:

۱	تابع اصلی:
۲	متغیر پشته وکتور <صحیح ۳۲> = وکتور {}
۳	پشته. بیفزا (1)
۴	پشته. بیفزا (2)
۵	پشته. بیفزا (3)
۶	چاپ "اندازه"، پشته. طول () بعنوان صحیح ۳۲
۷	تاوقتی پشته. طول () > 0:
۸	چاپ "برداشت"، پشته. دریاور ()
۹	پایان
۱۰	چاپ "اندازه"، پشته. طول () بعنوان صحیح ۳۲
۱۱	پایان

اینجا متد تازه‌ای می‌بینیم: دریاور () (معادل pop) که آخرین عنصر را برمی‌دارد و برمی‌گرداند. خروجی چنین است: نخست اندازه ۳، سپس برداشت ۳ و ۲ و ۱ (به ترتیب معکوس)، و در آخر اندازه ۰.

۳.۷ آرایه یا بردار؟ کدام را برگزینیم؟

آرایه (T[N])	بردار (وکتور <T>)
اندازه ثابت	اندازه رشدپذیر
وقتی تعداد را از پیش می‌دانید	وقتی تعداد نامعلوم است
ساده و سریع	انعطاف‌پذیر
نیاز به آزادسازی دستی ندارد	بهتر است با آزاد() آزاد شود

نکته

قاعده سرانگشتی: اگر تعداد عناصر را پیش از اجرا می‌دانید، آرایه برگزینید. اگر نه — مثلاً داده‌ها را یکی‌یکی از کاربر یا از یک فایل می‌گیرید — بردار را.

نگاهی به درون: بردار چگونه رشد می‌کند (اختیاری، برای خوانندگان پیشرفته)

بردار درون خود یک آرایه نگه می‌دارد. وقتی این آرایه پُر می‌شود و شما عنصر تازه‌ای می‌افزایید، بردار آرایه‌ای بزرگ‌تر می‌سازد (معمولاً دوبرابر)، عناصر را به آن می‌کوچاند و آرایه کهنه را آزاد می‌کند. به همین دلیل افزودن یک عنصر «معمولاً» سریع است، اما گاه‌به‌گاه که آرایه باید بزرگ شود، کمی کندتر می‌شود. این رفتار — که به آن «بزرگ‌سازی سرشکن‌شده» می‌گویند — در عمل بسیار کارآمد است.

۴.۷ خلاصه فصل

- آرایه مجموعه‌ای هم‌نوع با اندازه ثابت است: $T[N]$.
- اندیس‌ها از صفر آغاز می‌شوند.
- len (آرایه) طول آرایه را می‌دهد.
- بردار (وکتور $\langle T \rangle$) مجموعه‌ای رشدپذیر است.
- متدهای بردار: push /بافز، get /بگیر، len /طول، pop /دریافت، free /آزاد.

۵.۷ تمرین‌ها

- تمرین ۱.۷.** آرایه‌ای از ۶ عدد بسازید و بزرگ‌ترین آن‌ها را با یک حلقه پیدا کنید.
- تمرین ۲.۷.** آرایه‌ای از دماهای یک هفته بسازید و میانگینشان را حساب کنید.
- تمرین ۳.۷.** برداری بسازید و اعداد ۱ تا ۱۰ را در آن بریزید، سپس فقط اعداد زوج را چاپ کنید.
- تمرین ۴.۷.** با کمک یک بردار به‌عنوان پشته، یک رشته را وارونه کنید (هر نویسه را در پشته بگذارید و سپس یکی‌یکی بردارید).
- تمرین ۵.۷.** برنامه‌ای بنویسید که آرایه‌ای از اعداد را بگیرد و تعداد عناصر بزرگ‌تر از میانگین را بشمارد.

فصل ۸

رشته‌ها و کاراکترها

متن همه‌جا هست: نام‌ها، پیام‌ها، خروجی برنامه‌ها. در این فصل عمیق‌تر با رشته‌ها کار می‌کنیم: آن‌ها را به هم می‌چسبانیم، تکه‌تکه می‌کنیم، می‌شماریم و دگرگون می‌کنیم.

۱.۸ یادآوری: رشته چیست؟

رشته دنباله‌ای از نویسه‌هاست که میان دو نقل قول نوشته می‌شود: دنیا "سلام". هر رشته از کنار هم قرار گرفتن نویسه‌ها (کاراکترها) ساخته می‌شود. سلام از یونی‌کد پشتیبانی می‌کند، پس می‌توانید در رشته‌ها از فارسی، انگلیسی، نشانه‌ها و حتی شکلک‌ها استفاده کنید.

۲.۸ چسباندن رشته‌ها

ساده‌ترین کار با رشته‌ها، چسباندن آن‌هاست. دو راه دارید: عملگر + یا متد پیوست(). این مثال واقعی هر دو راه را در عمل نشان می‌دهد:

۱	// وارونه‌سازی رشته
۲	تابع اصلی:
۳	س رشته = "Salam"
۴	متغیر r رشته = ""
۵	متغیر i صحیح ۳۲ = s.len() - 1
۶	تا وقتی $i \geq 0$:
۷	$r = \text{concat}(s.\text{substr}(i), r)$
۸	$i = i - 1$
۹	پایان
۱۰	چاپ "وارونه‌ی", "س", "می‌شود", r
۱۱	پایان

این برنامه یک رشته را وارونه می‌کند. بیایید بشکافیمش:

- $s.\text{len}()$ طول رشته را می‌دهد؛ پس $s.\text{len}() - 1$ اندیس آخرین نویسه است.
 - حلقه از آخرین نویسه تا نخستین ($i \geq 0$) عقب می‌رود.
 - $s.\text{substr}(i, 1)$ (معادل 1 زیررشته i) یک زیررشته به طول ۱ از موقعیت i می‌گیرد — یعنی تنها یک نویسه.
 - $\text{concat}(\dots)$ (معادل پیوست $\dots$) آن نویسه را به انتهای r می‌چسباند.
- خروجی: malaS می‌شود Salam وارونه‌ی.

یادداشت

متدهای رشته نیز — مانند متدهای بردار — هم نام فارسی دارند و هم انگلیسی: $\text{len}/\text{طول}$ ، $\text{substr}/\text{زیررشته}$ ، $\text{concat}/\text{پیوست}$. در یک برنامه بهتر است یک شیوه را برگزینید و به آن پایبند بمانید.

۳.۸ پیمایش نویسه‌به‌نویسه

بسیاری از کارهای متنی نیازمند آن‌اند که رشته را نویسه‌به‌نویسه بررسی کنیم. الگوی آن همیشه یکسان است: یک حلقه از ۰ تا $s.\text{len}() - 1$ ، و در هر گام گرفتن یک زیررشته تک‌نویسه‌ای. این مثال واقعی مصوت‌های یک متن را می‌شمارد:

<code>fa LANG: //</code>	۱
تابع اصلی:	۲
س رشته = <code>"salam" donya</code>	۳
متغیر تعداد صحیح ۳۲ = 0	۴
برای متغیر <code>i = 0</code> ; <code>< س.طول()</code> ; <code>i = i + 1</code> :	۵
ک رشته = <code>س.زیررشته(i, 1)</code>	۶
اگر <code>ک == "a"</code> <code>ک == "e"</code> <code>ک == "i"</code> <code>ک == "o"</code> <code>ک == "u"</code> :	۷
تعداد = تعداد + 1	۸
پایان	۹
پایان	۱۰
چاپ "تعداد مصوت‌ها در", س, "=", تعداد	۱۱
پایان	۱۲

این الگو — «برای هر نویسه، کاری انجام بده» — را در شمارش، جست‌وجو، فیلتر کردن و بسیاری کارهای دیگر به کار خواهید برد.

هشدار

برای مقایسه دو رشته از `==` استفاده کنید، نه از کارهای دیگر. در `"a" == "a"` درست است و دو رشته را نویسه‌به‌نویسه می‌سنجد. به یاد داشته باشید که مقایسه به بزرگی و کوچکی حروف حساس است: `"A"` با `"a"` برابر نیست.

۴.۸ ابزارهای آماده کتابخانه رشته

نوشتن همه کارهای متنی از صفر خسته‌کننده است. خوشبختانه سلام یک کتابخانه آماده رشته دارد که با `"str"` وارد کردن در دسترس قرار می‌گیرد. به این مثال واقعی نگاه کنید:

۱	<code>fa LANG: //</code>
۲	وارد کردن "str"
۳	تابع اصلی:
۴	ثابت متن رشته = <code>Donya "Salam"</code>
۵	چاپ ("طول:", رشته.طول(متن))
۶	چاپ ("بزرگ:", رشته.بزرگ(متن))
۷	چاپ ("کوچک:", رشته.کوچک(متن))
۸	چاپ ("وارونه:", رشته.وارونه("Salam"))
۹	چاپ ("پیوست:", رشته.پیوست("سلام", "دنیا"))
۱۰	پایان

با "str" وارد کردن ماژول رشته را وارد می‌کنیم و سپس با نام فارسی آن، رشته، به توابعش دسترسی پیدا می‌کنیم. مهم‌ترین توابع:

فارسی	انگلیسی	کار
رشته.طول(م)	str.Len	طول رشته
رشته.بزرگ(م)	str.Upper	تبدیل به حروف بزرگ
رشته.کوچک(م)	str.Lower	تبدیل به حروف کوچک
رشته.وارونه(م)	str.Reverse	وارونه‌سازی
رشته.پیوست(آ, ب)	str.Concat	چسباندن دو رشته

یادداشت

دو شیوه کار با رشته را دیدیم: یکی به صورت متد (مثل متن.طول()) و دیگری به صورت تابع ماژول (مثل رشته.طول(متن)). هر دو درست‌اند؛ ماژول str توابع پیشرفته‌تری (مانند جداسازی، جست‌وجو و تبدیل به عدد) هم دارد که در فصل کتابخانه استاندارد (فصل ۱۵) بیشتر می‌بینیم.

۵.۸ دنباله‌های گریز

گاهی می‌خواهیم نویسه‌هایی در رشته بگذاریم که نوشتنشان مستقیم دشوار است — مثلاً خود نقل قول، یا رفتن به خط تازه. برای این کار از دنباله‌های گریز استفاده می‌کنیم که با یک \ (بک اسلش) آغاز می‌شوند:

معنا	دنباله
خط تازه	\n
تب (فاصله افقی)	\t
نقل قول دوتایی	\"
خودِ بک اسلش	\\

برای نمونه، دو "یک\nخط" خط چاپ دو خط چاپ می‌کند، چون \n میان آن‌ها یک شکست خط می‌گذارد.

۶.۸ کاراکتر در برابر رشته

در فصل ۲ دیدیم که کاراکتر ('A') و رشته ("A") دو چیز متفاوت‌اند. کاراکتر تنها یک نویسه است و میان تک نقل قول می‌آید، حال آنکه رشته می‌تواند هیچ، یک، یا هزاران نویسه داشته باشد و میان نقل قول دوتایی می‌آید. در عمل، برای کار با متن بیشتر با رشته‌ها سروکار دارید؛ کاراکتر بیشتر در کارهای سطح پایین به کار می‌آید.

۷.۸ مثال کامل: شمارش واژه‌ها

بیاید آموخته‌ها را در یک برنامه کوچک به کار ببندیم که نویسه‌های یک رشته را می‌شمارد و تعداد فاصله‌ها را نیز می‌یابد — که تقریباً برابر تعداد واژه‌هاست:

<code>fa LANG: //</code>	۱
<code>تابع اصلی:</code>	۲
<code>متن رشته = "سلام دنیا و سلام بر شما"</code>	۳
<code>متغیر فاصله صحیح ۰ = ۳۲</code>	۴
<code>برای متغیر i = ۰; i < متن.طول(); i = i + 1:</code>	۵
<code>اگر متن.زیررشته(i), (1 == ":</code>	۶
<code>فاصله = فاصله + 1</code>	۷
<code>پایان</code>	۸
<code>پایان</code>	۹
<code>چاپ "تعداد نویسه‌ها: ", متن.طول()</code>	۱۰
<code>چاپ "تعداد واژه‌ها: ", فاصله + 1</code>	۱۱
<code>پایان</code>	۱۲

۸.۸ خلاصه فصل

- رشته دنباله‌ای از نویسه‌هاست و از یونی‌کد پشتیبانی می‌کند.
- با + یا پیوست () رشته‌ها را به هم می‌چسبانیم.
- طول () طول و n زیررشته (i)، یک تکه از رشته را می‌دهد.
- الگوی پیمایش نویسه‌به‌نویسه برای شمارش و جست‌وجو بسیار کاربردی است.
- مازول str توابع آماده‌ای مانند بزرگ، کوچک و وارونه دارد.
- دنباله‌های گریز مانند \n نویسه‌های ویژه را در رشته می‌گنجانند.

۹.۸ تمرین‌ها

- تمرین ۱.۸.** برنامه‌ای بنویسید که یک رشته بگیرد و بگوید چند نویسه دارد.
- تمرین ۲.۸.** با کمک مازول str، یک نام را هم با حروف بزرگ و هم با حروف کوچک چاپ کنید.
- تمرین ۳.۸.** برنامه‌ای بنویسید که بشمارد حرف «ا» چند بار در یک متن فارسی آمده است.
- تمرین ۴.۸.** یک رشته را وارونه کنید (یک بار با حلقه دستی و یک بار با رشته وارونه) و نتیجه‌ها را مقایسه کنید.
- تمرین ۵.۸.** برنامه‌ای بنویسید که یک رشته را بگیرد و بررسی کند آیا «متقارن» (palin-drome) است — یعنی از چپ و راست یکسان خوانده می‌شود.

فصل ۹

ساختارها و شمارشی‌ها

تا اینجا با داده‌های ساده (عدد، رشته، منطقی) و مجموعه‌های هم‌نوع (آرایه، بردار) کار کردیم. اما دنیای واقعی پیچیده‌تر است: یک «دانشجو» هم نام دارد، هم شماره، هم معدل. چگونه این‌ها را با هم نگه داریم؟ پاسخ، **ساختار** است. در این فصل ساختارها و خویشاوندشان، **شمارشی‌ها**، را می‌شناسیم.

۱.۹ ساختار: ساختن نوع دلخواه خود

ساختار به شما اجازه می‌دهد چند داده را زیر یک نام گرد آورید و یک نوع تازه بسازید. به این مثال واقعی نگاه کنید که یک «مستطیل» می‌سازد:

۱	ساختار مستطیل:
۲	طول اعشاری ۶۴ = 0.0
۳	عرض اعشاری ۶۴ = 0.0
۴	تابع مساحت () اعشاری ۶۴:
۵	بازگشت این.طول * این.عرض
۶	پایان
۷	تابع محیط () اعشاری ۶۴:
۸	بازگشت 0.2 * (این.طول + این.عرض)
۹	پایان
۱۰	پایان
۱۱	تابع اصلی:
۱۲	متغیر r مستطیل = مستطیل { طول = 0.4, عرض = 0.3 }
۱۳	چاپ ("مساحت:", () مساحت. r.
۱۴	چاپ ("محیط:", () محیط. r.
۱۵	پایان

این مثال چند مفهوم تازه دارد که یکی یکی می‌شکافیم.

۱.۱.۹ میدان‌ها (fields)

۱	ساختار مستطیل:
۲	طول اعشاری ۶۴ = 0.0
۳	عرض اعشاری ۶۴ = 0.0

طول و عرض میدان‌های ساختار هستند — یعنی داده‌هایی که هر مستطیل نگه می‌دارد. هر میدان نامی، نوعی و یک مقدار پیش‌فرض دارد. مقدار پیش‌فرض (0.0) وقتی به کار می‌آید که هنگام ساختن مستطیل، مقداری برای آن میدان ندهید.

۲.۱.۹ ساختن یک نمونه

۱	متغیر r مستطیل = مستطیل { طول = 0.4, عرض = 0.3 }
---	--

این خط یک نمونه (instance) از مستطیل می‌سازد. نحو { ... } مستطیل یک مستطیل تازه با میدان‌های مشخص شده می‌سازد. حالا r یک مستطیل واقعی با طول ۴ و عرض ۳ است.

۳.۱.۹ متدها: توابع درون ساختار

۱	تابع مساحت () اعشاری ۶۴:
۲	بازگشت این.طول * این.عرض
۳	پایان

تابعی که درون یک ساختار تعریف شود، متد نام دارد. متدها روی داده‌های همان ساختار کار می‌کنند. واژه کلیدی این (معادل this) به همان نمونه‌ای اشاره می‌کند که متد رویش صدا زده شده. پس این.طول یعنی «طول همین مستطیل».

متدها را با نقطه پس از نام نمونه صدا می‌زنیم: () مساحت.۲. این دقیقاً همان شیوه‌ای است که در فصل بردارها برای .بیفزا() و .طول() دیدیم — و حالا می‌دانید که آن‌ها هم متد بودند.

۲.۹ تغییر داده‌ها از درون متد

متدها فقط نمی‌خوانند؛ می‌توانند داده‌های ساختار را تغییر هم بدهند. این مثال واقعی یک «شمارنده» می‌سازد:

۱	ساختار شمارنده:
۲	مقدار $i32 = 0$
۳	تابع افزایش():
۴	این.مقدار = این.مقدار + 1
۵	پایان
۶	تابع بگیر() $i32$:
۷	بازگشت این.مقدار
۸	پایان
۹	پایان
۱۰	تابع اصلی:
۱۱	متغیر ش شمارنده = شمارنده { مقدار = 5 }
۱۲	ش.افزایش()
۱۳	ش.افزایش()
۱۴	چاپ ش.بگیر()
۱۵	پایان

متد افزایش مقدار درونی شمارنده را یکی زیاد می‌کند. پس از دو بار صدا زدن، مقدار

از ۵ به ۷ می‌رسد و خروجی ۷ می‌شود. این الگو — نگاه داشتن داده در ساختار و تغییر آن از راه متدها — بنیاد برنامه‌نویسی شیء‌گراست.

۳.۹ مثال کامل: حساب بانکی

این مثال واقعی همهٔ آموخته‌ها را گرد می‌آورد و یک حساب بانکی ساده می‌سازد که واریز و برداشت دارد:

۱	ساختار حساب:
۲	موجودی صحیح $0 = ۳۲$
۳	تابع واریز (مبلغ صحیح ۳۲): این.موجودی = این.موجودی + مبلغ پایان
۴	تابع برداشت (مبلغ صحیح ۳۲):
۵	اگر مبلغ $\leq$ این.موجودی: این.موجودی = این.موجودی - مبلغ
۶	وگرنه:
۷	چاپ "موجودی کافی نیست"
۸	پایان
۹	پایان
۱۰	تابع اصلی:
۱۱	متغیر ح حساب = حساب { موجودی = 100 }
۱۲	ح.واریز(50)
۱۳	ح.برداشت(30)
۱۴	ح.برداشت(200)
۱۵	چاپ "موجودی"، "ح.موجودی"
۱۶	پایان

دقت کنید که متد برداشت پیش از کم کردن، بررسی می‌کند که موجودی کافی باشد. این یک قاعدهٔ مهم در طراحی ساختارهاست: متدها از درستی داده‌ها پاسداری می‌کنند. برداشت ۲۰۰ از حسابی که فقط ۱۲۰ دارد، رد می‌شود و خروجی نهایی $120 =$ موجودی است.

نکته

به دو شیوهٔ دسترسی به میدان دقت کنید: از بیرون ساختار با نام نمونه (ح.موجودی)، و از درون متد با این (این.موجودی). هر دو به همان داده اشاره می‌کنند.

۴.۹ عملگرها برای ساختارها

در فصل ۳ به اختصار دیدیم که می‌توان عملگرها را برای ساختارها تعریف کرد. اکنون که ساختارها را می‌شناسید، این مثال کامل‌تر روشن‌تر است:

۱	ساختار بردار:
۲	ایکس صحیح $0 = ۳۲$
۳	وای صحیح $0 = ۳۲$
۴	تابع عملگر + (دیگری بردار) بردار:
۵	بازگشت بردار { ایکس = این.ایکس + دیگری.ایکس، وای = این.وای +
۶	دیگری.وای }
۷	پایان
۸	تابع عملگر == (دیگری بردار) منطقی:
۹	بازگشت این.ایکس == دیگری.ایکس && این.وای == دیگری.وای
۱۰	پایان
۱۱	پایان
۱۲	تابع اصلی:
۱۳	متغیر الف بردار = بردار { ایکس = 1، وای = 2 }
۱۴	متغیر ب بردار = بردار { ایکس = 3، وای = 4 }
۱۵	متغیر ج بردار = الف + ب
۱۶	چاپ (ج.ایکس، ج.وای)
۱۷	چاپ (الف == ب)
۱۸	چاپ (ج == بردار { ایکس = 4، وای = 6 })
	پایان

با تعریف عملگر + و عملگر ==، حالا ب + الف و ب == الف روی بردارها معنا پیدا می‌کنند. خروجی به ترتیب: 4 6، false و true.

یادداشت

سلام بسیاری عملگر را برای ساختارها می‌پذیرد: حسابی ($+$ $-$ $*$ $/$ $\%$ $**$)، مقایسه‌ای ($==$ $<$ $>$ $<=$ $>=$)، یکانی ($!$ و $-$) و حتی اندیس‌گذاری ($[]$). اگر $==$ را تعریف کنید اما $!=$ را نه، سلام به‌طور خودکار $!=$ را از روی $==$ می‌سازد.

۵.۹ شمارشی: مجموعه‌ای از نام‌های ثابت

گاهی متغیری داریم که فقط چند مقدار مشخص می‌تواند بگیرد: روزهای هفته، جهت‌های اصلی، رنگ‌های چراغ راهنما. برای این کار، شمارشی (enum) را داریم. به این مثال واقعی نگاه کنید:

شمارش رنگ:	۱
قرمز	۲
سبز = 5	۳
آبی	۴
پایان	۵
تابع اصلی:	۶
چاپ (رنگ. قرمز بعنوان صحیح ۳۲)	۷
چاپ (رنگ. سبز بعنوان صحیح ۳۲)	۸
چاپ (رنگ. آبی بعنوان صحیح ۳۲)	۹
پایان	۱۰

رنگ شمارش یک نوع تازه می‌سازد که سه مقدار ممکن دارد: قرمز، سبز و آبی. هر عضو شمارشی در پس پرده یک عدد صحیح است که از صفر آغاز می‌شود. اما می‌توانید مقدار دلخواه هم بدهید: اینجا $5 = \text{سبز}$ گذاشته‌ایم، پس قرمز برابر ۰، سبز برابر ۵ و آبی برابر ۶ (یکی پس از سبز) می‌شود. با صحیح ۳۲ بعنوان می‌توان عدد پشت هر عضو را دید.

۱.۵.۹ شکل یک خطی و کاربرد واقعی

شمارشی‌ها را می‌توان فشرده‌تر هم نوشت. این مثال واقعی «روزهای هفته» شکل یک خطی را نشان می‌دهد و شمارشی را در یک تابع به کار می‌برد:

۱	شمارش روز: دوشنبه، سه‌شنبه، چهارشنبه، پنجشنبه، جمعه، شنبه، یکشنبه پایان
۲	تابع آخر هفته_ است (ر روز) منطقی: بازگشت ر == روز. شنبه ر == روز. یکشنبه پایان
۳	تابع اصلی:
۴	ر روز = روز. شنبه
۵	چاپ "شماره‌ی روز"، " = ر بعنوان صحیح ۳۲
۶	اگر آخر هفته_ است (ر): چاپ "آخر هفته است"
۷	وگرنه: چاپ "روز کاری است"
۸	پایان
۹	پایان

اینجا تابع آخر هفته_ است یک روز می‌گیرد و با مقایسه آن با روز. شنبه و روز. یکشنبه، تصمیم می‌گیرد. شمارشی‌ها کد را خواناتر و ایمن‌تر می‌کنند: به جای عددهای جادویی بی‌معنا (۵، ۶)، نام‌های گویا (شنبه، یکشنبه) به کار می‌بریم.

نکته

هر جا متغیری دارید که فقط چند حالت مشخص می‌گیرد — وضعیت سفارش (در انتظار، ارسال‌شده، تحویل‌شده)، نوع کاربر (مهمان، عضو، مدیر) — یک شمارشی بسازید. این کار از خطاهای فراوان جلوگیری می‌کند، چون کامپایلر نمی‌گذارد مقداری بیرون از فهرست به آن متغیر بدهید.

۶.۹ خلاصه فصل

- ساختار چند داده را زیر یک نام گرد می‌آورد و نوعی تازه می‌سازد.
- میدان‌ها داده‌های ساختارند و می‌توانند مقدار پیش‌فرض داشته باشند.
- متدها توابع درون ساختارند و با این به نمونه جاری دسترسی دارند.
- نمونه را با نحو { مقدار = میدان } نام می‌سازیم.
- عملگرها را می‌توان برای ساختارها تعریف کرد.
- شمارشی مجموعه‌ای از نام‌های ثابت است که کد را خوانا و ایمن می‌کند.

۷.۹ تمرین‌ها

تمرین ۱.۹. ساختاری به نام دایره با میدان شعاع بسازید و متدهای مساحت و محیط را برایش بنویسید (از $3.1416 \approx \pi$ استفاده کنید).

تمرین ۲.۹. ساختار دانشجو با میدان‌های نام، شماره و معدل بسازید و متدی بنویسید که بگوید دانشجو مشروط است یا نه (معدل زیر ۱۲).

تمرین ۳.۹. شمارشی جهت با اعضای شمال، جنوب، شرق و غرب بسازید و تابعی بنویسید که جهت مقابل هر جهت را برگرداند.

تمرین ۴.۹. به ساختار بردار متد عملگر- (تفریق) را نیز بیفزایید.

تمرین ۵.۹. ساختار حساب را گسترش دهید تا تاریخچه تراکنش‌ها را در یک بردار نگه دارد.

فصل ۱۰

واسط‌ها و چندریختی

در فصل ۹ دیدیم که هر ساختار می‌تواند متدهای خود را داشته باشد. اما گاهی چند ساختار متفاوت، رفتار مشترکی دارند: هم مربع و هم دایره «مساحت» دارند، هر چند فرمولشان فرق می‌کند. چگونه می‌توان با همه آن‌ها به یک شیوه یکسان کار کرد؟ پاسخ، واسط است.

۱.۱۰ مسئله: رفتار مشترک، پیاده‌سازی متفاوت

فرض کنید چند شکل هندسی دارید و می‌خواهید تابعی بنویسید که مساحت هر شکلی را چاپ کند — بی‌آنکه برای هر شکل، تابعی جدا بنویسید. مشکل اینجاست که مربع و مستطیل دو نوع متفاوت‌اند؛ یک تابع چگونه می‌تواند هر دو را بپذیرد؟
واسط (interface) راه حل است: قراردادی که می‌گویید «هر نوعی که این متدها را داشته باشد، یک شکل به‌شمار می‌آید.»

۲.۱۰ تعریف و پیاده‌سازی واسط

به این مثال واقعی نگاه کنید که قرارداد «شکل» را تعریف و سپس دو نوع را با آن هماهنگ می‌کند:

۱	واسط شکل:
۲	تابع مساحت () صحیح ۳۲
۳	تابع نام () رشته
۴	پایان
۵	ساختار مربع:
۶	ضلع صحیح ۳۲ = 0
۷	تابع مساحت () صحیح ۳۲: بازگشت این.ضلع * این.ضلع پایان
۸	تابع نام () رشته: بازگشت "مربع" پایان
۹	پایان
۱۰	ساختار مستطیل:
۱۱	طول صحیح ۳۲ = 0
۱۲	عرض صحیح ۳۲ = 0
۱۳	تابع مساحت () صحیح ۳۲: بازگشت این.طول * این.عرض پایان
۱۴	تابع نام () رشته: بازگشت "مستطیل" پایان
۱۵	پایان
۱۶	تابع توصیف (ش dyn شکل):
۱۷	چاپ (ش.نام), ">", "ش.مساحت"()
۱۸	پایان
۱۹	تابع اصلی:
۲۰	توصیف (مربع { ضلع = 5 })
۲۱	توصیف (مستطیل { طول = 4, عرض = 6 })
۲۲	پایان

این مثال چند بخش کلیدی دارد:

۱۰.۲.۱۰ اعلان واسط

۱	واسط شکل:
۲	تابع مساحت () صحیح ۳۲
۳	تابع نام () رشته
۴	پایان

شکل واسط می‌گوید: «هر چیزی که بخواهد یک شکل باشد، باید دو متد داشته باشد: مساحت که یک عدد می‌دهد، و نام که یک رشته می‌دهد.» دقت کنید که در واسط فقط امضای متدها آمده، نه بدنه‌شان. واسط می‌گوید چه رفتاری لازم است، نه چگونه.

۲.۲.۱۰ پیاده‌سازی واسط

مربع و مستطیل هر دو متدهای مساحت و نام را دارند، پس هر دو واسطِ شکل را برآورده می‌کنند. در سلام لازم نیست صریحاً بنویسید «مربع از شکل پیروی می‌کند»؛ همین که متدهای لازم را داشته باشد، کافی است.

۳.۲.۱۰ استفاده همگانی با dyn

۱	تابع توصیف (ش dyn شکل):
۲	چاپ (ش. نام)، ">" ش. مساحت()
۳	پایان

اینجا جادو رخ می‌دهد. تابع توصیف یک پارامتر از نوع شکل dyn می‌گیرد — یعنی «هر چیزی که یک شکل باشد». حالا می‌توانیم هم مربع و هم مستطیل را به آن بدهیم و هر کدام متد خودش را اجرا می‌کند. خروجی:

۱	مربع
۲	مساحت 25 ->
۳	24 ->

به این توانایی — اینکه یک تابع با انواع گوناگون کار کند و هر نوع رفتار خود را نشان دهد — چندریختی (polymorphism) می‌گویند؛ واژه‌ای یونانی به معنای «چند شکلی».

نگاهی به درون: dyn چگونه کار می‌کند (اختیاری، برای خوانندگان پیشرفته)

وقتی یک مقدار را به صورت شکل dyn می‌فرستید، سلام در پس پرده یک «اشاره‌گر چاق» (fat pointer) می‌سازد: یک اشاره به خود داده، و یک اشاره به جدولی از متدهای آن نوع (که به آن جدول مجازی یا vtable می‌گویند). وقتی ش. مساحت() را صدا می‌زنید، سلام در زمان اجرا از این جدول، متد درست همان نوع را پیدا و اجرا می‌کند. به همین دلیل به dyn «آرسال پویا» (dynamic dispatch) می‌گویند: تصمیم اینکه کدام متد اجرا شود، در زمان اجرا گرفته می‌شود، نه زمان کامپایل.

۳.۱۰ چرا واسط‌ها ارزشمندند؟

تصور کنید فردا بخواهید شکل تازه‌ای — مثلاً «مثلث» — بیفزایید. کافی است یک ساختارِ مثلث با متدهای مساحت و نام بسازید. تابعِ توصیفِ بدونِ هیچِ تغییری با مثلث هم کار می‌کند! این یعنی کد شما گسترش‌پذیر است: می‌توانید انواع تازه بیفزایید بی‌آنکه کد موجود را دست بزنید.

نکته

واسط‌ها به شما اجازه می‌دهند کد را بر پایه رفتار بنویسید، نه بر پایه نوع مشخص. این یکی از مهم‌ترین ابزارهای طراحی نرم‌افزارِ بزرگ و قابل نگه‌داری است. هر جا دیدید که یک تابع را برای چند نوع مشابه تکرار می‌کنید، احتمالاً یک واسط می‌تواند آن تکرار را از میان بردارد.

۴.۱۰ واسط در برابر کلی‌سازی

سلام دو راه برای نوشتنِ کدی که با انواع گوناگون کار کند دارد: واسط (این فصل) و کلی‌سازی (فصل بعد). تفاوتشان ظریف اما مهم است:

- **واسط با dyn:** انعطاف‌پذیر است؛ می‌توانید انواع مختلف را در یک بردار بریزید و در زمان اجرا تصمیم بگیرید. اما اندکی هزینه‌ی اجرایی دارد (به‌خاطر همان جدول مجازی).

- **کلی‌سازی:** در زمان کامپایل به کد مخصوص هر نوع تبدیل می‌شود، پس هیچ هزینه‌ی اجرایی ندارد و سریع‌تر است؛ اما کمی انعطاف کمتری دارد.

در فصل بعد کلی‌سازی را خواهیم دید و آن‌گاه می‌توانید آگاهانه میان این دو انتخاب کنید.

۵.۱۰ خلاصه فصل

- واسط قراردادی است که می‌گوید یک نوع چه متدهایی باید داشته باشد.

- هر ساختاری که متدهای لازم را داشته باشد، واسط را برآورده می‌کند — بی‌نیاز به اعلان صریح.
- با واسط dyn می‌توان تابعی نوشت که با هر نوع سازگار کار کند.
- چندریختی یعنی یک تابع با انواع گوناگون کار کند و هر نوع رفتار خود را نشان دهد.
- واسط‌ها کد را گسترش‌پذیر می‌کنند.

۶.۱۰ تمرین‌ها

- تمرین ۱.۱۰.** واسط شکل را گسترش دهید و یک ساختار دایره بیفزایید که آن را برآورده کند، سپس آن را به توصیف بدهید.
- تمرین ۲.۱۰.** واسطی به نام جاندار با متد رشته صدا() بسازید و ساختارهای گربه و سگ را برایش پیاده کنید.
- تمرین ۳.۱۰.** تابعی بنویسید که یک شکل dyn بگیرد و بگوید مساحتش از ۱۰۰ بیشتر است یا نه.
- تمرین ۴.۱۰.** واسطی به نام قابل_چاپ با متد رشته_به_رشته() بسازید و دو نوع متفاوت را برایش پیاده کنید.

فصل ۱۱

کلی سازی

تصور کنید تابعی نوشته‌اید که بزرگ‌ترِ دو عدد صحیح را برمی‌گرداند. حالا همان را برای اعداد اعشاری هم می‌خواهید. آیا باید تابع را دوباره — با همان منطق و فقط نوع متفاوت — بنویسید؟ خوشبختانه نه. **کلی سازی** (generics) به ما اجازه می‌دهد یک بار کدی بنویسیم که با هر نوعی کار کند.

۱.۱۱ مسئله: تکرار یک منطق برای انواع گوناگون

بدون کلی سازی، ناچار بودیم چنین بنویسیم:

تابع بیشینه_صحیح(الف صحیح ۳۲, ب صحیح ۳۲) صحیح ۳۲:	۱
اگر الف > ب: بازگشت الف پایان	۲
بازگشت ب	۳
پایان	۴
تابع بیشینه_اعشاری(الف اعشاری ۶۴, ب اعشاری ۶۴) اعشاری ۶۴:	۵
اگر الف > ب: بازگشت الف پایان	۶
بازگشت ب	۷
پایان	۸

این دو تابع منطق یکسانی دارند و تنها در نوع فرق می‌کنند. این تکرار هم خسته کننده است و هم خطاخیز: اگر روزی منطق را اصلاح کنید، باید در هر دو جا تغییر دهید.

۲.۱۱ راه حل: پارامتر نوع

با کلی سازی، نوع را هم مانند یک پارامتر می گیریم. این مثال واقعی را ببینید:

۱	تابع بیشینه <ت> (الف ت, ب ت) ت:
۲	اگر الف > ب:
۳	بازگشت الف
۴	پایان
۵	بازگشت ب
۶	پایان
۷	ساختار جعبه <ت>:
۸	مقدار ت
۹	پایان
۱۰	تابع اصلی:
۱۱	چاپ (بیشینه 3), (9))
۱۲	چاپ (بیشینه 5.2, 5.1))
۱۳	متغیر ج جعبه <صحیح ۳۲> = جعبه { مقدار = 42 }
۱۴	چاپ (ج. مقدار)
۱۵	پایان

به بیشینه <ت> دقت کنید. آن ت درون <> یک پارامتر نوع است — یک نوع «جای خالی» که هنوز مشخص نیست. سپس در امضای تابع، پارامترها و خروجی همگی از همان نوع ت هستند. معنایش این است: «هر دو ورودی و خروجی، از یک نوع یکسان اند — هر نوعی که باشد.»

حالا وقتی می نویسیم:

• (9 بیشینه 3): سلام می بیند که ورودی ها صحیح اند، پس ت را صحیح می گیرد.

• (1.5 بیشینه 2.5): ورودی ها اعشاری اند، پس ت را اعشاری می گیرد.

یک تابع، دو کاربرد — بی هیچ تکراری!

یادداشت

دقت کنید که هنگام صدا زدن بیشینه، لازم نبود نوع را بنویسیم؛ سلام آن را از روی ورودی ها حدس زد. این را «استنتاج نوع» می گویند و کد را تمیزتر می کند.

۳.۱۱ ساختارهای کلی

نه تنها توابع، بلکه ساختارها هم می توانند کلی باشند. در مثال بالا:

۱	ساختار جعبه <ت>:
۲	مقدار ت
۳	پایان

جعبه <ت> یک ساختار کلی است: جعبه ای که می تواند هر نوعی را در خود نگه دارد. وقتی می نویسیم جعبه <صحیح ۳۲>، یک جعبه مخصوص اعداد صحیح می سازیم. می توانستیم جعبه <رشته> هم بسازیم تا متن نگه دارد.

نکته

بردار (وکتور <T>) و نگاشت (<V> نگاشت <K>) که در فصل های پیش دیدیم، خودشان ساختارهای کلی اند! به همین دلیل می توانستیم هم وکتور <صحیح ۳۲> و هم وکتور <رشته> بسازیم. اکنون می دانید که در پس این انعطاف، همین کلی سازی نهفته است.

۴.۱۱ کلی سازی با چند پارامتر نوع

یک ساختار یا تابع می تواند بیش از یک پارامتر نوع داشته باشد. برای نمونه، یک «جفت» که دو مقدار احتمالاً متفاوت را نگه می دارد:

۱	ساختار جفت <آ، ب>:
۲	اول آ
۳	دوم ب
۴	پایان
۵	تابع اصلی:
۶	متغیر ج جفت <رشته، صحیح ۳۲> = جفت { اول = "سن"، دوم = 30 }
۷	چاپ (ج.اول، ج.دوم)
۸	پایان

اینجا آ و ب دو پارامتر نوع مستقل اند؛ پس می توان جفتی ساخت که عضو نخستش رشته و عضو دومش عدد باشد. همان گونه که <V> نگاشت <K>، از کلید و مقدار متفاوت

بهره می‌برد.

۵.۱۱ چرا کلی‌سازی سریع است؟

نگاهی به درون: تک‌ریخت‌سازی (اختیاری، برای خوانندگان پیشرفته)

سلام کلی‌سازی را با روشی به نام تک‌ریخت‌سازی (monomorphization) پیاده می‌کند. وقتی شما (9 بیشینه 3) و (1.5 بیشینه 2.5) را می‌نویسید، سلام در زمان کامپایل دو نسخه جداگانه از تابع می‌سازد: یکی مخصوص صحیح و یکی مخصوص اعشاری. یعنی همان کدی که در آغاز این فصل دستی نوشتیم، این بار به‌طور خودکار ساخته می‌شود. نتیجه این است که کلی‌سازی در سلام هیچ هزینه اجرایی ندارد: کد نهایی دقیقاً به سرعت کدی است که برای هر نوع جدا نوشته باشید. شما راحتی نوشتن یک تابع را دارید و رایانه سرعت چند تابع تخصصی را. این یکی از زیباترین معامله‌های برنده-برنده در طراحی زبان است.

۶.۱۱ کلی‌سازی در برابر واسط: کدام را برگزینیم؟

اکنون که هر دو ابزار را دیدید، این راهنمای ساده کمکتان می‌کند:

کلی‌سازی را برگزینید اگر...	واسط dyn را برگزینید اگر...
سرعت بسیار مهم است	به انعطاف زمان اجرا نیاز دارید
نوع‌ها در زمان کامپایل معلوم‌اند	می‌خواهید انواع مختلف را در یک بردار بریزید
منطق یکسان برای انواع گوناگون	رفتار مشترک با پیاده‌سازی متفاوت

نکته

قاعده سرانگشتی: اگر شک دارید، کلی‌سازی را برگزینید. در بیشتر موارد سریع‌تر و ایمن‌تر است. تنها وقتی به dyn روی بیاورید که واقعاً به ذخیره انواع متفاوت در کنار هم یا تصمیم‌گیری زمان اجرا نیاز داشته باشید.

۷.۱۱ خلاصه فصل

- کلی سازی به ما اجازه می دهد یک بار کدی بنویسیم که با هر نوعی کار کند.
- پارامتر نوع (مثل ت) درون < > نوشته می شود.
- هم توابع و هم ساختارها می توانند کلی باشند.
- می توان چند پارامتر نوع داشت: ب< جفت> آ.
- سلام نوع را از روی ورودی ها استنتاج می کند.
- کلی سازی با تکریمت سازی پیاده می شود و هیچ هزینه اجرایی ندارد.

۸.۱۱ تمرین ها

- تمرین ۱.۱۱.** تابع کلی کمینه < ت> را بنویسید که کوچک تر دو مقدار را برگرداند.
- تمرین ۲.۱۱.** تابع کلی جابجا < ت> را بنویسید که... (راهنمایی: به مفهوم ارجاع در فصل ۱۳ نیاز دارید؛ فعلاً دو مقدار بگیرید و آن ها را در یک جفت جابه جاشده برگردانید).
- تمرین ۳.۱۱.** ساختار کلی پشته < ت> را با متدهای بیفزا و دریاور بسازید (می توانید درونش از یک بردار استفاده کنید).
- تمرین ۴.۱۱.** ساختار ب< جفت> آ، را گسترش دهید و متدی بیفزایید که جای دو عضو را عوض کند و یک آ< جفت> ب، برگرداند.
- تمرین ۵.۱۱.** توضیح دهید چرا برای ساختن یک «فهرست نام ها و سن ها»، نگاشت بهتر از دو بردار جداگانه است.

فصل ۱۲

بستارها و عبارت‌های لامبدا

تا اینجا توابع را همیشه با نام می‌شناختیم. اما در سلام، تابع خودش یک مقدار است: می‌توان آن را در متغیر گذاشت، به تابع دیگر داد، یا از یک تابع بازگرداند. در این فصل با توابع بی‌نام (لامبدا) و بستارها آشنا می‌شویم — ابزارهایی که برنامه‌نویسی را انعطاف‌پذیرتر می‌کنند.

۱.۱۲ تابع به‌مثابه مقدار

به این مثال واقعی نگاه کنید. اینجا توابع را در متغیر می‌گذاریم و حتی به تابع دیگری می‌دهیم:

تابع اعمال (ف) (صحیح ۳۲) func(صحیح ۳۲), مقدار صحیح ۳۲ صحیح ۳۲:	۱
بازگشت ف(مقدار)	۲
پایان	۳
تابع اصلی:	۴
دو برابر (صحیح ۳۲) func(صحیح ۳۲) = (ن صحیح ۳۲) => ۲ * ن	۵
مربع (صحیح ۳۲) func(صحیح ۳۲) = (ن صحیح ۳۲) => ن * ن	۶
چاپ اعمال (دو برابر), (21)	۷
چاپ اعمال (مربع), (9)	۸
چاپ (دو برابر (مربع 3))	۹
پایان	۱۰

چند چیز تازه اینجا هست:

فصل ۱۲. بستارها و عبارت‌های لامبدا. ۲. بستار: لامبدایی که محیطش را به خاطر می‌سپارد

۱.۱.۱۲ نوع تابع

۱ (صحیح ۳۲) func(صحیح ۳۲)

این یک نوع است — مانند صحیح یا رشته، اما نوع یک تابع. خوانده می‌شود: «تابعی که یک عدد صحیح می‌گیرد و یک عدد صحیح برمی‌گرداند.» پس دوبرابر متغیری است که نه عدد، بلکه یک تابع در آن نگاه‌داشته می‌شود.

۲.۱.۱۲ عبارت لامبدا

۱ (ن صحیح ۳۲) $\Rightarrow n * 2$

این یک لامبدا است: یک تابع کوچک بی‌نام. ساختارش ساده است: درون پرانتز پارامترها، سپس نشانه $\Rightarrow$ ، و سپس عبارتی که نتیجه را می‌دهد. این لامبدا یک عدد می‌گیرد و دوبرابرش را برمی‌گرداند — درست مانند یک تابع معمولی، اما بدون نام و در یک خط.

۳.۱.۱۲ تابعی که تابع می‌گیرد

تابع اعمال در پارامتر نخستش یک تابع می‌گیرد (ف) و آن را روی مقدار اجرا می‌کند. به همین دلیل توانستیم هم دوبرابر و هم مربع را به آن بدهیم. تابعی که تابع دیگر می‌گیرند یا برمی‌گردانند را توابع مرتبه بالاتر می‌نامند و در پردازش داده بسیار کاربرد دارند.

یادداشت

به آخرین خط دقت کنید: دوبرابر(مربع ۳). نخست مربع(۳) برابر ۹، سپس دوبرابر(۹) برابر ۱۸ می‌شود. چون این‌ها متغیرهایی حاوی تابع‌اند، می‌توان مستقیم صدایشان زد — درست مانند توابع معمولی.

۲.۱۲ بستار: لامبدایی که محیطش را به خاطر می‌سپارد

اینجا به یکی از قدرتمندترین ایده‌های برنامه‌نویسی می‌رسیم. یک لامبدا می‌تواند متغیرهای بیرون خودش را «به یاد بسپارد». به چنین لامبدایی بستار (closure)

می‌گویند. این مثال واقعی را ببینید:

۱	تابع سازنده جمع (ن) $i32$: $\text{func}(i32)$
۲	بازگشت $(x \Rightarrow i32)$ $x + n$
۳	پایان
۴	تابع اصلی:
۵	دو برابر $(n = i32 \text{ func}(i32))$ $2 * n \Rightarrow i32$
۶	چاپ دو برابر (21)
۷	جمع ۱۰ خودکار = سازنده جمع (10)
۸	چاپ جمع ۱۰ (5)
۹	پایان

تابع سازنده جمع یک عدد (ن) می‌گیرد و یک لامبدا برمی‌گرداند. اما دقت کنید: آن لامبدا $n + x \Rightarrow i32(x)$ از ن استفاده می‌کند که پارامتر تابع بیرونی است! وقتی سازنده جمع (10) را صدا می‌زنیم، لامبدای بازگشتی مقدار $10 = n$ را با خود نگه می‌دارد. پس جمع ۱۰ (5) برابر $10 + 5 = 15$ می‌شود. این یعنی جمع ۱۰ یک تابع تازه است که «۱۰ را به یاد دارد». اگر سازنده جمع (100) می‌ساختیم، تابعی می‌داشتیم که ۱۰۰ را به یاد می‌آورد.

۳.۱۲ بستار با چند دستور

لامبدهایی که دیدیم یک عبارت داشتند. اما گاهی به چند خط منطق نیاز داریم. برای این، شکل بلندتر لامبدا را داریم که با $\Rightarrow$ باز و با پایان بسته می‌شود. این مثال واقعی یک شمارنده می‌سازد:

۱	تابع ساخت_شمارنده () func() صحیح ۳۲:
۲	متغیر ن صحیح ۳۲ = 0
۳	بازگشت () => :
۴	ن = ن + 1
۵	بازگشت ن
۶	پایان
۷	پایان
۸	تابع اصلی:
۹	بعدی func() صحیح ۳۲ = ساخت_شمارنده ()
۱۰	چاپ بعدی(), بعدی(), بعدی(), بعدی()
۱۱	پایان

اینجا اتفاق جالبی می‌افتد. ساخت_شمارنده متغیر ن را می‌سازد و یک بستار برمی‌گرداند که آن را یکی زیاد و سپس باز می‌گرداند. هر بار که بعدی() را صدا می‌زنیم، همان ن — که میان فراخوانی‌ها زنده می‌ماند — یکی بیشتر می‌شود. پس خروجی 1 2 3 4 است.

نگاهی به درون: بستار حافظه را کجا نگه می‌دارد؟ (اختیاری، برای خوانندگان پیشرفته)

وقتی یک بستار متغیری از محیط بیرون را می‌گیرد، سلام آن متغیر را در پشته نمی‌گذارد (که با پایان تابع از میان می‌رود)، بلکه در پشته آزاد (heap) نگه می‌دارد و یک «محیط بسته‌شده» می‌سازد که همراه خود تابع زندگی می‌کند. به همین دلیل است که ن حتی پس از پایان یافتن ساخت_شمارنده همچنان زنده می‌ماند. این مکانیزم — گرفتن متغیرها و نگه‌داشتنشان روی پشته آزاد — قلب کار بستارهاست.

نکته

بستارها به‌ویژه برای ساختن «کارخانه‌های تابع» (مانند سازنده جمع) و نگه‌داشتن حالت خصوصی (مانند شمارنده) عالی‌اند. در فصل کتابخانه استاندارد خواهیم دید که بسیاری از توابع پردازش داده، لامبدا می‌گیرند تا رفتارشان را سفارشی کنید.

۴.۱۲ خلاصه فصل

- در سلام، تابع یک مقدار است: می‌توان آن را در متغیر گذاشت، پاس داد و بازگرداند.
- نوع تابع چنین نوشته می‌شود: خروجی (ورودی) `func`.
- لامبدا تابعی بی‌نام است: عبارت `=>` (پارامتر).
- برای چند دستور، شکل بلند را به کار می‌بریم: `=> ()` ...پایان.
- بستار لامبدایی است که متغیرهای محیط خود را به یاد می‌سپارد.
- توابع مرتبه بالاتر، تابع دیگر را می‌گیرند یا برمی‌گردانند.

۵.۱۲ تمرین‌ها

- تمرین ۱.۱۲.** یک لامبدا بسازید که عددی بگیرد و مکعبش را برگرداند، و آن را در یک متغیر بگذارید و صدا بزنید.
- تمرین ۲.۱۲.** تابعی به نام سازنده ضرب بنویسید که عددی بگیرد و یک بستار برگرداند که هر ورودی را در آن عدد ضرب کند.
- تمرین ۳.۱۲.** با کمک ساخت شمارنده، دو شمارنده مستقل بسازید و نشان دهید که هر کدام حالت خودش را دارد.
- تمرین ۴.۱۲.** تابع اعمال دوبار را بنویسید که یک تابع و یک مقدار بگیرد و تابع را دو بار پشت سر هم روی مقدار اجرا کند.
- تمرین ۵.۱۲.** بستاری بسازید که مجموع همه اعدادی را که تا کنون به آن داده‌اید نگه دارد و در هر فراخوانی، مجموع تازه را برگرداند.

فصل ۱۳

مدیریتِ خطا و اجرای به تعویق افتاده

برنامه‌های واقعی همیشه در مسیرِ خوش پیش نمی‌روند: فایل ممکن است یافت نشود، ورودی کاربر ممکن است نادرست باشد، یا منبعی که گرفته‌ایم باید آزاد شود. در این فصل می‌آموزیم چگونه برنامه‌ای بنویسیم که با این موقعیت‌ها تمیز و مطمئن برخورد کند.

۱.۱۳ تعلیق: کاری که حتماً باید انجام شود

گاهی کاری هست که باید در پایانِ یک تابع انجام شود — هرچه پیش بیاید. مثلاً اگر فایلی را باز کرده‌اید، باید آن را ببندید؛ اگر حافظه‌ای گرفته‌اید، باید آزادش کنید. واژه تعلیق (معادل defer) دقیقاً برای همین است. به این مثال واقعی کوتاه نگاه کنید:

۱	تابع اصلی:
۲	تعلیق چاپ "پاکسازی"
۳	چاپ "شروع"
۴	چاپ "پایان"
۵	پایان

شاید انتظار داشته باشید خروجی به ترتیب نوشتن باشد، اما چنین نیست:

۱ شروع پای‌ان پاکسازی

دستوری که با تعلیق نشانه‌گذاری می‌شود، بی‌درنگ اجرا نمی‌شود؛ بلکه کنار گذاشته می‌شود تا درست پیش از پایانِ تابع اجرا گردد. پس هرچند «پاکسازی» را در خطِ نخست نوشتیم، در پایان اجرا شد.

۲۰۱۳ چرا تعلیق این قدر مفید است؟

ارزش واقعی تعلیق وقتی روشن می شود که تابع چند راه خروج دارد. این مثال واقعی را ببینید:

۱	تابع کار():
۲	تعلیق چاپ("۳. منبع آزاد شد")
۳	چاپ("۱. شروع کار")
۴	چاپ("۲. انجام کار")
۵	پایان
۶	تابع اصلی:
۷	کار()
۸	چاپ("۴. پایان برنامه")
۹	پایان

خروجی به ترتیب ۱، ۲، ۳، ۴ است. نکته اینجا است: هر مسیری که تابع از آن خارج شود — چه به طور طبیعی، چه با بازگشت زودهنگام — دستور تعلیق اجرا می شود. به همین دلیل، تعلیق بهترین جا برای «آزادسازی منابع» است: کافی است بلافاصله پس از گرفتن منبع، آزادسازی اش را تعلیق کنید و دیگر هرگز نگران فراموش کردنش نباشید.

نکته

الگوی طلایی: هر بار منبعی گرفتید (فایل باز کردید، حافظه گرفتید، قفل گرفتید)، بی درنگ در خط بعد آزادسازی اش را تعلیق کنید. این گونه گرفتن و آزاد کردن کنار هم می مانند و خواندن کد آسان می شود.

۱	// شبه کد
۲	ف = باز_کن("داده.txt")
۳	تعلیق ف.ببند()
۴	// ... هر کاری با ف، حتی با چند بازگشت زودهنگام ...

۳.۱۳ چند تعلیق و ترتیب اجرا

اگر چند تعلیق داشته باشید، به ترتیب معکوس اجرا می‌شوند — یعنی آخرین تعلیق نخست اجرا می‌شود:

۱	تابع اصلی:
۲	تعلیق چاپ "یک"
۳	تعلیق چاپ "دو"
۴	تعلیق چاپ "سه"
۵	چاپ "بدنه"
۶	پایان

خروجی:

۱ بدن سه دویک

نگاهی به درون: چرا ترتیب معکوس است؟ (اختیاری، برای خوانندگان پیشرفته)

تعلیق‌ها روی یک پشته انباشته می‌شوند. همان‌طور که در فصل بردارها دیدیم، در پشته «آخرین چیزی که گذاشتی، نخستین چیزی است که برمی‌داری». این ترتیب معکوس در عمل دقیقاً همان چیزی است که می‌خواهیم: اگر منبع «الف» را پیش از «ب» گرفته‌اید، طبیعی است که «ب» را پیش از «الف» آزاد کنید — درست مثل بازکردن لایه‌های یک پیاز از بیرون به درون.

۴.۱۳ مدیریت خطا با مقدار بازگشتی

سلام رویکردی صریح به خطاها دارد: به جای آنکه خطاها به‌طور پنهان رخ دهند، تابع‌ها معمولاً یک نشانه برمی‌گردانند که می‌گوید کار موفق بود یا نه. ساده‌ترین شکل، بازگرداندن یک مقدار منطقی یا یک کد وضعیت است:

۱	تابع تقسیم امن (الف صحیح ۳۲، ب صحیح ۳۲، نتیجه & صحیح ۳۲) منطقی:
۲	اگر $b == 0$:
۳	بازگشت نادرست
۴	پایان
۵	نتیجه = الف / ب
۶	بازگشت درست
۷	پایان
۸	تابع اصلی:
۹	متغیر خروجی صحیح ۳۲ = 0
۱۰	اگر تقسیم امن (10, 2 خروجی):
۱۱	چاپ "نتیجه:", خروجی
۱۲	وگرنه:
۱۳	چاپ "خطا: تقسیم بر صفر"
۱۴	پایان
۱۵	اگر تقسیم امن (5, 0 خروجی):
۱۶	چاپ "نتیجه:", خروجی
۱۷	وگرنه:
۱۸	چاپ "خطا: تقسیم بر صفر"
۱۹	پایان
۲۰	پایان

اینجا تابع درست یا نادرست برمی‌گرداند تا بگوید موفق شد یا نه، و نتیجه واقعی را از راه پارامتر /ارجاعی & صحیح ۳۲ نتیجه بیرون می‌دهد. (نشانه & یک /ارجاع است که به تابع اجازه می‌دهد متغیر بیرون را تغییر دهد؛ آن را در فصل ۲۰ کامل‌تر می‌بینیم.) فراخواننده موظف است وضعیت را بررسی کند — و همین صراحت، خطاها را آشکار و کنترل‌پذیر می‌کند.

هشدار

در زبان‌هایی که خطاها را پنهان مدیریت می‌کنند، آسان است که فراموش کنید خطایی ممکن است رخ دهد. رویکرد سلام شما را وادار می‌کند که آگاهانه درباره حالت خطا تصمیم بگیرید. این کمی بیشتر کد می‌خواهد، اما برنامه‌هایی می‌سازد که در برابر شرایط ناخواسته مقاوم‌اند.

۵.۱۳ بررسی مرزها و اجرای امن

یکی از رایج‌ترین خطاها، بیرون زدن از مرز آرایه است (که در فصل ۷ هشدارش را دادیم). سلام می‌تواند در حالت امن این خطاها را در زمان اجرا بگیرد. اگر برنامه را با گزینه safe -- بسازید:

```
۱ salam build app.salam --safe --output=app.exe
```

آنگاه هر دسترسی نامعتبر به آرایه، به جای رفتار نامشخص، برنامه را با یک پیام روشن متوقف می‌کند. این برای یافتن خطاها در حین توسعه بسیار ارزشمند است.

۶.۱۳ ترکیب تعلیق و مدیریت خطا

این دو ابزار با هم می‌درخشند. الگوی رایج چنین است: منبع را بگیر، آزادسازی‌اش را تعلیق کن، سپس کارها را انجام بده و در هر خطایی با خیال راحت بازگشت کن — چون می‌دانی تعلیق پاکسازی را تضمین کرده است:

۱	تابع پردازش(داده صحیح ۳۲) منطقی:
۲	تعلیق چاپ "منبع آزاد شد"
۳	اگر داده < 0:
۴	چاپ "ورودی نامعتبر"
۵	بازگشت نادرست
۶	پایان
۷	چاپ "پردازش:", داده
۸	بازگشت درست
۹	پایان
۱۰	تابع اصلی:
۱۱	پردازش(5)
۱۲	چاپ "----"
۱۳	پردازش(1-)
۱۴	پایان

در هر دو فراخوانی — چه موفق و چه ناموفق — پیام «منبع آزاد شد» چاپ می‌شود، چون با تعلیق نشانه‌گذاری شده بود.

۷.۱۳ خلاصه فصل

- تعلیق دستوری را تا پایان تابع به تعویق می‌اندازد.
- هر مسیر خروج از تابع، تعلیق‌ها را اجرا می‌کند — عالی برای آزادسازی منابع.
- چند تعلیق به ترتیب معکوس اجرا می‌شوند.
- سلام خطاها را صریح مدیریت می‌کند: تابع‌ها نشانه موفقیت برمی‌گردانند.
- گزینه safe — خطاهای مرز آرایه را در زمان اجرا می‌گیرد.
- ترکیب تعلیق و مدیریت صریح خطا، کد مقاوم می‌سازد.

۸.۱۳ تمرین‌ها

- تمرین ۱.۱۳.** تابعی بنویسید که در آغازش با تعلیق پیامی چاپ کند و سپس دو خط دیگر. ترتیب خروجی را پیش‌بینی و سپس آزمایش کنید.
- تمرین ۲.۱۳.** تابعی به نام ریشه دوم امن بنویسید که اگر ورودی منفی بود، نادرست برگرداند و نتیجه را از راه پارامتر ارجاعی بدهد.
- تمرین ۳.۱۳.** تابعی بنویسید که سه تعلیق داشته باشد و با اجرای آن، ترتیب معکوس را تأیید کنید.
- تمرین ۴.۱۳.** برنامه‌ای بنویسید که آرایه‌ای بسازد و عمداً به اندیسی بیرون از مرز دسترسی پیدا کند؛ سپس آن را با safe — بسازید و رفتار را ببینید.
- تمرین ۵.۱۳.** تابعی بنویسید که یک سن بگیرد و اگر بیرون از بازه ۰ تا ۱۵۰ بود، با نادرست خطا را اعلام کند.

فصل ۱۴

ماژول‌ها و بسته‌ها

تا اینجا همه برنامه‌ها را در یک فایل نوشتیم. اما برنامه‌های واقعی بزرگ‌اند و نمی‌توان همه را در یک فایل گنجاند. **ماژول‌ها و بسته‌ها** به ما اجازه می‌دهند برنامه را به فایل‌های کوچک‌تر و معنادار بشکنیم و کدها را میان آن‌ها به اشتراک بگذاریم.

۱.۱۴ چرا برنامه را بشکنیم؟

تصور کنید یک برنامه بزرگ دارید: بخشی برای محاسبات ریاضی، بخشی برای کار با متن، بخشی برای رابط کاربری. اگر همه را در یک فایل بریزید، یافتن هر چیز دشوار می‌شود. با تقسیم برنامه به ماژول‌ها:

- هر فایل یک مسئولیت روشن دارد.
- کدها را می‌توان میان پروژه‌ها به اشتراک گذاشت و دوباره به کار برد.
- چند نفر می‌توانند هم‌زمان روی بخش‌های مختلف کار کنند.

۲.۱۴ وارد کردن کتابخانه‌های آماده

ساده‌ترین شکل ماژول، کتابخانه‌های آماده خود سلام‌اند. با واژه وارد کردن (معادل `import`) آن‌ها را به برنامه می‌آوریم. این مثال واقعی ماژول ریاضی را وارد می‌کند:

۱	واردکردن "ریاضی"
۲	تابع اصلی:
۳	چاپ ریاضی. توان (0.2, 0.10)
۴	چاپ ریاضی. جذر (0.144)
۵	پایان

پس از "ریاضی" واردکردن، می‌توانیم با نامِ ماژول و یک نقطه، به توابعش دسترسی پیدا کنیم: ریاضی.توان (...)، ریاضی.جذر (...). این نقطه می‌گوید «توانِ متعلق به ماژولِ ریاضی».

یادداشت

نام ماژول‌های استاندارد هم — مانند همه چیزِ دیگر در سلام — دوزبانه است. واردکردن "ریاضی" و "math" واردکردن یک کار می‌کنند، و می‌توانید با ریاضی.جذر یا math.Sqrt به توابعش برسید. در فصل ۱۵ کتابخانه استاندارد را به تفصیل می‌بینیم.

۳.۱۴ ساختنِ ماژولِ خودمان

می‌توانیم کدهای خود را در یک فایل جدا بگذاریم و در فایلِ دیگر از آن‌ها استفاده کنیم. نخست یک فایلِ کتابخانه می‌سازیم. این مثالِ واقعی، فایلی به نام `_falib.salam` است:

۱	بسته <code>falib</code>
۲	عمومی تابع جمع (a, b) i32, b i32: بازگشت <code>b + a</code> پایان
۳	عمومی ثابت پاسخ <code>42</code>

دو نکتهٔ کلیدی:

- `falib` بسته (معادل `package`) به این فایل یک نام می‌دهد و می‌گوید که این یک بستهٔ کتابخانه‌ای است، نه یک برنامهٔ اجرایی.
- عمومی (معادل `pub`) پیش از جمع تابع و پاسخ ثابت می‌گوید که این‌ها بیرون از فایل هم در دسترس‌اند.

۱.۳.۱۴ عمومی در برابرِ خصوصی

به‌طورِ پیش‌فرض، هر چیزی که در یک ماژول تعریف می‌کنید خصوصی است؛ یعنی فقط درونِ همان فایل دیده می‌شود. تنها چیزهایی که با عمومی نشانه‌گذاری کنید، از بیرون

قابل دسترس‌اند. این یک اصل مهم طراحی است: فقط آنچه را که دیگران واقعاً لازم دارند، عمومی کنید و باقی را خصوصی نگه دارید.

نکته

چرا چیزها را خصوصی نگه داریم؟ چون هرچه کمتر عمومی باشد، آزادی شما برای تغییر درون مازول بیشتر است. اگر همه چیز عمومی باشد، هر تغییر کوچک ممکن است کد دیگران را بشکند. مرز روشن «عمومی/خصوصی» به شما اجازه می‌دهد بی‌دغدغه درون مازول را بهبود دهید.

۲.۳.۱۴ استفاده از مازول خودمان

حالا در فایل اصلی، مازول را وارد و از آن استفاده می‌کنیم. این مثال واقعی نشان می‌دهد چگونه با یک نام مستعار مازول را وارد کنیم:

۱	واردکردن <code>fl "falib.salam"</code>
۲	تابع اصلی:
۳	چاپ <code>(fl.جمع 2)</code> , <code>((3))</code>
۴	چاپ <code>(fl.پاسخ)</code>
۵	پایان

اینجا `fl "falib.salam"` واردکردن فایل را وارد می‌کند و به آن نام کوتاه `fl` می‌دهد. سپس با `(3, جمع 2)` و `fl.پاسخ` به عضوهای عمومی آن دسترسی پیدا می‌کنیم. خروجی: 5 و 42.

یادداشت

نام مستعار (اینجا `fl`) به‌ویژه وقتی مفید است که نام مازول طولانی باشد یا دو مازول نام یکسان داشته باشند. می‌توانید هر نام کوتاه و گویایی برگزینید.

۴.۱۴ دوزبانه‌سازی مازول‌های خودمان

یکی از زیباترین ویژگی‌های سلام این است که می‌توانید برای کد خودتان هم نام‌های فارسی و انگلیسی تعریف کنید. این کار با حاشیه‌نویسی‌های @en و @fa انجام می‌شود. این مثال واقعی یک ساختار کاملاً دوزبانه می‌سازد:

```

1 "Point" @en
2 "نقطه" @fa
3 Point: struct
4 "x" @en
5 "ایکس" @fa
6 0 = i32 x
7 "y" @en
8 "وی" @fa
9 0 = i32 y
10 "Sum" @en
11 "جمع" @fa
12 end this.y + this.x ret i32: Sum() func
13 end
14 تابع اصلی:
15 { Point = Point p mut
16 چاپ ایکس.p
17 چاپ جمع.p
18 چاپ lang()
19 پایان

```

با "Point" @en و "نقطه" @fa به ساختار دو نام دادیم. حالا در یک برنامه فارسی می‌توان میدان‌ها و متدها را با نام فارسی صدا زد (ایکس.p، جمع.p)، هرچند خود ساختار با نام‌های انگلیسی تعریف شده است. تابع ویژه lang() نیز زبان جاری را برمی‌گرداند ("fa" یا "en").

نگاهی به درون: چگونه یک کد، دو زبان دارد؟ (اختیاری، برای خوانندگان پیشرفته)
وقتی شما "نقطه" @fa می‌نویسید، کامپایلر یک نگاشت میان نام فارسی و نام انگلیسی

اصلی می‌سازد. در زمان کامپایل، هر جا که نقطه ببیند، آن را به همان Point اصلی ترجمه می‌کند. پس در کد نهایی تنها یک نوع وجود دارد؛ دو نام فقط دو درگاه ورودی به همان چیزند. به همین دلیل است که دوزبانه‌بودن هیچ هزینه اجرایی ندارد — همه‌چیز در زمان کامپایل حل می‌شود.

نکته

این ویژگی برای نوشتن کتابخانه‌هایی که هم فارسی‌زبانان و هم انگلیسی‌زبانان از آن‌ها استفاده می‌کنند فوق‌العاده است. یک بار کتابخانه را با @en و @fa می‌نویسید و هر کاربر با زبان خودش از آن بهره می‌برد — درست همان کاری که خود کتابخانه استاندارد سلام انجام می‌دهد.

۵.۱۴ ساختار یک پروژه چندفایلی

یک پروژه متوسط معمولاً چنین سامان می‌یابد:

```

۱ my-project/
۲   main.salam          // اینجاست «اصلی تابع اصلی، برنامه»
۳   math_utils.salam    // ریاضی کمکی بسته
۴   text_utils.salam    // متن کمکی بسته

```

main.salam با واردکردن مازول‌های دیگر را می‌آورد و آن‌ها را به هم می‌پیوندد. هنگام ساخت برنامه، سلام به‌طور خودکار همه فایل‌های واردشده را پیدا و کامپایل می‌کند؛ کافی است فایل اصلی را به آن بدهید:

```

۱ salam build main.salam --output=app.exe

```

۶.۱۴ خلاصه فصل

- واردکردن کتابخانه‌های آماده یا مازول‌های خودمان را می‌آورد.
- بسته به یک فایل نام می‌دهد و آن را کتابخانه می‌کند.

- هر چیز به‌طور پیش‌فرض خصوصی است؛ عمومی آن را بیرون‌دیدنی می‌کند.
- با نام مستعار ("..." fl وارد کردن) می‌توان ماژول را کوتاه صدا زد.
- با @en و @fa می‌توان به کد خود نام‌های دوزبانه داد.
- lang() زبان جاری را برمی‌گرداند.

۷.۱۴ تمرین‌ها

- تمرین ۱.۱۴.** یک ماژول به نام هندسه بسازید که توابع عمومی مساحت_دایره و محیط_دایره داشته باشد، و در فایل اصلی از آن استفاده کنید.
- تمرین ۲.۱۴.** در ماژول هندسه یک تابع کمکی خصوصی بگذارید و نشان دهید که از بیرون قابل دسترس نیست.
- تمرین ۳.۱۴.** ماژولی بسازید که هم نام فارسی و هم انگلیسی برای توابعش داشته باشد و آن را یک بار از یک برنامه فارسی و یک بار از یک برنامه انگلیسی صدا بزنید.
- تمرین ۴.۱۴.** ماژول ریاضی استاندارد را وارد کنید و با ریاضی_جذر و ریاضی_توان مساحت یک دایره و قطر آن را حساب کنید.
- تمرین ۵.۱۴.** یک پروژه سه‌فایلی طراحی کنید (اصلی + دو ماژول) و توضیح دهید هر فایل چه مسئولیتی دارد.

فصل ۱۵

کتابخانه استاندارد

هیچ برنامه نویسی همه چیز را از صفر نمی سازد. سلام با یک کتابخانه استاندارد غنی همراه است: مجموعه ای از ماژول های آماده برای ریاضی، متن، ساختارهای داده، زمان، تصادف و بسیار چیزهای دیگر. در این فصل پرکاربردترین ها را می بینیم.

۱.۱۵ ریاضی

ماژول ریاضی (`math`) توابع ریاضی رایج را در اختیار می گذارد. این مثال واقعی چند تای آنها را نشان می دهد:

۱	وارد کردن " <code>math</code> "
۲	تابع اصلی:
۳	چاپ ("جذر", "۱۴۴"; ریاضی.جذر(0.144))
۴	چاپ ("بیشینه:", ریاضی.بیشینه(0.3, 0.9))
۵	چاپ ("عدد پی:", ریاضی.پی)
۶	پایان

مهم ترین توابع و ثابت های ماژول ریاضی:

کار	فارسی	انگلیسی
جذر	ریاضی.جذر (x)	Sqrt
توان	(y ریاضی.توان (x),	Pow
قدر مطلق	ریاضی.قدر مطلق (x)	Abs
گرد کردن به پایین	ریاضی.کف (x)	Floor
گرد کردن به بالا	ریاضی.سقف (x)	Ceil
سینوس	ریاضی.سینوس (x)	Sin
کسینوس	ریاضی.کسینوس (x)	Cos
لگاریتم	ریاضی.لگاریتم (x)	Log
بزرگ تر	(b ریاضی.بیشینه a,	Max
عدد π	ریاضی.پی	PI

هشدار

بیشتر توابع ریاضی روی اعداد اعشاری کار می کنند. به همین دلیل ریاضی.جذر(144.0) نوشتیم، نه ریاضی.جذر(144). اگر عدد صحیح دارید، نخست آن را با اعشاری ۶۴ بعنوان تبدیل کنید.

۲۰۱۵ رشته

ماژول رشته (str) را در فصل ۸ معرفی کردیم. علاوه بر متدهای پایه، توابع پیشرفته تری نیز دارد. این مثال واقعی چند تای آن ها را نشان می دهد:

۱	تابع اصلی:
۲	ثابت متن رشته = "salam,donya"
۳	چاپ "طول-متن", متن.طول()
۴	چاپ "بخش", متن.زیررشته(0, 5)
۵	چاپ "یافتن", متن.بیاب("donya")
۶	چاپ "پیوست", متن.پیوست("؛")
۷	متغیر بخش‌ها وکتور <رشته> = متن.بشکاف(",")
۸	چاپ "تعداد-بخش", بخش‌ها.طول()
۹	چاپ "عدد", "۴۲".پیراست()
۱۰	چاپ "به‌صحیح", "42".به_صحیح() + 1
۱۱	پایان

توابع تازه‌ای که اینجا می‌بینیم:

- بیاب("...") (Find): جای نخستین یافتن یک زیررشته را برمی‌گرداند.
- بشکاف(",") (Split): رشته را بر اساس یک جداکننده به یک برد/راز تکه‌ها می‌شکند.
- پیراست() (Trim): فاصله‌های آغاز و پایان را می‌زداید.
- به_صحیح() (ToInt): یک رشته عددی را به عدد صحیح تبدیل می‌کند.

نکته

بشکاف() یکی از پرکاربردترین توابع در پردازش داده است. هر وقت داده‌ای دارید که با کاما، فاصله یا نشانه دیگری جدا شده (مثل یک خطِ فایل، CSV) با بشکاف آن را به اجزایش تقسیم کنید.

۳۰۱۵ نگاشت: واژه‌نامه کلید-مقدار

نگاشت (HashMap) ساختاری است که هر کلید را به یک مقدار وصل می‌کند — درست مانند یک واژه‌نامه که هر واژه را به معنایش وصل می‌کند. این مثال واقعی سن‌های چند نفر را نگه می‌دارد:

۱	تابع اصلی:
۲	متغیر سن‌ها نگاشت <رشته، صحیح ۳۲> = نگاشت {}
۳	سن‌ها. درج ("سارا"، 30)
۴	سن‌ها. درج ("علی"، 25)
۵	سن‌ها. درج ("رضا"، 40)
۶	چاپ ("سن سارا"، سن‌ها. بگیر ("سارا"))
۷	چاپ ("علی موجود است؟"، سن‌ها. دارد ("علی"))
۸	چاپ ("حسن موجود است؟"، سن‌ها. دارد ("حسن"))
۹	چاپ ("تعداد"، سن‌ها. اندازه())
۱۰	پایان

نوع صحیح ۳۲ <نگاشت >رشته، یعنی «نگاشتی که کلیدهایش رشته و مقدارهایش عدد صحیح‌اند». متدهای اصلی:

فارسی	انگلیسی	کار
مقدار. درج (کلید،	put	افزودن یا به‌روزرسانی
.بگیر (کلید)	get	گرفتن مقدار یک کلید
.دارد (کلید)	has	آیا کلید هست؟
.حذف (کلید)	remove	حذف یک کلید
.اندازه()	size	تعداد عناصر
.پیمایش()	iter	ساختن پیمایشگر

۱.۳.۱۵ پیمایش یک نگاشت

برای گذشتن از همهٔ عناصر یک نگاشت، یک پیمایشگر می‌سازیم. این مثال واقعی، مقدارهای یک نگاشت را جمع می‌زند:

۱	تابع اصلی:
۲	متغیر امتیاز نگاشت <رشته، صحیح ۳۲> = نگاشت {}
۳	امتیاز.درج("الف"، 3)
۴	امتیاز.درج("ب"، 7)
۵	متغیر مجموع صحیح ۳۲ = 0
۶	متغیر پیمایه پیمایشگر نگاشت <رشته، صحیح ۳۲> = امتیاز.پیمایش()
۷	تاوقتی پیمایه.دارد_بعدی():
۸	مجموع = مجموع + پیمایه.مقدار()
۹	پیمایه.بعدی()
۱۰	پایان
۱۱	چاپ "جمع"، مجموع
۱۲	پایان

پیمایشگر سه متد کلیدی دارد: دارد_بعدی() (آیا عنصر دیگری مانده؟)، مقدار() (مقدار عنصر جاری) و بعدی() (رفتن به عنصر بعد). این الگو — «تا وقتی عنصری مانده، پردازش کن و جلو برو» — شیوه استاندارد گذشتن از مجموعه‌هاست.

نکته

نگاشت برای جست‌وجوی سریع عالی است. اگر می‌خواهید بدانید «سن سارا چند است؟»، نگاشت پاسخ را تقریباً بی‌درنگ می‌دهد — بی‌آنکه مجبور باشید مثل یک بردار، همه‌چیز را یکی‌یکی بگردید. هر جا نیاز به نگاشتن «نام به داده» داشتید (شماره دانشجو به نمره، نام کالا به قیمت)، نگاشت ابزار درست است.

۴.۱۵ دیگر ماژول‌های کاربردی

کتابخانه استاندارد سلام بسیار گسترده است. در ادامه چند ماژول پرکاربرد دیگر را به اختصار معرفی می‌کنیم؛ شیوه کارشان همان است: واردکردن، سپس دسترسی با نام ماژول و نقطه.

کاربرد	ماژول
ورودی و خروجی؛ خواندن ورودی کاربر، چاپ، کار با فایل	io
زمان و تاریخ؛ Now، سال، ماه، روز، Sleep	time
تولید عددهای تصادفی؛ Seed، Int، Float	rand
قالب‌بندی متن؛ ساختن رشته از عدد و مقدارهای دیگر	fmt
سیستم‌عامل؛ آرگومان‌ها، متغیرهای محیطی، فایل‌ها	os
خواندن و نوشتن داده‌های JSON	json
نوشتن آزمون؛ Assert و گزارش نتیجه	testing
هم‌روندی؛ نخ‌ها و قفل‌ها (فصل ۱۷)	sync
درخواست‌های وب؛ Post، Get	http

ماژول‌های io، os، time و http را در فصل‌های آینده (۱۶ و ۱۷) بیشتر می‌بینیم؛ testing را در فصل ۲۰.

۱۰۴۰۱۵ نمونه: عدد تصادفی

۱	وارد کردن "rand"
۲	تابع اصلی:
۳	rand.Seed(7)
۴	چاپ "یک عدد تصادفی:"، rand.Int()
۵	پایان

نخست با Seed «بذر» مولد تصادف را می‌کاریم (تا دنباله اعداد قابل بازتولید باشد)، سپس با Int یک عدد تصادفی می‌گیریم.

نگاهی به درون: تصادف واقعی یا ساختگی؟ (اختیاری، برای خوانندگان پیشرفته)

رایانه نمی‌تواند عدد «واقعاً» تصادفی بسازد؛ آنچه می‌سازد دنباله‌ای است که تصادفی به نظر می‌رسد اما در واقع از یک فرمول معین و یک «بذر» آغازین زاده می‌شود. به همین دلیل اگر بذر یکسان بدهید، همان دنباله را می‌گیرید — که برای آزمون و اشکال‌زدایی بسیار مفید است. اگر تصادف غیرقابل پیش‌بینی می‌خواهید، بذر را از زمان جاری بگیرید.

۵.۱۵ خلاصه فصل

- ماژول ریاضی توابع ریاضی و ثابت‌هایی مانند π دارد.
- ماژول رشته توابع پیشرفته متن دارد: بشکاف، بیاب، پیراست، به_صحیح.
- نگاشت هر کلید را به یک مقدار وصل می‌کند و جست‌وجو را سریع می‌سازد.
- پیمایشگر با دارد_بعدی، مقدار و بعدی از مجموعه‌ها می‌گذرد.
- ماژول‌های فراوان دیگری برای زمان، تصادف، فایل، شبکه و آزمون هست.

۶.۱۵ تمرین‌ها

- تمرین ۱.۱۵.** با ماژول ریاضی، طول وترِ مثلثی با اضلاع ۳ و ۴ را حساب کنید (قضیه فیثاغورس).
- تمرین ۲.۱۵.** یک نگاشت بسازید که نام چند میوه را به قیمتشان وصل کند، سپس قیمت یک میوه را بپرسید.
- تمرین ۳.۱۵.** با پیمایش یک نگاشت قیمت‌ها، گران‌ترین کالا را پیدا کنید.
- تمرین ۴.۱۵.** رشته "۱۰،۲۰،۳۰،۴۰" را با بشکاف به اعداد تبدیل کنید و مجموعشان را بیابید.
- تمرین ۵.۱۵.** با ماژول rand، یک «تاس» شش‌وجهی بسازید که عددی میان ۱ تا ۶ بدهد (راهنمایی: از باقی‌مانده تقسیم بر ۶ کمک بگیرید).

فصل ۱۶

کار با فایل و سیستم

تا اینجا داده‌ها را درون برنامه می‌ساختیم و خروجی را روی صفحه چاپ می‌کردیم. اما برنامه‌های واقعی با دنیای بیرون هم در ارتباط‌اند: فایل می‌خوانند و می‌نویسند، آرگومان‌های خط فرمان می‌گیرند و با سیستم عامل کار می‌کنند. در این فصل این پل‌ها را می‌سازیم.

۱.۱۶ نوشتن و خواندن فایل

ماژول‌های `io` و `os` ابزارهای کار با فایل را فراهم می‌کنند. این مثال واقعی یک فایل می‌نویسد، آن را باز می‌خواند و سپس پاک می‌کند:

```
۱ package main
۲ import (
۳     "os"
۴     "io"
۵     "str"
۶ )
۷ func main():
۸     path str = "io_example_out.txt"
۹     io.WriteFile(path, "the quick brown fox\njumps over\nthe lazy dog\n")
۱۰    text str = io.ReadFile(path)
۱۱    lines Vector<str> = str.Split(str.Trim(text), "\n")
۱۲    io.Println("read lines:")
۱۳    __ str.Join(lines, " | ")
```

```

۱۴ os.Remove(path)
۱۵ lines.free()
۱۶ end

```

یادداشت

این نمونه به انگلیسی نوشته شده تا با کد واقعی کتابخانه هماهنگ باشد. به یاد داشته باشید که `__` (دو زیرخط) شکل کوتاه چاپ `println` است؛ پس `str.Join(...)` یعنی «نتیجه را چاپ کن». همین برنامه را می‌توان با واژگان فارسی نیز نوشت.

سه عمل کلیدی فایل را اینجا می‌بینیم:

- محتوا، `io.WriteFile`: یک رشته را در فایل می‌نویسد (و اگر فایل نبود، می‌سازد).
- `io.ReadFile` (مسیر): همه محتوای فایل را به صورت یک رشته برمی‌گرداند.
- `os.Remove` (مسیر): فایل را پاک می‌کند.

پس از خواندن، با `str.Split(str.Trim(text), "\n")` متن را به خط‌هایش شکستیم — همان بشکاف/`Split` که در فصل ۱۵ دیدیم. این الگو — بخوان، با `Split` بشکن، پردازش کن — ستون فقرات کار با فایل‌های متنی است.

هشدار

خواندن و نوشتن فایل می‌تواند شکست بخورد: فایل ممکن است وجود نداشته باشد، یا اجازه دسترسی نباشد. در برنامه‌های جدی، پیش از خواندن با `os.Exists` (مسیر) بررسی کنید که فایل هست، و همان‌طور که در فصل ۱۳ گفتیم، حالت خطا را آگاهانه مدیریت کنید.

۲۰۱۶ آرگومان‌های خط فرمان

وقتی برنامه‌ای را اجرا می‌کنید، می‌توانید همان لحظه به آن داده بدهید. مثلاً:

۱ `./app.exe سلام`

اینجا «سلام» و «دنیا» دو آرگومان‌اند. با `os.Args()` می‌توانید آن‌ها را در برنامه بگیرید — که یک بردار از رشته‌ها برمی‌گرداند:

```

۱ package main
۲ import ("os" "io" "str")
۳ func main:
۴     a Vector<str> = os.Args()
۵     io.Print("called with ")
۶     io.Print(str.FromInt((a.len() - 1) as i64))
۷     io.Println(" argument(s)")
۸     mut i = 1
۹     while i < a.len():
۱۰         io.Println(a.get(i)[0])
۱۱         i = i + 1
۱۲     end
۱۳     a.free()
۱۴ end

```

یادداشت

نخستین عنصر `os.Args()` (یعنی اندیس ۰) همیشه نام خود برنامه است؛ به همین دلیل حلقه را از اندیس ۱ آغاز کردیم و برای شمارش آرگومان‌های واقعی، یکی از طول کم کردیم $(a.len() - 1)$. این قراردادی است که در بیشتر زبان‌ها مشترک است.

۳.۱۶ متغیرهای محیطی

سیستم‌عامل اطلاعاتی را به‌صورت متغیرهای محیطی نگه می‌دارد — مانند نام کاربر یا مسیر خانه. برنامه‌ها می‌توانند این‌ها را بخوانند. ماژول `os` تابع `Env` را برای این کار دارد:

```

۱ package main
۲ import ("os" "io")

```

```

۳ func main:
۴     user str = os.Env("USERNAME")
۵     io.Println(user)
۶ end

```

نکته

متغیرهای محیطی جای خوبی برای نگه داشتن تنظیمات حساس (مانند کلیدهای دسترسی) هستند، چون درون کد شما نوشته نمی شوند و در هر سیستم می توانند متفاوت باشند. این یک عادت مهم امنیتی است: رمزها و کلیدها را هرگز مستقیم در کد ننویسید.

۴.۱۶ تشخیص سیستم عامل در زمان کامپایل

گاهی برنامه باید روی ویندوز، مک و لینوکس کمی متفاوت رفتار کند — مثلاً جداکننده مسیرها در ویندوز \ و در لینوکس / است. سلام یک پیش پردازنده دارد که اجازه می دهد بر اساس سیستم عامل، بخش هایی از کد را در زمان کامپایل انتخاب کنید. این مثال واقعی را ببینید:

```

۱ @if SALAM_OS_WINDOWS
۲     const PATH_SEP = "\\"
۳     const SO_EXT   = ".dll"
۴ @else SALAM_OS_MAC
۵     const PATH_SEP = "/"
۶     const SO_EXT   = ".dylib"
۷ @else SALAM_OS_UNIX
۸     const PATH_SEP = "/"
۹     const SO_EXT   = ".so"
۱۰ @end
۱۱ func main:
۱۲     @if SALAM_OS_WINDOWS
۱۳         _ "Running on Windows"
۱۴     @else
۱۵         _ "Running on a Unix-like system"
۱۶     @end

```

```

۱۷  __ "Path separator:", PATH_SEP
۱۸  end

```

دستورهای `@if`، `@else` و `@end` پیش از کامپایل اصلی اجرا می‌شوند و فقط شاخه درست را نگه می‌دارند. پرچم‌هایی مانند `SALAM_OS_WINDOWS`، `SALAM_OS_MAC` و `SALAM_OS_UNIX` را خود کامپایلر بر اساس سیستم مقصد تعیین می‌کند.

نگاهی به درون: تفاوت پیش‌پردازنده و شرط معمولی (اختیاری، برای خوانندگان پیشرفته)

چه فرقی است میان `@if` و اگر؟ اگر در زمان اجرا تصمیم می‌گیرد و هر دو شاخه در برنامه نهایی حضور دارند. اما `@if` در زمان کامپایل تصمیم می‌گیرد و شاخه ناخواسته اصلاً وارد برنامه نهایی نمی‌شود. به همین دلیل `@if` برای کدی که فقط روی یک سیستم معنا دارد (مثل فراخوانی یک کتابخانه مخصوص ویندوز) ضروری است: نمی‌خواهیم آن کد روی لینوکس حتی کامپایل شود.

می‌توانید پرچم‌های دلخواه خود را هم هنگام ساخت تعریف کنید:

```

۱  salam build app.salam -DDEBUG --output=app.exe

```

و در کد با `@end` ... `@ifdef DEBUG` به آن پاسخ دهید — روشی عالی برای گذاشتن پیام‌های اشکال‌زدایی که فقط در نسخه توسعه فعال‌اند.

۵.۱۶ خلاصه فصل

- با `io.ReadFile` و `io.WriteFile` فایل می‌نویسیم و می‌خوانیم.
- `os.Remove` و `os.Exists` برای مدیریت فایل‌اند.
- `os.Args()` آرگومان‌های خط فرمان را می‌دهد (اندیس ۰ نام برنامه است).
- `os.Env(...)` متغیرهای محیطی را می‌خواند.
- پیش‌پردازنده (`@end/@else/@if`) کد را بر اساس سیستم عامل در زمان کامپایل انتخاب می‌کند.

۶.۱۶ تمرین‌ها

تمرین ۱۰.۱۶. برنامه‌ای بنویسید که یک پیام را در فایلی بنویسد و سپس آن را بخواند و چاپ کند.

تمرین ۲۰.۱۶. برنامه‌ای بنویسید که نام شما را به‌عنوان آرگومان بگیرد و «سلام، <نام>» چاپ کند.

تمرین ۳۰.۱۶. فایلی متنی بسازید که چند خط دارد، آن را بخوانید و تعداد خط‌ها را بشمارید.

تمرین ۴۰.۱۶. با کمک `@if`، برنامه‌ای بنویسید که روی هر سیستم‌عامل پیام متفاوتی چاپ کند.

تمرین ۵۰.۱۶. برنامه‌ای بنویسید که چند عدد را به‌عنوان آرگومان بگیرد و مجموعشان را حساب کند (راهنمایی: از به‌صحت `ToInt` استفاده کنید).

فصل ۱۷

همروندی

تا اینجا برنامه‌های ما یک کار را در هر لحظه انجام می‌دادند. اما رایانه‌های امروز چند هسته دارند و می‌توانند چند کار را هم‌زمان پیش ببرند. همروندی (concurrency) هنر بهره‌گیری از این توانایی است. در این فصل با نخ‌ها و هماهنگی میانشان آشنا می‌شویم.

هشدار

این فصل از پیشرفته‌ترین مباحث کتاب است. اگر تازه‌کارید، می‌توانید فعلاً از آن بگذرید و پس از تسلط بر فصل‌های پیشین بازگردید. همروندی قدرتمند است، اما نیازمند دقت.

۱.۱۷ نخ چیست؟

یک نخ (thread) مانند یک کارگر مستقل است که می‌تواند هم‌زمان با کارگران دیگر کار کند. اگر چهار نخ بسازید و رایانه شما چهار هسته داشته باشد، هر چهار کار می‌توانند واقعاً هم‌زمان پیش بروند — و برنامه تا چهار برابر سریع‌تر شود. ماژول sync ابزارهای کار با نخ را فراهم می‌کند. دو تابع کلیدی:

- spawn(تابع): یک نخ تازه می‌سازد که آن تابع را اجرا می‌کند، و یک شناسه برمی‌گرداند.
- join(شناسه): منتظر می‌ماند تا آن نخ کارش را تمام کند.

۲.۱۷ مسئلهٔ مشترک: داده‌ای که چند نخ به آن دست می‌زنند

وقتی چند نخ به یک داده دست می‌زنند، دردسر آغاز می‌شود. تصور کنید دو نخ هم‌زمان می‌خواهند به یک شمارنده یکی اضافه کنند؛ ممکن است هر دو مقدارِ کهنه را بخوانند و یکی از افزایش‌ها گم شود. به این مشکل **رقابت** (race condition) می‌گویند. راهِ حل، **قفل** (Mutex) است: ابزاری که تضمین می‌کند در هر لحظه فقط یک نخ به داده دست می‌زند. این مثال واقعی — که جمعِ اعداد را میانِ چهار نخ تقسیم می‌کند — همهٔ این ابزارها را به کار می‌گیرد:

```

۱ package main
۲ import ("sync" "str" "io")
۳ extern "C":
۴     func salam_alloc(size u64) void*
۵     func salam_free(ptr void*)
۶ end
۷ const N = 100000
۸ mut sum i64* = null
۹ mut lock sync.Mutex = sync.NewMutex()
۱۰ mut wg sync.WaitGroup = sync.NewWaitGroup()
۱۱ func add_range(lo i32, hi i32):
۱۲     mut local i64 = 0
۱۳     mut i = lo
۱۴     while i < hi:
۱۵         local = local + i as i64
۱۶         i = i + 1
۱۷     end
۱۸     mut acc i64* = sum
۱۹     sync.Lock(lock)
۲۰     acc[0] = acc[0] + local
۲۱     sync.Unlock(lock)
۲۲     sync.Done(wg)
۲۳ end
۲۴ func q1(): add_range(0, N / 4) end
۲۵ func q2(): add_range(N / 4, N / 2) end
۲۶ func q3(): add_range(N / 2, N * 3 / 4) end
۲۷ func q4(): add_range(N * 3 / 4, N) end

```

```

۲۸ func main:
۲۹     sum = salam_alloc(8 as u64) as i64*
۳۰     sum[0] = 0
۳۱     sync.Add(wg, 4 as i64)
۳۲     t1 i64 = spawn(q1)
۳۳     t2 i64 = spawn(q2)
۳۴     t3 i64 = spawn(q3)
۳۵     t4 i64 = spawn(q4)
۳۶     sync.Wait(wg)
۳۷     join(t1)
۳۸     join(t2)
۳۹     join(t3)
۴۰     join(t4)
۴۱     io.Print("sum of 0..")
۴۲     io.Print(str.FromInt(N as i64))
۴۳     io.Print(" = ")
۴۴     io.Println(str.FromInt(sum[0]))
۴۵     salam_free(sum as void*)
۴۶ end

```

این برنامه پیچیده به‌نظر می‌رسد، اما ایده‌اش روشن است. بیا یاد آرام بشکافیمش.

۱۰۲۰۱۷ تقسیم کار

به‌جای آنکه یک نخ همهٔ اعداد ۰ تا ۱۰۰,۰۰۰ را جمع بزنند، کار را به چهار بخش تقسیم کرده‌ایم. هر یک از توابع q1 تا q4 یک‌چهارم بازه را جمع می‌زنند. این «تقسیم و غلبه» جانِ کلام هم‌روندی است.

۲۰۲۰۱۷ محافظت با قفل

هر نخ نخست جمع محلی خود را در local حساب می‌کند (که هیچ نخ دیگری به آن دست نمی‌زند، پس امن است). تنها در پایان، که می‌خواهد نتیجه را به مجموع مشترک بیفزاید، قفل می‌گیرد:

```

۱ sync.Lock(lock)
۲ acc[0] = acc[0] + local
۳ sync.Unlock(lock)

```

میان Lock و Unlock فقط یک نخ می‌تواند باشد. پس افزودن نتیجه هرگز دچار رقابت نمی‌شود.

۳.۲.۱۷ انتظار با گروه انتظار

WaitGroup ابزاری است که می‌گوید «منتظر پایان همه نخ‌ها بمان». با `sync.Add(wg)` (4) گفتیم چهار کار در راه است؛ هر نخ در پایان `sync.Done(wg)` را صدا می‌زند؛ و `sync.Wait(wg)` در تابع اصلی صبر می‌کند تا هر چهار «انجام‌شده» اعلام شوند.

نکته

الگوی استاندارد همروندی ایمن: (۱) کار را به بخش‌های مستقل تقسیم کن، (۲) بگذار هر نخ روی داده محلی خودش کار کند، (۳) فقط در پایان و با قفل، نتیجه‌ها را با هم بیامیز، (۴) با `Wait` و `join` منتظر پایان همه بمان. اگر این الگو را رعایت کنید، از بیشتر دردهای همروندی در امان می‌مانید.

نگاهی به درون: چرا جمع محلی؟ (اختیاری، برای خوانندگان پیشرفته)

اگر هر نخ در هر گام قفل می‌گرفت تا به مجموع مشترک اضافه کند، نخ‌ها بیشتر وقتشان را پشت قفل منتظر می‌ماندند و همروندی هیچ سودی نداشت — حتی شاید کندتر می‌شد! ترفند «جمع محلی، سپس یک‌بار آمیختن» تعداد دفعات قفل‌گیری را از صدها هزار به فقط چهار کاهش می‌دهد. این یک اصل مهم است: قفل را تا جای ممکن کوتاه و کم نگه دارید.

۳.۱۷ خطرهای همروندی

همروندی قدرتمند است اما تله‌هایی دارد که باید بشناسید:

- **رقابت:** دو نخ بی‌محافظت به یک داده دست می‌زنند. درمان: قفل.
- **بن‌بست (deadlock):** دو نخ هر کدام منتظر قفلی هستند که دیگری گرفته، و هیچ‌کدام پیش نمی‌رود. درمان: همیشه قفل‌ها را به ترتیب یکسان بگیرید.

- **فراموشی آزادسازی:** اگر Unlock را فراموش کنید، نخ‌های دیگر تا ابد منتظر می‌مانند. درمان: Lock و Unlock را همیشه جفت بنویسید.

۴.۱۷ خلاصه فصل

- نخ کارگری مستقل است که هم‌زمان با دیگران کار می‌کند.
- با spawn نخ می‌سازیم و با join منتظرش می‌مانیم.
- قفل (Mutex) تضمین می‌کند فقط یک نخ به داده دست می‌زند.
- WaitGroup برای انتظارِ پایانِ همه نخ‌هاست.
- بهترین الگو: کار مستقل، داده محلی، آمیختن کوتاه با قفل.
- مراقب رقابت و بن‌بست باشید.

۵.۱۷ تمرین‌ها

- تمرین ۱.۱۷. برنامه‌ای بنویسید که دو نخ بسازد و هر کدام پیام متفاوتی چاپ کند، و در تابع اصلی منتظر هر دو بماند.
- تمرین ۲.۱۷. جمع اعداد ۱ تا ۱۰۰۰۰ را یک بار با یک نخ و یک بار با چهار نخ حساب کنید و نتیجه‌ها را مقایسه کنید.
- تمرین ۳.۱۷. توضیح دهید چرا در مثال این فصل، استفاده از local به جای افزودن مستقیم به sum مهم است.
- تمرین ۴.۱۷. یک سناریوی بن‌بست را روی کاغذ شرح دهید و بگویید چگونه می‌توان از آن پرهیز کرد.
- تمرین ۵.۱۷. بررسی کنید اگر در مثال این فصل sync.Lock و sync.Unlock را حذف کنیم، چه مشکلی ممکن است پیش بیاید.

فصل ۱۸

ارتباط با کد خارجی

دنیای نرم‌افزار پر از کتابخانه‌های آماده است که در زبان C نوشته شده‌اند: از توابع ریاضی سیستم تا پایگاه‌های داده و کتابخانه‌های گرافیکی. سلام به شما اجازه می‌دهد مستقیماً این کتابخانه‌ها را صدا بزنید. به این پل، **رابط تابع خارجی (FFI)** می‌گویند.

۱.۱۸ فراخوانی یک تابع خارجی

برای صدا زدن یک تابع C، نخست با یک بلوک خارجی (معادل `extern`) به سلام معرفی‌اش می‌کنیم — یعنی می‌گوییم نامش چیست و چه ورودی و خروجی دارد. این مثال واقعی چند تابع ریاضی کتابخانه استاندارد C را صدا می‌زند:

```
۱ extern "C":
۲     func printf(format str, ...) i32
۳     func sqrt(x f64) f64
۴     func pow(base f64, exp f64) f64
۵     func hypot(x f64, y f64) f64
۶ end
۷ const PI f64 = 3.14159265358979
۸ func main:
۹     printf("sqrt(2)    = %.6f\n", sqrt(2.0))
۱0    printf("2^10      = %.0f\n", pow(2.0, 10.0))
۱۱    printf("hypot(3,4) = %.1f\n", hypot(3.0, 4.0))
۱۲ end
```

بلوک "C" extern می‌گوید: «این توابع در جایی دیگر — در کد C — تعریف شده‌اند؛ من فقط امضایشان را اعلام می‌کنم.» سپس می‌توانید آن‌ها را درست مانند توابع سلام صدا بزنید: sqrt(2.0)، pow(2.0, 10.0).

یادداشت

به ... در امضای printf دقت کنید. این نشانه می‌گوید که تابع تعداد متغیری از آرگومان می‌پذیرد. printf تابع معروف چاپ در C است که یک «رشته قالب» می‌گیرد و سپس هر تعداد مقدار که آن قالب بخواهد. نشانه‌هایی مانند %.6f (عدد اعشاری با ۶ رقم) و %d (عدد صحیح) جای مقدارها را در رشته مشخص می‌کنند.

۲۰۱۸ همان مفهوم به فارسی

واژه خارجی نیز — مانند همه واژگان سلام — معادل فارسی extern است. این مثال واقعی نشان می‌دهد که بلوک خارجی را می‌توان در یک برنامه فارسی هم نوشت:

```

۱ خارجی: "C"
۲ تابع sqlite3_libversion()
۳ پایان
۴ تابع اصلی:
۵ sqlite3_libversion() = str v
۶ اگر v != ""
۷ چاپ("کتابخانه sqlite متصل شد")
۸ وگرنه:
۹ چاپ("اتصال برقرار نشد")
۱۰ پایان
۱۱ پایان

```

۳۰۱۸ پیوند به یک کتابخانه

برخی توابع (مانند printf و sqrt) در کتابخانه‌های پایه‌ای‌اند که به‌طور خودکار در دسترس‌اند. اما برای کتابخانه‌های دیگر (مانند sqlite) باید به سلام بگویید که به کدام

کتابخانه پیوند بزند. این کار با پویا پیوند (dynamic) (link) انجام می‌شود، و معمولاً همراه پیش‌پردازنده برای انتخاب نام درست در هر سیستم عامل:

```

1 SALAM_OS_WINDOWS @if
2     پیوند پویا "winsqlite3"
3     @else
4     پیوند پویا "sqlite3"
5     @end
6     خارجی: "C"
7     تابع sqlite3_libversion()
8     پایان
9     تابع اصلی:
10    چاپ sqlite3_libversion()
11    پایان
    
```

اینجا روی ویندوز به winsqlite3 و روی سیستم‌های دیگر به sqlite3 پیوند می‌زنیم — نمونه‌ای زیبا از همکاری FFI و پیش‌پردازنده‌ای که در فصل پیش دیدیم.

۴.۱۸ کاربرد عملی: خواندن متغیر محیطی از C

این مثال واقعی، تابع getenv از C را برای خواندن نام کاربر صدا می‌زند:

```

1 extern "C":
2     func printf(format str, ...) i32
3     func getenv(name str) str
4     func strlen(s str) u64
5 end
6 func main:
7     user str = getenv("USERNAME")
8     printf("Your username is %d bytes.\n", strlen(user))
9 end
    
```

اینجا سه تابع C را با هم به کار گرفتیم: getenv (خواندن متغیر محیطی)، strlen (طول یک رشته C) و printf (چاپ). این نشان می‌دهد که چگونه سلام می‌تواند مستقیم با توانمندی‌های سیستم عامل از راه C کار کند.

هشدار

کار با FFI قدرتمند اما حساس است. وقتی به کد C می‌روید، از حفاظ‌های ایمنی سلام بیرون می‌آید: باید خودتان مطمئن شوید که نوع‌ها و امضاها دقیقاً درست‌اند. یک امضای نادرست (مثلاً i32 به جای i64) می‌تواند برنامه را به‌طور ناگهانی از کار بیندازد. همیشه مستندات کتابخانه C را با دقت بخوانید.

نگاهی به درون: FFI چگونه کار می‌کند (اختیاری، برای خوانندگان پیشرفته)

وقتی سلام برنامه شما را به C ترجمه می‌کند، توابع معمولی سلام را با یک نام ویژه «مزین» می‌کند تا با هم تداخل نکنند. اما توابع "C" extern را بدون این تزئین و با نام اصلی C صدا می‌زنند — دقیقاً همان‌طور که کامپایلر C انتظار دارد. به همین دلیل است که سلام می‌تواند بی‌واسطه با هر کتابخانه C کار کند: هر دو در نهایت به همان «ABI» زبان C می‌رسند، که زبان مشترک تقریباً همه نرم‌افزارهای سیستمی است.

۵.۱۸ خلاصه فصل

- FFI پلی است برای صدا زدن توابع C از درون سلام.
- با بلوک "C" خارجی (یا "extern C") توابع خارجی را اعلام می‌کنیم.
- ... در امضا یعنی تابع آرگومان‌های متغیر می‌گیرد (مثل printf).
- با "... پویا پیوند به یک کتابخانه پیوند می‌زنیم.
- پیش‌پردازنده به ما کمک می‌کند نام درست کتابخانه را در هر سیستم عامل برگزینیم.
- FFI شما را از حفاظ‌های ایمنی بیرون می‌برد؛ با دقت کار کنید.

۶.۱۸ تمرین‌ها

تمرین ۱.۱۸. با `FFI` تابع `sqrt` از `C` را صدا بزنید و جذر چند عدد را چاپ کنید.

تمرین ۲.۱۸. تابع `getenv` را صدا بزنید و مقدار متغیر محیطی `PATH` را بگیرید (می‌توانید فقط طولش را با `strlen` چاپ کنید).

تمرین ۳.۱۸. با `printf` و رشته‌های قالب گوناگون (`%d`، `%.2f`، `%s`)، چند مقدار را با قالب‌بندی زیبا چاپ کنید.

تمرین ۴.۱۸. توضیح دهید چرا امضای نادرست یک تابع خارجی خطرناک است.

تمرین ۵.۱۸. یک بلوک خارجی بنویسید که چند تابع ریاضی (`sin`، `cos`، `pow`) را اعلام کند و آن‌ها را به کار بگیرد.

فصل ۱۹

چارچوب وب و سامانه چیدمان

سلام تنها برای برنامه‌های متنی نیست؛ یک سامانه چیدمان (layout) دارد که با آن می‌توانید صفحه‌های وب بسازید — آن هم به زبان فارسی و راست‌به‌چپ! در این فصل یاد می‌گیریم چگونه با سلام یک صفحه وب کامل و زیبا طراحی کنیم.

۱.۱۹ نخستین صفحه وب

برنامه‌های چیدمان به‌جای اصلی تابع، با واژه صفحه (معادل layout) آغاز می‌شوند. این مثال واقعی یک صفحه ساده می‌سازد:

صفحه:	۱
عنوان = "سلام دنیا"	۲
جهت = "راست به چپ"	۳
جعبه:	۴
محتوا = "درود"	۵
رنگ = "آبی"	۶
پس‌زمینه = "سفید"	۷
پایان	۸
پایان	۹

برای ساختن صفحه وب از این کد، از فرمان layout build استفاده می‌کنیم:

```
۱ | salam layout build page.salam
```

این فرمان سه فایل می‌سازد: page.html، page.css و page.js. کافی است page.html

را در مرورگر باز کنید تا صفحه خود را ببینید! اگر یک فایل یکپارچه می‌خواهید، گزینه `--inline` را بیفزایید:

```
۱ | salam layout build page.salam --inline
```

۲۰۱۹ عنصرها و ویژگی‌ها

هر صفحه از **عنصرها** ساخته می‌شود (مانند جعبه، سرصفحه، دکمه)، و هر **عنصر ویژگی‌هایی** دارد (مانند رنگ، محتوا، پس‌زمینه). الگوی همیشگی چنین است:

نام‌عنصر: ویژگی = مقدار ... پایان

این مثال واقعی — که در پوشه مثال‌های سلام هست — یک صفحه کامل‌تر با سرصفحه می‌سازد:

صفحه:	۱
عنوان = "برنامه‌ی من"	۲
زبان = "فارسی"	۳
جهت = "راست به چپ"	۴
سرسیطه:	۵
پس‌زمینه = "آبی"	۶
رنگ = "سفید"	۷
فاصله داخلی = "20px"	۸
سرتیتر: اندازه 1 محتوا = "خوش آمدید" اندازه قلم = "32px" پایان	۹
پایان	۱۰
جعبه:	۱۱
محتوا = "این یک صفحه‌ی نمونه به زبان فارسی است."	۱۲
رنگ = "سیاه"	۱۳
پایان	۱۴
پایان	۱۵

می‌بینید که عنصرها می‌توانند تودرتو باشند: سرتیتر درون سرصفحه و آن درون صفحه. این درست مانند ساختار یک صفحه وب واقعی است که بخش‌هایی درون بخش‌های دیگر دارد.

یادداشت

ویژگی‌های زبان و جهت به‌ویژه برای فارسی مهم‌اند. با چپ به "راست" = جهت کل صفحه راست‌به‌چپ چیده می‌شود — همان‌طور که متن فارسی باید باشد. این یکی از جاهایی است که پشتیبانی بومی سلام از فارسی می‌درخشد.

۳.۱۹ عنصرهای پرکاربرد

سلام عنصرهای فراوانی برای ساختن صفحه دارد. مهم‌ترین‌ها:

کاربرد	معادل HTML	فارسی
ریشه صفحه	html/body	صفحه
بخش بالای صفحه	header	سرصفحه
بخش پایین صفحه	footer	پاصفحه
تیتر	h1..h6	سرتیتر
یک بخش کلی	div	جعبه
پاراگراف	p	بند
فهرست	ul	فهرست
یک عضو فهرست	li	مورد
دکمه	button	دکمه
فرم ورودی	form	فرم
جعبه ورودی	input	ورودی
پیوند	a	پیوند
تصویر	img	تصویر

۱.۳.۱۹ فهرست

این مثال واقعی یک فهرست از میوه‌ها می‌سازد:

صفحه:	۱
فهرست:	۲
مورد: محتوا = "سیب" پایان	۳
مورد: محتوا = "پرتقال" پایان	۴
مورد: محتوا = "موز" پایان	۵
پایان	۶
پایان	۷

۲.۳.۱۹ دکمه

صفحه:	۱
دکمه: نوع = "دکمه" محتوا = "کلیک کنید" پس زمینه = "آبی" رنگ = "سفید" پایان	۲
پایان	۳

۴.۱۹ ساختن یک فرم

فرم‌ها به کاربر اجازه می‌دهند داده وارد کند. این مثال واقعی یک فرم ورود با جعبه گذرواژه می‌سازد:

صفحه:	۱
فرم:	۲
ورودی: نوع = "گذرواژه" نام = "pass" جانگهدار = "رمز عبور" پایان	۳
دکمه: نوع = "ارسال" محتوا = "ورود" پایان	۴
پایان	۵
پایان	۶

عنصر ورودی ویژگی‌های کلیدی دارد:

- نوع: گونه ورودی — "متن"، "گذرواژه"، "ایمیل"، "ارسال" و مانند این‌ها.
- نام: نامی که داده با آن فرستاده می‌شود.
- جانگهدار: متن راهنمایی که پیش از تایپ کاربر نشان داده می‌شود.

نکته

نوع "گذرواژه" به مرورگر می‌گوید که نویسه‌های تایپ‌شده را به‌جای متن، با نقطه نشان دهد — تا کسی از روی شانه کاربر رمز را نبیند. این یک نکته کوچک اما مهم امنیتی و تجربه کاربری است.

۵.۱۹ سبک‌دهی و ظاهر

ویژگی‌هایی مانند رنگ، پس‌زمینه، فاصله داخلی و اندازه قلم ظاهر عنصرها را تعیین می‌کنند. سلام این‌ها را به CSS استاندارد ترجمه می‌کند. مقدارهای رنگ می‌توانند نام‌های فارسی ("آبی"، "سفید"، "سیاه") یا کدهای استاندارد باشند.

نگاهی به درون: از سلام تا HTML (اختیاری، برای خوانندگان پیشرفته)

وقتی salam layout build را اجرا می‌کنید، سلام درخت عنصرهای شما را می‌پیماید و برای هر کدام، HTML متناظر را تولید می‌کند؛ ویژگی‌های ظاهری را به کلاس‌های CSS تبدیل می‌کند (با نام‌هایی که به‌طور خودکار از روی محتوا ساخته می‌شوند تا یکتا باشند)؛ و رفتارها را در یک فایل JavaScript می‌گذارد. نتیجه یک صفحه وب کاملاً استاندارد است که در هر مرورگری کار می‌کند — اما شما آن را به فارسی و با واژگان آشنا نوشتید.

۶.۱۹ خلاصه فصل

- برنامه‌های چیدمان با صفحه آغاز می‌شوند، نه اصلی تابع.
- صفحه از عنصرهای تودرتو ساخته می‌شود؛ هر عنصر ویژگی‌هایی دارد.
- با salam layout build صفحه به HTML/CSS/JS تبدیل می‌شود.
- ویژگی‌های زبان و جهت پشتیبانی بومی از فارسی راست‌به‌چپ می‌دهند.
- عنصرهای فراوانی هست: جعبه، فهرست، دکمه، فرم، ورودی و بیشتر.
- ویژگی‌های ظاهری به CSS ترجمه می‌شوند.

۷.۱۹ تمرین‌ها

تمرین ۱۰.۱۹. یک صفحه وب بسازید که نام و یک پیام خوشامد را نشان دهد، با عنوان و جهت راست به چپ.

تمرین ۲۰.۱۹. صفحه‌ای با یک فهرست سه‌موردی از کارهای روزانه‌تان بسازید.

تمرین ۳۰.۱۹. یک فرم ثبت‌نام با ورودی‌های نام، ایمیل و گذرواژه و یک دکمه ارسال طراحی کنید.

تمرین ۴۰.۱۹. صفحه‌ای با یک سرصفحه رنگی و یک جعبه محتوا بسازید و آن را با inline- بسازید.

تمرین ۵۰.۱۹. یک کارت معرفی بسازید: جعبه‌ای با یک سرتیتر (نام)، یک بند (توضیح) و یک دکمه (تماس).

فصل ۲۰

نکات پیشرفته و بهترین روش‌ها

به فصل پایانی بخش اصلی کتاب رسیدیم. تا اینجا ابزارهای زبان را آموختید؛ اکنون وقت آن است که یاد بگیریم چگونه از آن‌ها خوب استفاده کنیم: کدی که سریع، ایمن، خوانا و قابل‌اعتماد باشد. این فصل، صندوقچهٔ ابزار یک برنامه‌نویس حرفه‌ای است.

۱.۲۰ پاس دادن به ارجاع

در فصل ۶ دیدیم که پارامترها معمولاً یک رونوشت از مقدار می‌گیرند؛ پس تغییر آن‌ها درون تابع، اصل متغیر بیرون را عوض نمی‌کند. اما گاهی می‌خواهیم تابع خود متغیر را تغییر دهد. برای این از ارجاع (با نشانه &) استفاده می‌کنیم. این مثال واقعی را ببینید:

۱	تابع افزایش (ایکس & صحیح ۳۲):
۲	ایکس = ایکس + 1
۳	پایان
۴	تابع جابجایی (الف & صحیح ۳۲, ب & صحیح ۳۲):
۵	متغیر موقت صحیح ۳۲ = الف
۶	الف = ب
۷	ب = موقت
۸	پایان
۹	تابع اصلی:
۱۰	متغیر ن صحیح ۳۲ = 10
۱۱	افزایش (ن)
۱۲	افزایش (ن)
۱۳	چاپ ("ن", " = ن)
۱۴	متغیر پ صحیح ۳۲ = 1
۱۵	متغیر ک صحیح ۳۲ = 2
۱۶	جابجایی (پ, ک)
۱۷	چاپ ("پ", " = پ", "ک", " = ک)
۱۸	پایان

پارامتر & صحیح ۳۲ ایکس یک / ارجاع به متغیر اصلی است، نه رونوشت آن. پس وقتی افزایش (ن) را دو بار صدا می‌زنیم، ن واقعاً به ۱۲ می‌رسد. به همین شکل، جابجایی می‌تواند جای دو متغیر بیرونی را عوض کند — کاری که بدون ارجاع ناممکن بود.

یادداشت

درون بدنه تابع، با ارجاع درست مانند یک متغیر معمولی کار می‌کنید — ایکس = ایکس + 1. نیازی به نشانه ویژه نیست. تنها در /مضای تابع است که & می‌گوید این پارامتر یک ارجاع است.

۲۰۲۰ قالب‌بندی خودکار کد

کد مرتب و یکدست، خواندن و نگهداری را آسان می‌کند. سلام یک قالب‌بند (formatter) داخلی دارد که کد را به‌طور خودکار و طبق سبک استاندارد زبان مرتب می‌کند:

۱	salam fmt app.salam	# فایل یک مرتب‌کردن
۲	salam fmt	# پروژه فایل‌های همه مرتب‌کردن
۳	salam fmt --check	# نامرتب‌اند فایل‌ها کدام بده گزارش فقط

نکته

بحث بر سر «چند فاصله تورفتگی» یا «پرانتز کجا باشد» اتلاف وقت است. بگذارید salam fmt این تصمیم‌ها را بگیرد. تیم‌های حرفه‌ای معمولاً قالب‌بند را طوری تنظیم می‌کنند که پیش از هر ثبت (commit) خودکار اجرا شود؛ این‌گونه کلِ کدِ پروژه همیشه یکدست می‌ماند.

۳.۲۰ آزمون نویسی

چگونه مطمئن شویم که برنامه‌مان درست کار می‌کند — و پس از هر تغییر، همچنان درست می‌ماند؟ پاسخ، آزمون است. ماژول testing ابزارهای آزمون را فراهم می‌کند:

۱	واردکردن "testing"
۲	تابع جمع (الف صحیح ۳۲، ب صحیح ۳۲) صحیح ۳۲: بازگشت الف + ب پایان
۳	تابع اصلی:
۴	جمع, testing.AssertEqInt(2) 3), 5)
۵	جمع, testing.AssertEqInt(1) 1), 0)
۶	جمع, testing.AssertEqInt(0) 0), 0)
۷	testing.Summary()
۸	پایان

تابع موردانتظار، واقعی) AssertEqInt بررسی می‌کند که نتیجه تابع همان چیزی است که انتظار داشتیم. اگر همه درست باشند، آزمون‌ها سبز می‌شوند؛ اگر یکی شکست بخورد، دقیقاً می‌گوید کجا. Summary() در پایان خلاصه‌ای از نتیجه نشان می‌دهد.

هشدار

عادتِ مهلک: «بعداً آزمون می‌نویسم.» در عمل، «بعداً» هرگز نمی‌آید. آزمون‌ها را همراهِ کد بنویسید، نه پس از آن. هر تابعِ مهم باید دست‌کم چند آزمون داشته باشد — به‌ویژه برای حالت‌های مرزی (صفر، عددِ منفی، رشتهٔ خالی) که بیشترین خطاها در آن‌ها رخ می‌دهد.

۴.۲۰ اشکال‌زدایی

وقتی برنامه آن‌طور که می‌خواهید کار نمی‌کند، به ابزار **اشکال‌زدا** (debugger) نیاز دارید که اجازه دهد گام‌به‌گام برنامه را پیش ببرید و مقدارِ متغیرها را ببینید. سلام با گزینهٔ **g**- اطلاعاتِ اشکال‌زدایی تولید می‌کند:

```
۱ salam debug app.salam # اجرای و اشکال‌زدایی اطلاعات با ساخت
    اشکال‌زدا
```

افزون بر این، برای یافتنِ نشت‌های حافظه، فرمان **memcheck** را دارید:

```
۱ salam memcheck app.salam # حافظه و ارسای ابزار زیر برنامه اجرای
```

نکته

ساده‌ترین ابزارِ اشکال‌زدایی همان چاپ است! وقتی گیر کردید، چند چاپ در نقاطِ کلیدی بگذارید تا ببینید مقدارها چه هستند و برنامه به کجا می‌رسد. این روشِ ساده، اغلب سریع‌ترین راهِ یافتنِ خطاست. اما برای مشکلاتِ پیچیده، اشکال‌زدای واقعی ارزشش را دارد.

۵.۲۰ کارایی و حافظه

سلام به سرعتِ C نزدیک است، چون برنامهٔ شما در نهایت به کدِ ماشین تبدیل می‌شود. اما چند نکته به برنامه‌های سریع‌تر و امن‌تر می‌انجامد:

- **نوع درست را برگزینید:** برای شماره‌های کوچک صحیح ۳۲ کافی است؛ صحیح ۶۴ را فقط وقتی لازم است به کار ببرید.
- **منابع را آزاد کنید:** بردارها و نگاشت‌هایی که عمرشان تمام شده را با `free()` آزاد کنید. (به یاد دارید که تعلیق بهترین جا برای این کار است؟)
- **از بازگشت بیش از اندازه عمیق بپرهیزید:** هر فراخوانی بازگشتی فضای پشته می‌گیرد. برای عمق زیاد، گاهی حلقه بهتر از بازگشت است.
- **در حلقه‌های داغ، کار تکراری را بیرون بکشید:** اگر چیزی در هر تکرار حلقه ثابت است، آن را پیش از حلقه حساب کنید.

نگاهی به درون: امنیت حافظه (اختیاری، برای خوانندگان پیشرفته)

بسیاری از خطاهای جدی و آسیب‌پذیری‌های امنیتی از سوءاستفاده حافظه می‌آیند: دسترسی بیرون از مرز آرایه، استفاده از حافظه‌ای که آزاد شده، یا فراموشی آزادسازی (نشت حافظه). سلام چند سپر در برابر این‌ها دارد: گزینه `--safe` دسترسی آرایه را واری می‌کند، `memcheck` نشت‌ها را می‌یابد، و تعلیق آزادسازی را تضمین می‌کند. در حین توسعه از این ابزارها بهره بگیرید تا خطاها را زود — پیش از رسیدن به دست کاربر — بگیرید.

۶.۲۰ اصول کد خوب

در پایان، چند اصل کلی که برنامه‌نویسان باتجربه به آن‌ها پایبندند:

۱. **نام‌های گویا:** مجموع‌نمرات بهتر از `m` است. کد را برای *انسان* بنویسید.
۲. **توابع کوچک:** هر تابع یک کار روشن انجام دهد. اگر تابعی خیلی بزرگ شد، احتمالاً باید آن را بشکنید.
۳. **از تکرار بپرهیزید:** اگر کدی را دو بار نوشتید، آن را به یک تابع تبدیل کنید.
۴. **تغییرناپذیری را پیش‌فرض بگیرید:** متغیرها را تا وقتی واقعاً نیاز به تغییر نیست، تغییرناپذیر بگذارید.

۵. خطاها را آگاهانه مدیریت کنید: هیچ حالت خطایی را نادیده نگیرید.
۶. آزمون بنویسید: کد بدون آزمون، کد مشکوک است.
۷. توضیح «چرا»، نه «چه»: توضیحات باید بگویند چرا کاری انجام می‌شود، نه چه کاری — چون «چه» را خود کد می‌گوید.

نکته

بهترین کد، کدی نیست که هوشمندانه‌ترین باشد، بلکه کدی است که ساده‌ترین و خوانا‌ترین باشد. کسی که شش ماه بعد کد شما را می‌خواند — که اغلب خود شما هستید! — باید بتواند آن را به آسانی بفهمد. سادگی، بالاترین هنر برنامه‌نویسی است.

۷۰۲۰. راه پیش‌رو

این کتاب پایان یافت، اما سفر شما تازه آغاز شده است. برنامه‌نویسی مهارتی است که تنها با تمرین به دست می‌آید. پیشنهادهایی برای ادامه:

- پروژه‌های کوچک خود را بسازید: یک ماشین حساب، یک بازی حدس عدد، یک دفترچه یادداشت.
- کد مثال‌های پوشه examples سلام را بخوانید و تغییر دهید.
- هر مفهومی را که یاد گرفتید، با نوشتن یک برنامه کوچک بیازمایید.
- از خطاها نترسید؛ هر خطا یک فرصت یادگیری است.

۸۰۲۰. خلاصه فصل

- با ارجاع (&) می‌توان متغیر بیرون را از درون تابع تغییر داد.
- `salam fmt` کد را خودکار مرتب می‌کند.
- `testing` برای آزمون‌نویسی است.

- `memcheck` و `salam debug` برای اشکال‌زدایی و یافتن نشت حافظه‌اند.
- نوع درست، آزادسازی منابع و پرهیز از تکرار، کلیدهای کارایی‌اند.
- کد خوب، کد ساده و خواناست.

۹.۲۰ تمرین‌ها

- تمرین ۱.۲۰.** تابع جابجایی را برای دو رشته بازنویسی کنید (با ارجاع).
- تمرین ۲.۲۰.** چند آزمون برای تابع فاکتوریل فصل ۶ بنویسید، از جمله برای حالت‌های مرزی.
- تمرین ۳.۲۰.** یک قطعه کد نامرتب بنویسید، آن را با `salam fmt` مرتب کنید و تفاوت را ببینید.
- تمرین ۴.۲۰.** یکی از برنامه‌های فصل‌های پیش را بردارید و آن را با نام‌های گویاتر و توابع کوچک‌تر بازنویسی کنید.
- تمرین ۵.۲۰.** یک پروژه کوچک کامل طراحی کنید (مثلاً یک دفتر تلفن با نگاشت) که از دست‌کم پنج مفهوم این کتاب استفاده کند، و برایش آزمون بنویسید.

پیوست آ

فهرست کامل واژگان کلیدی

این پیوست مرجع کامل معادل‌های فارسی و انگلیسی زبان سلام است. هر واژه را می‌توان به هر دو زبان نوشت و هر دو به یک برنامه یکسان تبدیل می‌شوند.

۱.آ واژگان کلیدی

معنا	انگلیسی	فارسی
تعریف تابع	func	تابع
نام تابع آغازین	main	اصلی
بازگرداندن مقدار	ret	بازگشت
شرط	if	اگر
شاخه جایگزین	else	وگرنه
شرط زنجیره‌ای	else if	اگر وگرنه
حلقه شرطی	while	تاوقتی
حلقه شمارنده‌دار	for	برای
حلقه تکرار	repeat	تکرار
کران بالای بازه	to	تا
گام بازه	every	هر
خروج از حلقه	break	بشکن
پرش به تکرار بعد	continue	ادامه
پایان بلوک	end	پایان

فارسی	انگلیسی	معنا
متغیر	mut	متغیر تغییرپذیر
ثابت	const	مقدار تغییرناپذیر
نوع	type	تعریف نام نوع
خودکار	auto	استنتاج نوع
ساختار	struct	ساختار
شمارش	enum	شمارشی
واسط	interface	واسط
این	this	نمونه جاری
عملگر	operator	سربارگذاری عملگر
تعلیق	defer	اجرای به تعویق افتاده
وارد کردن	import	وارد کردن ماژول
بسته	package	تعریف بسته
عمومی	pub	دسترسی عمومی
خارجی	extern	تابع خارجی (FFI)
بعنوان	as	تبدیل نوع
درست	true	مقدار درست
نادرست	false	مقدار نادرست
پوچ	null	مقدار تهی
چاپ	println	چاپ با خط تازه
بنویس	print	چاپ بدون خط تازه
بخوان	input	خواندن ورودی
صفحه	layout	صفحه چیدمان
مولفه	component	مولفه چیدمان

یادداشت

_ (یک زیرخط) شکل کوتاه بنویس/print و __ (دو زیرخط) شکل کوتاه چاپ/println است. این میان‌برها در نمونه‌های کوتاه به کار می‌روند.

۲.آ انواع داده

فارسی	انگلیسی	توضیح
صحیح / صحیح ۳۲	i32 / int	عدد صحیح ۳۲ بیتی
صحیح ۶۴	i64	عدد صحیح ۶۴ بیتی
صحیح ۸	i8	عدد صحیح ۸ بیتی
صحیح ۱۶	i16	عدد صحیح ۱۶ بیتی
—	u8 .. u64	اعداد صحیح بی‌علامت
اعشاری ۶۴	f64	عدد اعشاری ۶۴ بیتی
اعشاری ۳۲	f32	عدد اعشاری ۳۲ بیتی
منطقی	bool	درست/نادرست
کاراکتر	char	یک نویسه
رشته	str	متن
خودکار	auto	استنتاج خودکار
—	void	بی‌مقدار (فقط خروجی)
وکتور <T>	Vector<T>	بردار رشدپذیر
نگاشت <K,V>	HashMap<K,V>	نگاشت کلید-مقدار

۳.آ ماژول‌های کتابخانه استاندارد

وارد کردن	نام فارسی	کاربرد
"math"	ریاضی	توابع ریاضی
"str"	رشته	پردازش متن
"io"	—	ورودی/خروجی و فایل
"os"	—	سیستم‌عامل، آرگومان‌ها، محیط
"time"	—	زمان و تاریخ
"rand"	—	عددهای تصادفی

وارد کردن	نام فارسی	کاربرد
"fmt"	—	قالب‌بندی متن
"json"	—	دادهٔ JSON
"sync"	—	نخ‌ها و قفل‌ها
"http"	—	درخواست‌های وب
"net"	—	شبکه
"testing"	—	آزمون‌نویسی
"mem"	—	مدیریت حافظه
"option"	—	مقدارهای اختیاری
"encoding"	—	رمزگذاری، Base۶۴ (Hex)
"regex"	—	عبارت‌های باقاعده
"db/sqlite"	—	پایگاه‌دادهٔ SQLite
"db/mysql"	—	پایگاه‌دادهٔ MySQL

۴.آ فرمان‌های کامپایلر

فرمان	کار
salam exec app.salam	اجرای بی‌درنگ با مفسر
salam build app.salam --output=app.exe	ساخت فایل اجرایی
salam build app.salam --safe	ساخت با واریسی مرز آرایه
salam build app.salam --keep-c	نگه‌داشتن کد C تولیدشده
salam obj app.salam	ساخت فایل شیء (۰.۰)
salam layout build page.salam	ساخت صفحهٔ وب (HTML/CSS/JS)
salam layout build page.salam --inline	صفحهٔ یکپارچه
salam fmt app.salam	قالب‌بندی کد
salam fmt --check	گزارش فایل‌های نامرتب
salam debug app.salam	ساخت و اجرای اشکال‌زدا
salam memcheck app.salam	واریسی نشت حافظه
salam app.salam --emit-tokens-xml	نمایش نشانه‌ها

کار	فرمان
نمایش درخت نحوی	salam app.salam --emit-ast-xml
تعریف پرچم پیش‌پردازنده	salam -DFLAG build app.salam

۵.آ راهنماهای فایل

معنا	راهنما
زبانِ فایل فارسی است (fa یا en)	// LANG: fa
نام انگلیسیِ یک تعریف	@en "..."
نام فارسیِ یک تعریف	@fa "..."
پیش‌پردازنده شرطی	@if / @else / @end
پیش‌پردازنده بر پایه وجود پرچم	@ifdef / @ifndef

پیوست ب

دستور زبان (گرامر)

این پیوست ساختارهای نحوی اصلی زبان سلام را به صورت فشرده گرد آورده است. هدف، مرجعی سریع برای یادآوری شکل درست هر ساختار است، نه تعریف صوری کامل.

ب.۱ ساختار کلی برنامه

یک برنامه اجرایی از چند تعریف (تابع، ساختار، شمارشی، ...) ساخته می شود و حتماً یک تابع اصلی دارد:

۱	// تعریف های دلخواه (توابع، ساختارها، ...)
۲	تابع اصلی:
۳	// بدنه برنامه
۴	پایان

ب.۲ اعلان متغیر

۱	متغیر نام نوع = مقدار // تغییرپذیر
۲	ثابت نام نوع = مقدار // تغییرناپذیر
۳	نام نوع = مقدار // تغییرناپذیر (شکل ساده)
۴	نام خودکار = مقدار // استنتاج نوع

ب.۳. تابع

۱	تابع نام (پارامتر ۱ نوع، پارامتر ۲ نوع) نوع خروجی:
۲	// بدنه
۳	بازگشت مقدار
۴	پایان
۵	
۶	تابع نام (پارامتر نوع): // بدون مقدار بازگشتی
۷	// بدنه
۸	پایان

پارامتر ارجاعی با & مشخص می‌شود:

۱	تابع نام (پارامتر & نوع):
۲	// تغییر پارامتر، متغیر بیرون را عوض می‌کند
۳	پایان

ب.۴. شرط

۱	اگر شرط:
۲	// ...
۳	وگرنه اگر شرط دیگر:
۴	// ...
۵	وگرنه:
۶	// ...
۷	پایان

ب.۵. حلقه‌ها

۱	تاوقتی شرط:
۲	... //
۳	پایان
۴	
۵	برای متغیر i = آغاز; شرط; گام:
۶	... //
۷	پایان
۸	
۹	تکرار N: // N بار
۱۰	تکرار آ تا ب: // از آ تا ب شامل
۱۱	تکرار آ تا ب هر گ: // با گام گ

کنترل حلقه: بشکن (خروج) و ادامه (پرش به تکرار بعد).

ب.۶ ساختار و شمارشی

۱	ساختار نام:
۲	میدان نوع = پیش فرض
۳	تابع متد (نوع خروجی):
۴	بازگشت این.میدان
۵	پایان
۶	پایان
۷	
۸	// ساختن نمونه:
۹	نام { میدان = مقدار }
۱۰	
۱۱	شمارش نام:
۱۲	عضو ۱
۱۳	عضو ۲ = 5
۱۴	عضو ۳
۱۵	پایان
۱۶	// دسترسی: نام.عضو و نام.عضو بعنوان صحیح ۳۲

ب.۷ واسط و چندریختی

۱	واسط نام:
۲	تابع متد (نوع خروجی) // فقط امضا
۳	پایان
۴	
۵	تابع کاربر (پارامتر dyn نام):
۶	پارامتر.متد()
۷	پایان

ب.۸ کلی سازی

۱	تابع نام <ت> (پارامتر ت) ت:
۲	// ...
۳	پایان
۴	
۵	ساختار نام <آ, ب>:
۶	میدان ۱ آ
۷	میدان ۲ ب
۸	پایان
۹	// کاربرد: نام <صحیح ۳۲> یا نام <رشته, صحیح ۳۲>

ب.۹ لامبدا و بستار

۱	// نوع تابع:
۲	(نوع ورودی) func نوع خروجی
۳	
۴	// لامبدای یک عبارتی:
۵	(پارامتر نوع) => عبارت
۶	
۷	// لامبدای چنددستوری:
۸	() => :
۹	// دستورها
۱۰	بازگشت مقدار
۱۱	پایان

ب. ۱۰. ماژول و دسترسی

۱	بسته نام	// در فایل کتابخانه
۲	عمومی تابع ...	// عضو عمومی
۳	عمومی ثابت ...	
۴		
۵	واردکردن "نام ماژول"	// واردکردن ساده
۶	واردکردن مستعار "فایل.salam"	// با نام مستعار

ب. ۱۱. اجرای به تعویق افتاده و خارجی

۱	تعليق دستور	// در پایان تابع اجرا می شود
۲		
۳	خارجی: "C"	
۴	تابع نام خارجی (پارامتر نوع) نوع خروجی	
۵	پایان	
۶		
۷	پیوند بویا "نام کتابخانه"	// پیوند به کتابخانه خارجی

ب.۱۲. چیدمان (وب)

صفحه:	۱
ویژگی = مقدار	۲
عنصر:	۳
ویژگی = مقدار	۴
عنصر تودرتو: ویژگی = مقدار پایان	۵
پایان	۶
پایان	۷

ب.۱۳. قاعده‌های کلی

- بلوک‌ها با : باز و با پایان بسته می‌شوند.
- سازنده‌های مقدار (ساختار، آرایه) از {} و [] استفاده می‌کنند، نه پایان.
- توضیحات تک‌خطی با // آغاز می‌شوند.
- رشته‌ها میان " و کاراکترها میان ' می‌آیند.
- ارقام محاسباتی همیشه لاتین‌اند (2، نه ۲).
- اندیس آرایه‌ها و بردارها از صفر آغاز می‌شود.

پیوست ج

برگه مرجع سریع

این پیوست رایج‌ترین الگوهای زبان سلام را در یک نگاه گرد آورده است — برای وقتی که فقط می‌خواهید شکل درست چیزی را سریع به یاد بیاورید.

ج ۱ اسکت برنامه

۱	تابع اصلی:
۲	چاپ "سلام، دنیا!"
۳	پایان

ج ۲ متغیر و چاپ

۱	متغیر شمار صحیح ۳۲ = 0 // تغییرپذیر
۲	نام رشته = "سارا" // تغییرناپذیر
۳	سن خودکار = 30 // استنتاج
۴	چاپ نام، سن // چاپ با خط تازه
۵	بنویس "بدون خط تازه"

ج.۳. عملگرها

۱	+ - * / % ** // حسابی
۲	== != < > <= >= // مقایسه
۳	&& ! // منطقی
۴	الف + " " + ب // پیوند رشته

ج.۴. شرط

۱	اگر سن >= 18:
۲	چاپ "بزرگسال"
۳	وگرنه اگر سن >= 13:
۴	چاپ "نوجوان"
۵	وگرنه:
۶	چاپ "کودک"
۷	پایان

ج.۵. حلقه‌ها

۱	تاوقتی شرط: ... پایان
۲	برای متغیر i = 0; < 10; + i = 1: ... پایان
۳	تکرار 5: ... پایان
۴	تکرار 1 تا 10: ... پایان
۵	تکرار 0 تا 100 هر 10: ... پایان
۶	// بشکن / ادامه

ج.۶. تابع

۱	تابع جمع (الف صحیح ۳۲، ب صحیح ۳۲) صحیح ۳۲:
۲	بازگشت الف + ب
۳	پایان

ج.۷ آرایه و بردار

۱	اعداد صحیح ۳۲ [3] 2, [1, =
۲	چاپ اعداد [0], len(اعداد)
۳	
۴	متغیر ب وکتور <صحیح ۳۲> = وکتور {}
۵	ب. بیفزای (10) // افزودن
۶	ب. بگیر (0) [0] // خواندن
۷	ب. طول () // طول
۸	ب. در بیاور () // برداشتن آخر

ج.۸ رشته

۱	س. طول () // طول
۲	س. زیر رشته (0), 3 // برش
۳	س. پیوست (!) // چسباندن
۴	س. بشکاف (", ") // شکستن به بردار
۵	"42" به صحیح () // به عدد

ج.۹ نگاشت

۱	متغیر م نگاشت <رشته، صحیح ۳۲> = نگاشت {}
۲	م. درج ("الف", 1) // افزودن
۳	م. بگیر ("الف") // خواندن
۴	م. دارد ("الف") // وجود؟
۵	م. اندازه () // تعداد

ج. ۱۰ ساختار و شمارشی

۱	ساختار نقطه:
۲	x صحیح ۳۲ = 0
۳	y صحیح ۳۲ = 0
۴	تابع جمع () صحیح ۳۲: بازگشت این x + این y پایان
۵	پایان
۶	متغیر p نقطه = نقطه $\{ x = 3, y = 4 \}$
۷	p جمع ()
۸	
۹	شمارش رنگ: قرمز، سبز، آبی پایان
۱۰	رنگ. قرمز بعنوان صحیح ۳۲

ج. ۱۱ واسط و کلی سازی

۱	واسط شکل:
۲	تابع مساحت () صحیح ۳۲
۳	پایان
۴	تابع توصیف (ش dyn شکل): چاپ ش. مساحت () پایان
۵	
۶	تابع بیشینه <ت> (الف ت, ب ت) ت:
۷	اگر الف > ب: بازگشت الف پایان
۸	بازگشت ب
۹	پایان

ج. ۱۲ لامبدا و تعلیق

۱	دوبرابر (صحیح ۳۲) func(صحیح ۳۲) = (ن صحیح ۳۲) => $2 * n$
۲	دوبرابر (21)
۳	
۴	تعلیق چاپ "پاکسازی" // در پایان تابع اجرا می شود

ج.۱۳. ماژول و خارجی

۱	واردکردن "ریاضی"
۲	ریاضی. جذر(0.144)
۳	
۴	خارجی "C":
۵	تابع printf(format رشته, ...) صحیح ۳۲
۶	پایان

ج.۱۴. فرمان‌های پرکاربرد

```
۱ salam exec app.salam
۲ salam build app.salam --output=app.exe
۳ salam layout build page.salam
۴ salam fmt app.salam
```


پیوست د

پاسخ تمرین‌ها

در این پیوست پاسخ تمرین‌های منتخب هر فصل آمده است. به یاد داشته باشید که در برنامه‌نویسی اغلب چند راه درست وجود دارد؛ اگر پاسخ شما با اینجا فرق دارد اما درست کار می‌کند، آن هم پاسخی معتبر است. پیش از دیدن پاسخ، خودتان بکوشید.

فصل ۱ – شروع کار

تمرین ۱.۱ (نام و سن):

۱	تابع اصلی:
۲	چاپ "نام: سارا"
۳	چاپ "سن: 12"
۴	پایان

تمرین ۲.۱ (8 * 12):

۱	تابع اصلی:
۲	چاپ 12 * 8 =, 8 * 12
۳	پایان

تمرین ۴.۱ (نام و نام خانوادگی در یک خط):

۱	تابع اصلی:
۲	بنویس "سارا"
۳	بنویس "
۴	چاپ "احمدی"
۵	پایان

فصل ۲ – متغیرها و انواع

تمرین ۱۰۲ (شهر، جمعیت، دما):

۱	تابع اصلی:
۲	شهر رشته = "اصفهان"
۳	جمعیت صحیح $64 = 2200000$
۴	دما اعشاری $64 = 5.16$
۵	چاپ (شهر، جمعیت، دما)
۶	پایان

تمرین ۴۰۲ (تبدیل نوع):

۱	تابع اصلی:
۲	عدد صحیح $32 = 9$
۳	اعشاری اعشاری $64 =$ عدد بعنوان اعشاری 64
۴	چاپ (عدد، اعشاری)
۵	پایان

فصل ۳ – عملگرها

تمرین ۱۰۳ (محیط و مساحت مستطیل):

۱	تابع اصلی:
۲	طول صحیح $32 = 8$
۳	عرض صحیح $32 = 5$
۴	چاپ "مساحت:", طول * عرض
۵	چاپ "محیط:", $2 * (طول + عرض)$
۶	پایان

تمرین ۲.۳ (بخش‌پذیری بر ۳):

۱	تابع اصلی:
۲	اگر $27 \% 3 == 0$:
۳	چاپ "بخش‌پذیر است"
۴	وگرنه:
۵	چاپ "بخش‌پذیر نیست"
۶	پایان
۷	پایان

فصل ۴ – شرط‌ها

تمرین ۱.۴ (مثبت، منفی، صفر):

۱	تابع علامت (ن صحیح ۳۲) رشته:
۲	اگر $n > 0$: بازگشت "مثبت" پایان
۳	اگر $n < 0$: بازگشت "منفی" پایان
۴	بازگشت "صفر"
۵	پایان
۶	تابع اصلی:
۷	چاپ علامت (5), علامت (3-), علامت (0)
۸	پایان

تمرین ۴.۴ (زوج یا فرد):

۱	تابع زوج_است (ن صحیح ۳۲) منطقی:
۲	بازگشت $n \% 2 == 0$
۳	پایان
۴	تابع اصلی:
۵	چاپ زوج_است (4), زوج_است (7)
۶	پایان

فصل ۵ – حلقه‌ها

تمرین ۲.۵ (جمع زوج‌های ۱ تا ۱۰۰):

۱	تابع اصلی:
۲	متغیر مجموع صحیح $0 = 32$
۳	متغیر i صحیح $2 = 32$
۴	تاوقتی $100: \leq i$
۵	مجموع = مجموع $i +$
۶	$2 + i = i$
۷	پایان
۸	چاپ مجموع
۹	پایان

تمرین ۵.۵ (جدول ضرب ۱ تا ۵):

۱	تابع اصلی:
۲	برای متغیر $i = 1; \leq i 5; + i = 1$
۳	متغیر خط رشته $''' =$
۴	برای متغیر $j = 1; \leq j 5; + j = 1$ خط = خط $(i +$ خط $) * (j +$ " پایان
۵	چاپ خط
۶	پایان
۷	پایان

فصل ۶ – توابع

تمرین ۳.۶ (فیوناچی):

۱	تابع فیوناچی (ن صحیح 32) صحیح 32 :
۲	اگر $n < 2$ بازگشت ن پایان
۳	بازگشت فیوناچی (ن) $- 1$ + فیوناچی (ن) $- 2$
۴	پایان
۵	تابع اصلی:
۶	چاپ فیوناچی (10)
۷	پایان

تمرین ۵.۶ (ب.م.م به روش اقلیدس):

۱	تابع بمم (الف صحیح ۳۲, ب صحیح ۳۲) صحیح ۳۲:
۲	تاوقتی ب $!= 0$:
۳	متغیر موقت صحیح ۳۲ = ب
۴	ب = الف % ب
۵	الف = موقت
۶	پایان
۷	بازگشت الف
۸	پایان
۹	تابع اصلی:
۱۰	چاپ بمم (48, 36)
۱۱	پایان

فصل ۷ – آرایه‌ها و بردارها

تمرین ۱۰۷ (بزرگ‌ترین آرایه):

۱	تابع اصلی:
۲	اعداد صحیح ۳۲ $[6] = [12, 56, 23, 7, 45]$ 9
۳	متغیر بیشینه صحیح ۳۲ = اعداد[0]
۴	برای متغیر $i = 1; < i 6; + i = 1$:
۵	اگر اعداد[i] > بیشینه: بیشینه = اعداد[i] پایان
۶	پایان
۷	چاپ "بیشینه:", بیشینه
۸	پایان

تمرین ۳۰۷ (زوج‌های یک بردار):

تابع اصلی:	۱
متغیر ب وکتور <صحیح ۳۲> = وکتور {}	۲
برای متغیر $i = 1; <= i 10; + i = i 1$: ب. بیفزای (i) پایان	۳
متغیر z صحیح ۳۲ = 0	۴
تاوقتی z < ب. طول():	۵
اگر ب. بگیر (i)[0] % 2 == 0: چاپ ب. بگیر (i)[0] پایان	۶
$1 + z = z$	۷
پایان	۸
پایان	۹

فصل ۸ - رشته‌ها

تمرین ۵.۸ (مقارن بودن):

تابع مقارن_است (س رشته) منطقی:	۱
متغیر چپ صحیح ۳۲ = 0	۲
متغیر راست صحیح ۳۲ = س. طول() - 1	۳
تاوقتی چپ < راست:	۴
اگر س. زیررشته (چپ, 1) != س. زیررشته (راست, 1):	۵
بازگشت نادرست	۶
پایان	۷
چپ = چپ + 1	۸
راست = راست - 1	۹
پایان	۱۰
بازگشت درست	۱۱
پایان	۱۲
تابع اصلی:	۱۳
چاپ مقارن_است ("radar"), مقارن_است ("salam")	۱۴
پایان	۱۵

فصل ۹ - ساختارها و شمارشی‌ها

تمرین ۱۰.۹ (دایره):

۱	ساختار دایره:
۲	شعاع اعشاری ۶۴ = 0.0
۳	تابع مساحت (اعشاری ۶۴): بازگشت 1416.3 * این.شعاع * این.شعاع پایان
۴	تابع محیط (اعشاری ۶۴): بازگشت 0.2 * 1416.3 * این.شعاع پایان
۵	پایان
۶	تابع اصلی:
۷	متغیر د دایره = دایره { شعاع = 0.5 }
۸	چاپ "مساحت": , د.مساحت ()
۹	چاپ "محیط": , د.محیط ()
۱۰	پایان

فصل ۱۰ – واسط‌ها

تمرین ۲۰۱۰ (جاندار):

۱	واسط جاندار:
۲	تابع صدا (رشته)
۳	پایان
۴	ساختار گربه:
۵	تابع صدا (رشته): بازگشت "میو" پایان
۶	پایان
۷	ساختار سگ:
۸	تابع صدا (رشته): بازگشت "واق" پایان
۹	پایان
۱۰	تابع بشنو (ج dyn جاندار): چاپ ج.صدا () پایان
۱۱	تابع اصلی:
۱۲	بشنو (گربه {})
۱۳	بشنو (سگ {})
۱۴	پایان

فصل ۱۱ – کلی‌سازی

تمرین ۱۰۱۱ (کمینه کلی):

تابع	کمینه > ت (الف ت, ب ت) ت:	۱
اگر الف < ب:	بازگشت الف پایان	۲
بازگشت ب		۳
پایان		۴
تابع اصلی:		۵
چاپ	کمینه (3, 9)	۶
چاپ	کمینه (5.2, 5.1)	۷
پایان		۸

فصل ۱۲ – بستارها و لامبدا

تمرین ۲۰۱۲ (سازنده ضرب):

تابع	سازنده ضرب (ضرب صحیح ۳۲) (صحیح ۳۲) func(صحیح ۳۲):	۱
بازگشت x	صحیح ۳۲ $x \Rightarrow * \text{ضرب}$	۲
پایان		۳
تابع اصلی:		۴
سه برابر	func(صحیح ۳۲) صحیح ۳۲ = سازنده ضرب (3)	۵
چاپ	سه برابر (7)	۶
پایان		۷

فصل ۱۳ – خطا و تعلیق

تمرین ۲۰۱۳ (ریشه دوم امن):

۱	واردکردن "math"
۲	تابع ریشه_امن (x اعشاری ۶۴, نتیجه & اعشاری ۶۴) منطقی:
۳	اگر $x < 0.0$: بازگشت نادرست پایان
۴	نتیجه = ریاضی. جذر (x)
۵	بازگشت درست
۶	پایان
۷	تابع اصلی:
۸	متغیر ر اعشاری ۶۴ = 0.0
۹	اگر ریشه_امن (0.16, r): چاپ r وگرنه: چاپ "خطا" پایان
۱۰	اگر ریشه_امن (-0.4, r): چاپ r وگرنه: چاپ "خطا" پایان
۱۱	پایان

فصل ۱۴ – مازول‌ها

تمرین ۱۰۱۴ (مازول هندسه). فایل هندسه.salam:

۱	بسته هندسه
۲	عمومی تابع مساحت_دایره (ش اعشاری ۶۴) اعشاری ۶۴: بازگشت 1416.3 * ش * ش پایان
۳	عمومی تابع محیط_دایره (ش اعشاری ۶۴) اعشاری ۶۴: بازگشت 1416.3 * 0.2 * ش پایان
	فایل اصلی:

۱	واردکردن ه "هندسه.salam"
۲	تابع اصلی:
۳	چاپ ه.مساحت_دایره (0.5)
۴	چاپ ه.محیط_دایره (0.5)
۵	پایان

فصل ۱۵ – کتابخانه استاندارد

تمرین ۱۰۱۵ (فیثاغورس):

```
۱ وارדکردن "math"
۲ تابع اصلی:
۳ a اعشاری ۶۴ = 0.3
۴ b اعشاری ۶۴ = 0.4
۵ چاپ "وتر:", ریاضی. جذر(a * a + b * b)
۶ پایان
```

تمرین ۲.۱۵ (نگاشتِ قیمت‌ها):

```
۱ تابع اصلی:
۲ متغیر قیمت نگاشت <رشته, صحیح ۳۲> = نگاشت {}
۳ قیمت.درج("سیب", 12000)
۴ قیمت.درج("موز", 18000)
۵ چاپ "قیمت سیب:", قیمت.بگیر("سیب")
۶ پایان
```

فصل ۱۶ – فایل و سیستم

تمرین ۱۰.۱۶ (نوشتن و خواندن فایل):

```
۱ package main
۲ import ("io")
۳ func main:
۴     io.WriteFile("note.txt", "سلام از سها")
۵     text str = io.ReadFile("note.txt")
۶     io.Println(text)
۷ end
```

فصل ۱۷ – هم‌روندی

تمرین ۳.۱۷ (چرا local؟): اگر هر نخ در هر گام مستقیم به sum اضافه می‌کرد، یا باید در هر گام قفل می‌گرفت (که هم‌روندی را بی‌اثر می‌کرد)، یا بدون قفل دچار رقابت می‌شد و نتیجه نادرست در می‌آمد. با جمع محلی، هر نخ آزادانه و بدون قفل کار می‌کند و تنها یک‌بار در پایان قفل می‌گیرد – هم سریع و هم درست.

فصل ۱۸ – ارتباط با کد خارجی

تمرین ۱۰.۱۸ sqrt از C):

```
۱ extern "C":  
۲     func printf(format str, ...) i32  
۳     func sqrt(x f64) f64  
۴ end  
۵ func main:  
۶     printf("sqrt(2) = %.4f\n", sqrt(2.0))  
۷     printf("sqrt(16) = %.4f\n", sqrt(16.0))  
۸ end
```

فصل ۱۹ – وب و چیدمان

تمرین ۲۰.۱۹ (فهرست کارها):

صفحه:	۱
عنوان = "کارهای امروز"	۲
جهت = "راست به چپ"	۳
فهرست:	۴
مورد: محتوا = "خواندن کتاب سلام" پایان	۵
مورد: محتوا = "نوشتن یک برنامه" پایان	۶
مورد: محتوا = "حل تمرین‌ها" پایان	۷
پایان	۸
پایان	۹

فصل ۲۰ – نکات پیشرفته

تمرین ۱۰.۲۰ (جابجایی دو رشته با ارجاع):

۱	تابع جابجایی (الف & رشته, ب & رشته):
۲	متغیر موقت رشته = الف
۳	الف = ب
۴	ب = موقت
۵	پایان
۶	تابع اصلی:
۷	متغیر x رشته = "اول"
۸	متغیر y رشته = "دوم"
۹	جابجایی (x, y)
۱۰	چاپ x, y
۱۱	پایان

به اینجا که رسیدید، دیگر یک برنامه‌نویس سلام هستید. راه یادگیری هرگز پایان نمی‌یابد — پس بنویسید، بیازمایید و لذت ببرید!